



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Session - 2.1

Microprocessor & Computer Architecture (μ pCA)



UE24CS251B

Introduction to Pipeline Processor

PROCESSOR

You're going
to love this

```
int main()  
{  
  a=a+b  
  c=a-b  
}
```

THE PROGRAMMER

I will execute one
by one

ADD R0 R0 R1
SUB R2, R0,R1



THE COMPUTER

PROCESSOR

You're going
to love this

```
int main()  
{  
  a=a+b  
  c=a-b  
}
```

THE PROGRAMMER

If I execute both at a
time, can I complete
soon?

ADD R0 R0 R1
SUB R2, R0,R1



THE COMPUTER

TECHNIQUE 1

Shifting One Brick takes 10 mins

Timer: 0

1

2

3



1

2

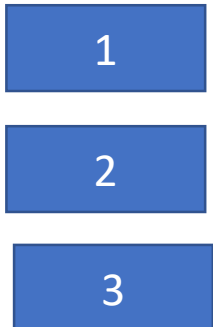
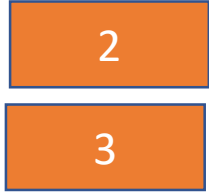
3



TECHNIQUE 1

Shifting One Brick takes 10 mins

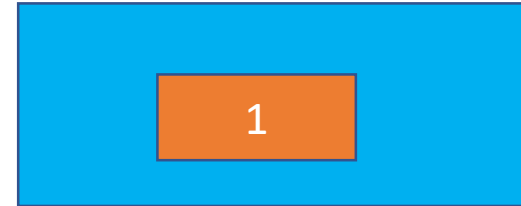
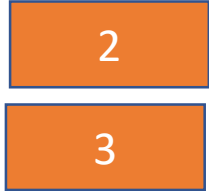
Timer: 5



TECHNIQUE 1

Shifting One Brick takes 10 mins

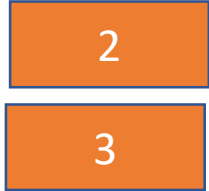
Timer: 10



TECHNIQUE 1

Shifting One Brick takes 10 mins

Timer: 10



TECHNIQUE 1

Shifting One Brick takes 10 mins

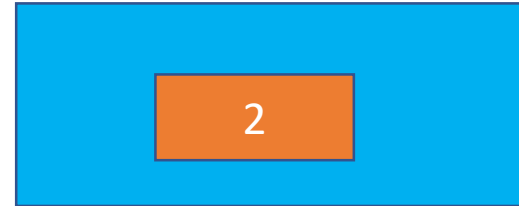
Timer: 15



TECHNIQUE 1

Shifting One Brick takes 10 mins

Timer: 20



TECHNIQUE 1

Shifting One Brick takes 10 mins

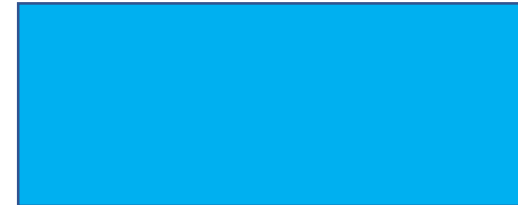
Timer: 20



TECHNIQUE 1

Shifting One Brick takes 10 mins

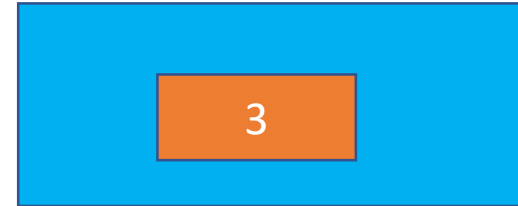
Timer: 25



TECHNIQUE 1

Shifting One Brick takes 10 mins

Timer: 30



TECHNIQUE 1

Shifting One Brick takes 10 mins

Timer: 30



1

2

3



1

2

3



TECHNIQUE 1

Shifting One Brick takes 10 mins

Timer: 35



1

2

3

1

2

3



TECHNIQUE 1

Shifting One Brick takes 10 mins

Timer: 40



1

2

3



2

3



TECHNIQUE 1

Shifting One Brick takes 10 mins

Timer: 40



1

2

3



1

2

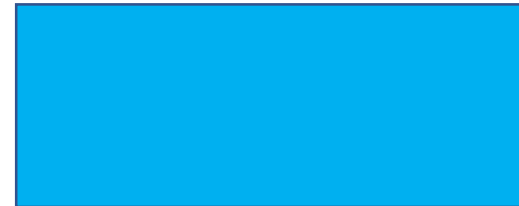
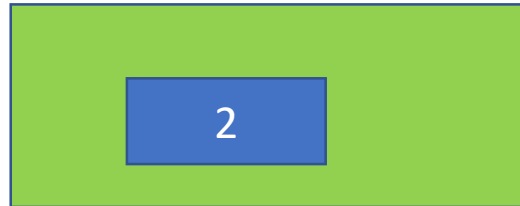
3



TECHNIQUE 1

Shifting One Brick takes 10 mins

Timer: 45



TECHNIQUE 1

Shifting One Brick takes 10 mins

Timer: 50



TECHNIQUE 1

Shifting One Brick takes 10 mins

Timer: 50



1

2

3



1

2

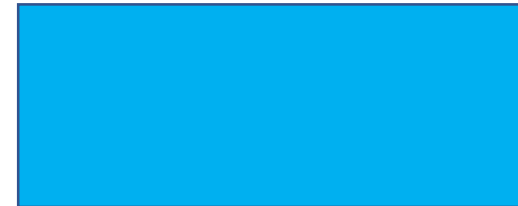
3



TECHNIQUE 1

Shifting One Brick takes 10 mins

Timer: 55



TECHNIQUE 1

Shifting One Brick takes 10 mins

Timer: 60



TECHNIQUE 1

Shifting One Brick takes 10 mins

Timer: 60



TECHNIQUE 2

Shifting One Brick takes 10 mins

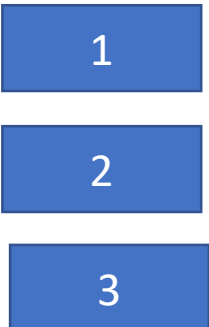
Timer: 0



5 Min



5 Min



TECHNIQUE 2

Shifting One Brick takes 10 mins

Timer: 5

2

3



5 Min



5 Min



1

2

3



TECHNIQUE 2

Shifting One Brick takes 10 mins

Timer: 10

2

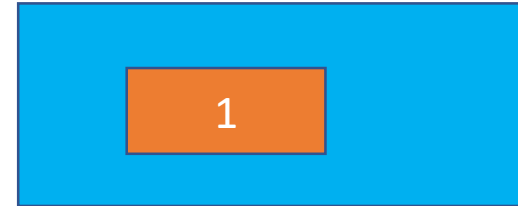
3



5 Min



5 Min



2



TECHNIQUE 2

Shifting One Brick takes 10 mins

Timer: 10

3



5 Min



5 Min



1



2

3



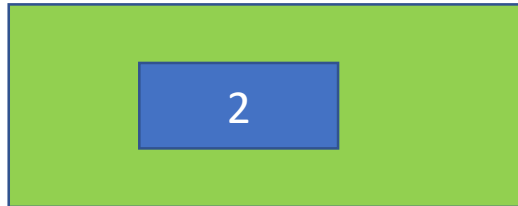
TECHNIQUE 2

Shifting One Brick takes 10 mins

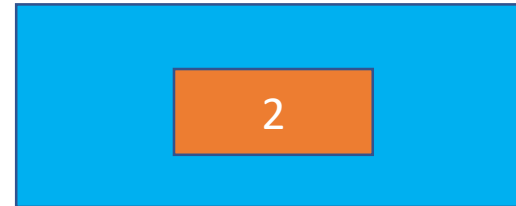
Timer: 15



5 Min



5 Min



TECHNIQUE 2

Shifting One Brick takes 10 mins

Timer: 15



5 Min



5 Min



TECHNIQUE 2

Shifting One Brick takes 10 mins

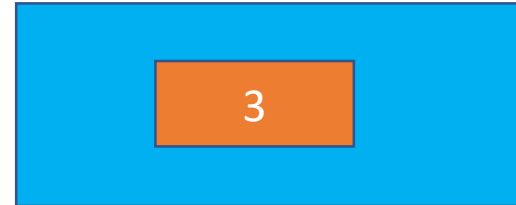
Timer: 25



5 Min



5 Min



TECHNIQUE 2

Shifting One Brick takes 10 mins

Timer: 30



5 Min



5 Min



TECHNIQUE 2

Shifting One Brick takes 10 mins

Timer: 35



I took 30 Mins to Complete



1

2

3

5 Min



5 Min



1

2

3



I took 30 Mins to Complete



But we together took 35 Mins to Complete

Technique 1 Vs Technique 2

Technique 1: Is called Non-Pipelined Execution

Technique 2: Is called Pipelined Execution

Lesson Learnt

Latency: Time taken to complete the task by each team in both Techniques is 30 Mins each

Throughput: Time taken to complete 2 tasks.

- Technique 1 took 60 Mins
- Technique 2 took 35 Mins.
- Since Resource and Time was shared without overlapping of the task.

Lesson Learnt 2

Time taken to complete the task in **Technique 1** is 6 bricks X 10 Mins= 60 Mins

In **Technique 2**,

1st Brick took 10 Mins

Rest of the bricks were shifted in every 5 Mins

$[(1 \text{ Brick} \times 10 \text{ Mins})] + [(6-1) \times 5] = 10 + 25 = 35 \text{ Mins}$

Which Technique Shows Best Performance?

Time taken by **Technique 1** = 60

Time taken by **Technique 2** = 35

How better is Technique 2 over Technique 1 ?

$$\frac{\text{Execution time of Technique 1}}{\text{Execution Time of Technique 2}} = \frac{60}{35} = \mathbf{1.714}$$

Technique 2 is **1.714** times faster than Technique 1

If Technique 2 is ***n*** times faster than Technique 1

$$n = \frac{\text{Execution Time of Technique 1}}{\text{Execution Time of Technique 2}}$$

Technique1 Vs Technique 2

Also, If Technique 2 is ***n*** times faster than Technique 1

$$n = \frac{\text{Performance of Technique2}}{\text{Performance of Technique1}}$$

$$\text{Performance} = \frac{1}{\text{Execution Time}}$$

Computer X vs Computer Y

If Computer X is ***n*** times faster than Computer Y

$$n = \frac{\text{Execution Time of Computer Y}}{\text{Execution Time of Computer X}}$$

OR

If Computer Y is ***n*** times faster than Computer X

$$n = \frac{\text{Performance of Computer X}}{\text{Performance of Computer Y}}$$

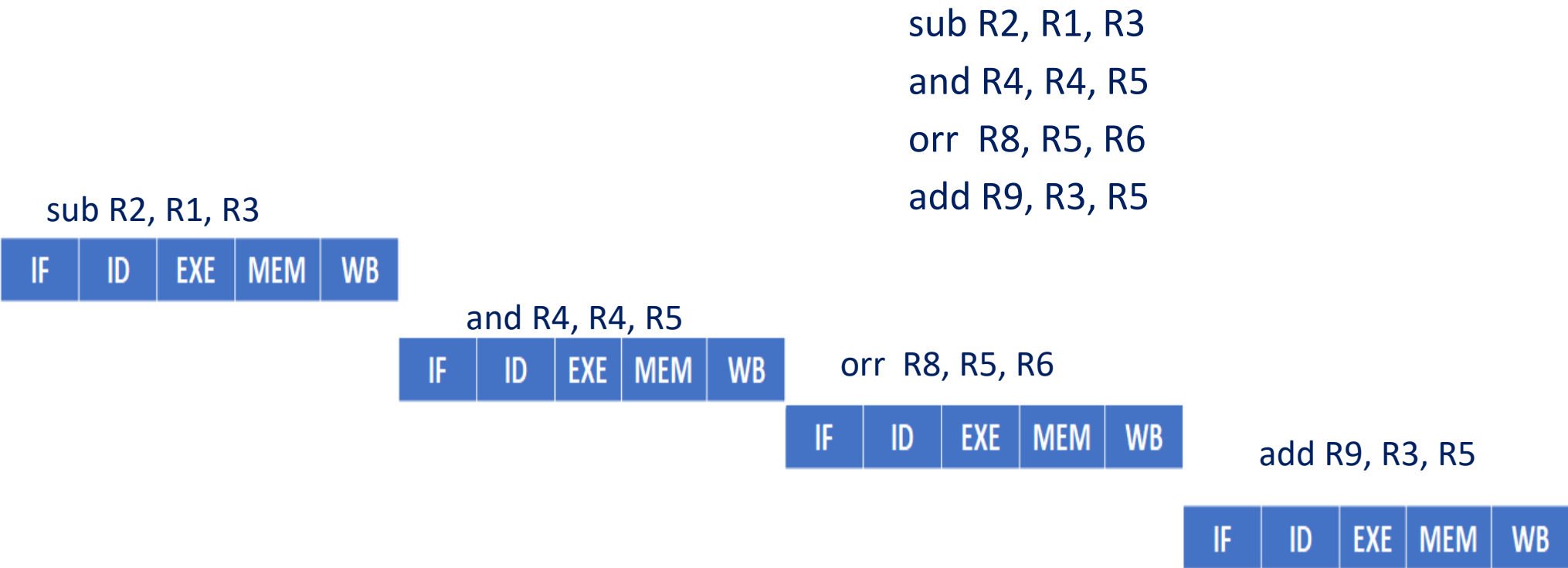
Execution Time

$\text{CPU Time} = \text{Instruction Count (IC)} \times \text{Clock Cycle} \times \text{CPI}$

Reducing any of the 3 factors will lead to improved performance.
Reducing Execution time is done by reducing 1 or more of the following factors -

- CPI: Cycles per instruction
- Clock Cycle
- Instruction count

Computer Y Without Overlapping of Time - Technique1



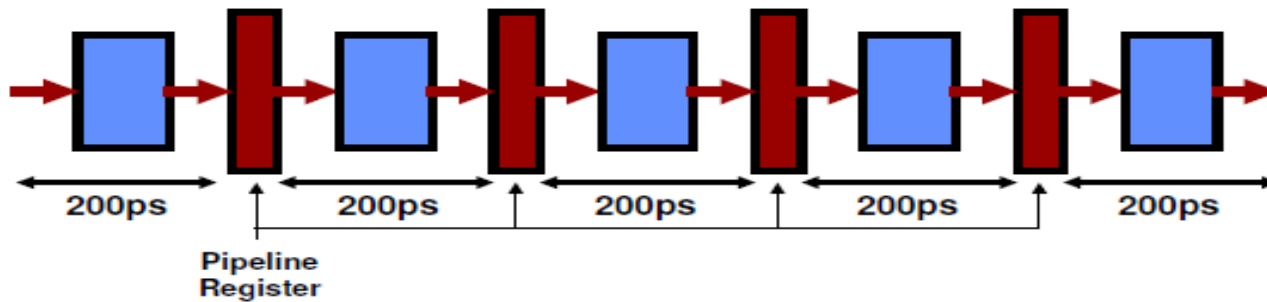
If Each stage takes 10ns, the latency will be 50 ns and the throughput is 200 ns

How to Reduce ?



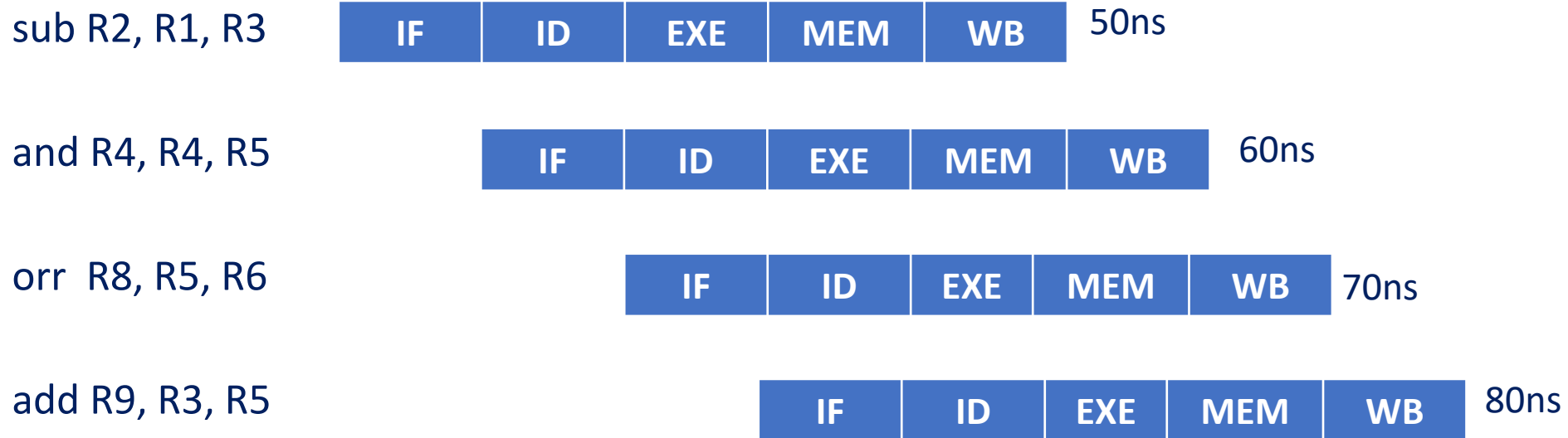
- Fetch - [IF]
- Decode – [ID]
- Execute – [EX]
- Buffer/Data or Memory Access-[MEM]
- Write back – [WB]

Step 1: Divide instruction execution into multiple stage with small Clock



Step 2: Overlap the Execution time of instructions such that, more than one instruction will use different stages in different time slice

Computer X With Overlapping of Time – Technique2



If Each stage take 10ns, the latency will be 50 ns

If Each stage take 10ns, the throughput is 80 ns instead of 200 ns

Which Computer Shows Best Performance?

Execution Time of Computer Y= 200

Execution Time of Computer X= 80

How better is Computer X over Computer Y

$$\frac{\text{Execution time of Computer Y}}{\text{Execution Time of Computer X}} = \frac{200}{80} = 2.5$$

Computer X is **2.5** times faster than Computer Y

**Technique 2 is Called Pipelining
or
Computer X has a Pipelined Processor**



1. Which of the following best defines pipelining in a processor?

- A) Executing multiple instructions completely at the same time
- B) Dividing instruction execution into stages and overlapping their execution
- C) Reducing the instruction count to improve performance
- D) Increasing clock cycle time to reduce latency

2. In a pipelined processor, which performance metric improves the most due to overlapping execution?

- A) Latency
- B) Clock cycle time
- C) Throughput
- D) Instruction count

1. Which of the following best defines pipelining in a processor?

- A) Executing multiple instructions completely at the same time
- B) Dividing instruction execution into stages and overlapping their execution
- C) Reducing the instruction count to improve performance
- D) Increasing clock cycle time to reduce latency

ANS- B) Dividing instruction execution into stages and overlapping their execution

2. In a pipelined processor, which performance metric improves the most due to overlapping execution?

- A) Latency
- B) Clock cycle time
- C) Throughput
- D) Instruction count

ANS- C) Throughput

3. A task consists of 8 identical jobs. Each job takes 12 minutes to complete in 2 stages of 6 minutes each, what is the total time taken in a pipelined architecture?

- A) 36 minutes
- B) 40 minutes
- C) 54 minutes
- D) 96 minutes

4. Computer Y (non-pipelined) has an execution time of 100 ns. Computer X (pipelined) has an execution time of 25 ns. How many times is Computer X faster than Computer Y?

- A) 1.5×
- B) 2×
- C) 2.5×
- D) 4×

3. A task consists of 8 identical jobs. Each job takes 12 minutes to complete in 2 stages of 6 minutes each, what is the total time taken in a pipelined architecture?

- A) 36 minutes
- B) 40 minutes
- C) 54 minutes
- D) 96 minutes

ANS- C) 54 minutes

4. Computer Y (non-pipelined) has an execution time of 100 ns. Computer X (pipelined) has an execution time of 25 ns. How many times is Computer X faster than Computer Y?

- A) 1.5×
- B) 2×
- C) 2.5×
- D) 4×

ANS- D) 4×



THANK YOU



Team MPCA

Department of Computer Science and
Engineering



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Session - 2.2

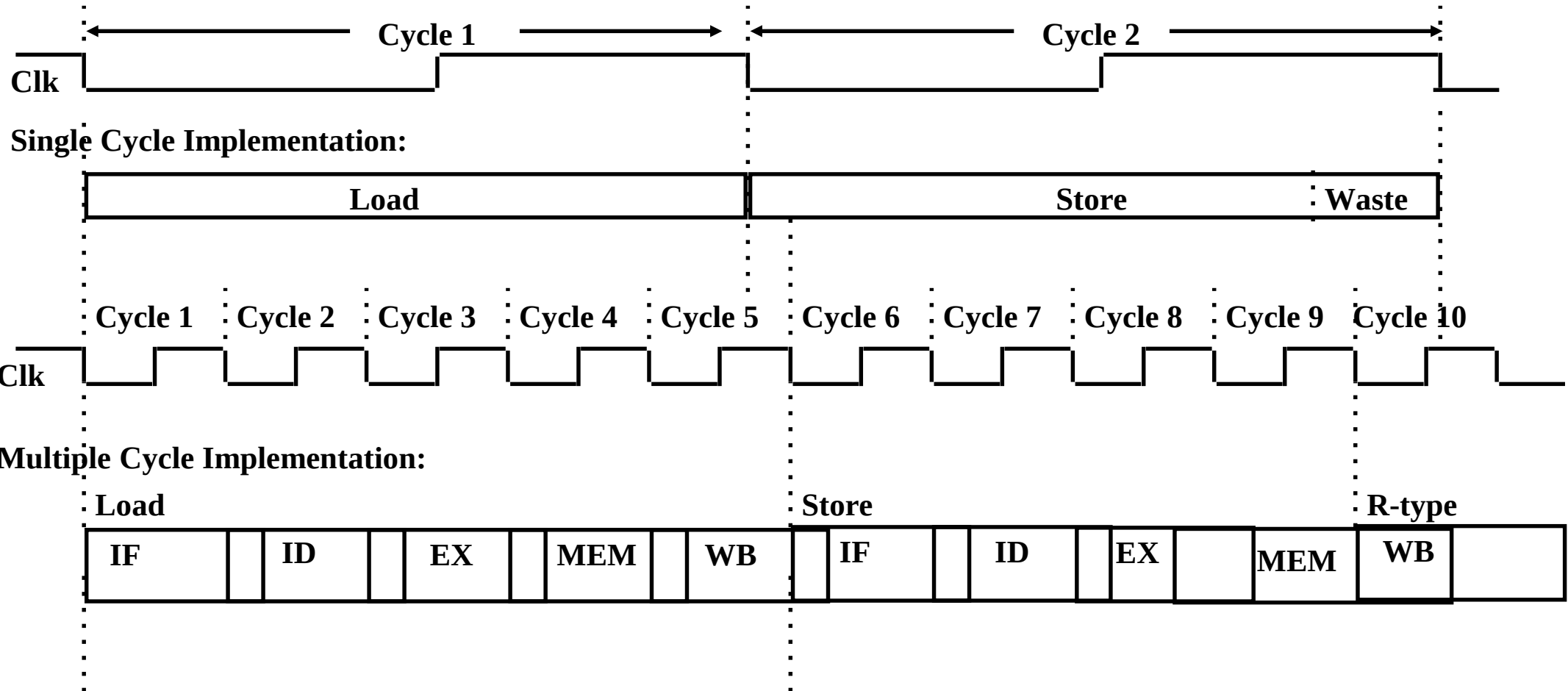
Microprocessor & Computer Architecture (μ pCA)



UE24CS251B

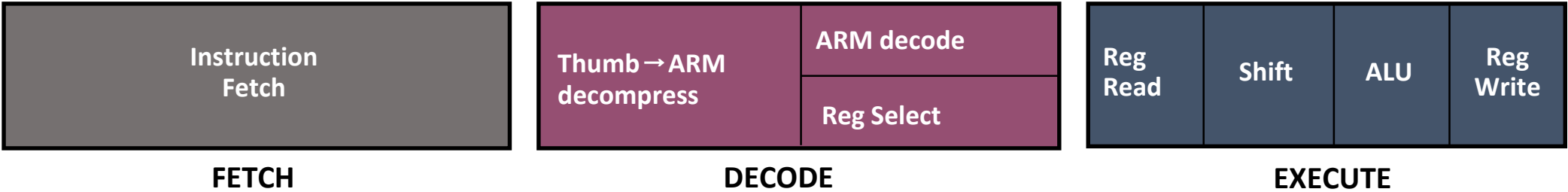
3 Stage & 5 Stage ARM Processor

Quick Recap - Single Cycle vs Multiple Cycle

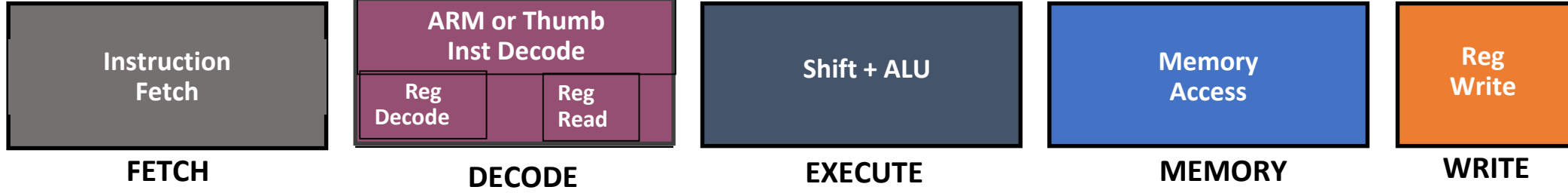


ARM Processors

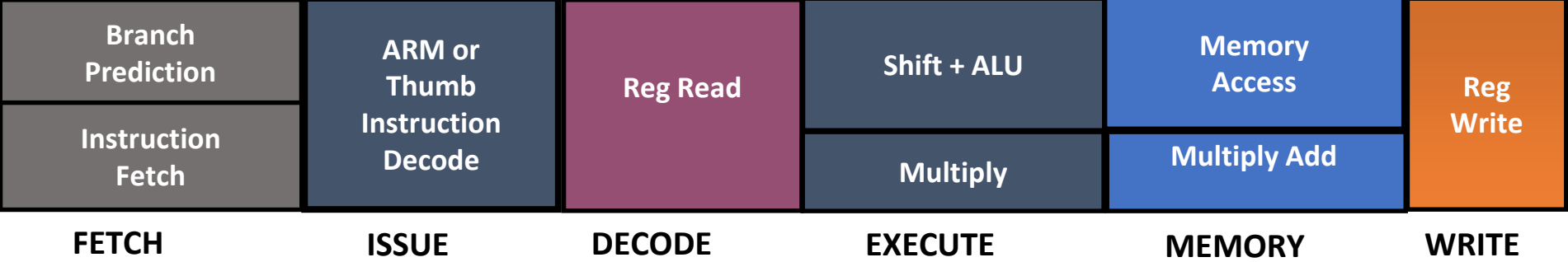
ARM7TDMI



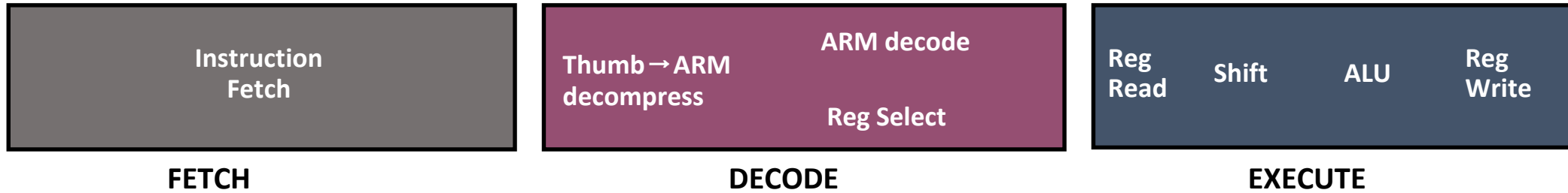
ARM9TDMI



ARM10

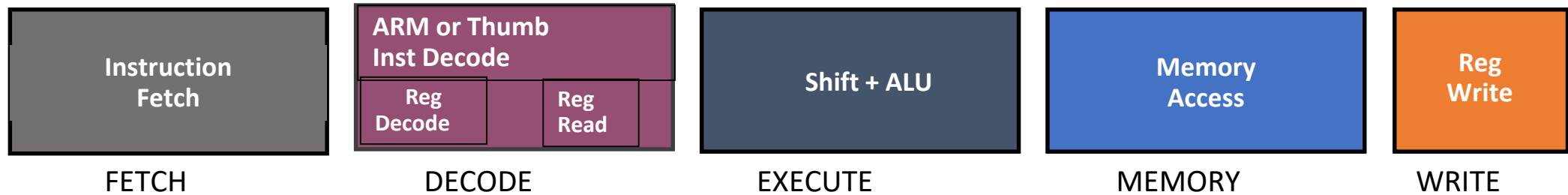


ARM7TDMI – 3 Stage

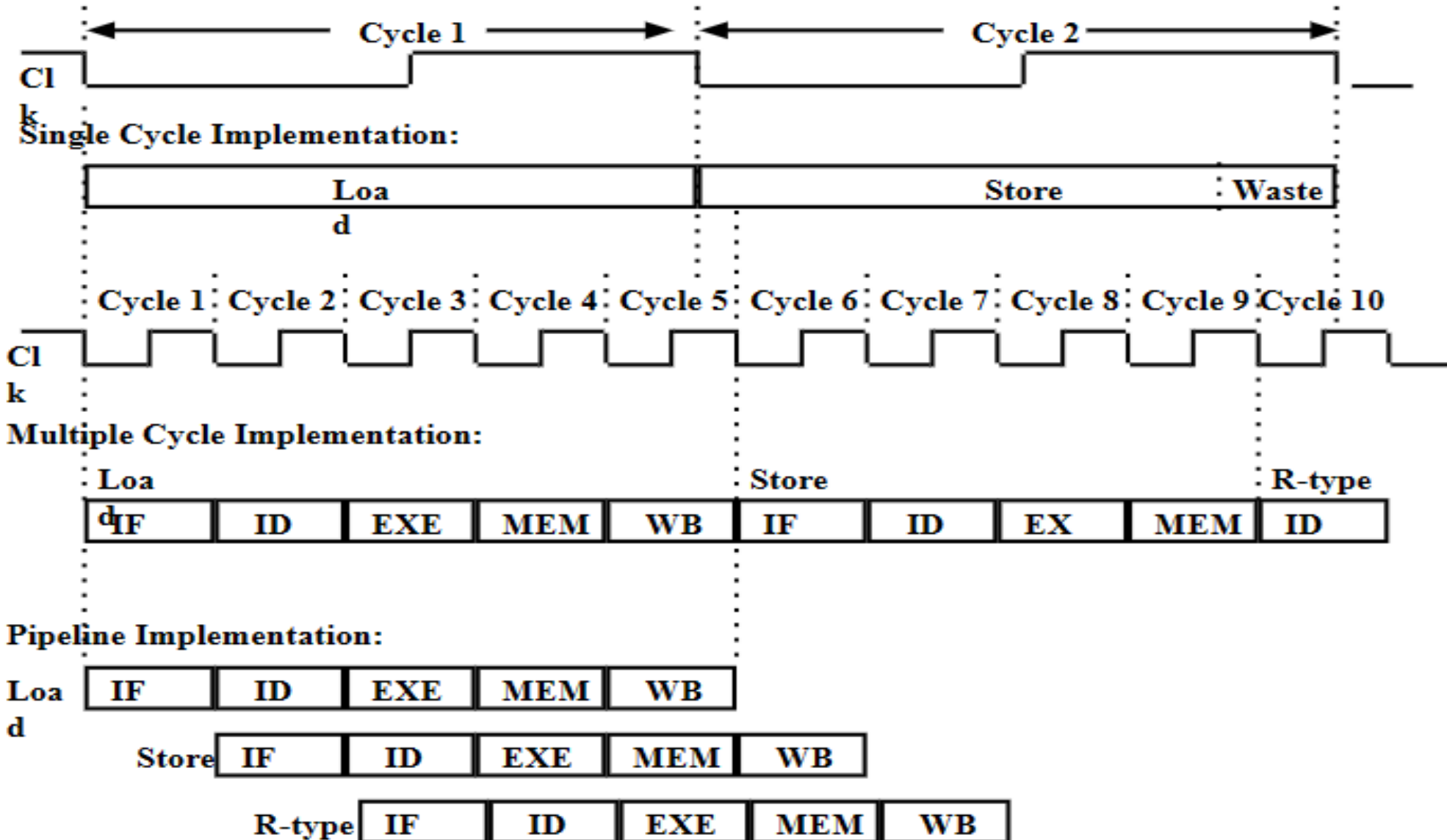


- **Fetch (IF)**
 - The instruction is fetched from memory and placed in the instruction pipeline
- **Decode (ID)**
 - The instruction is decoded and the datapath control signals prepared for the next cycle.
- **Execute (EX)**
 - The register bank is read, an operand shifted, the ALU result generated and written back into a destination register

ARM9TDMI- 5 Stage



Single Cycle, Multiple Cycle, vs. Pipeline



Pipelined Execution

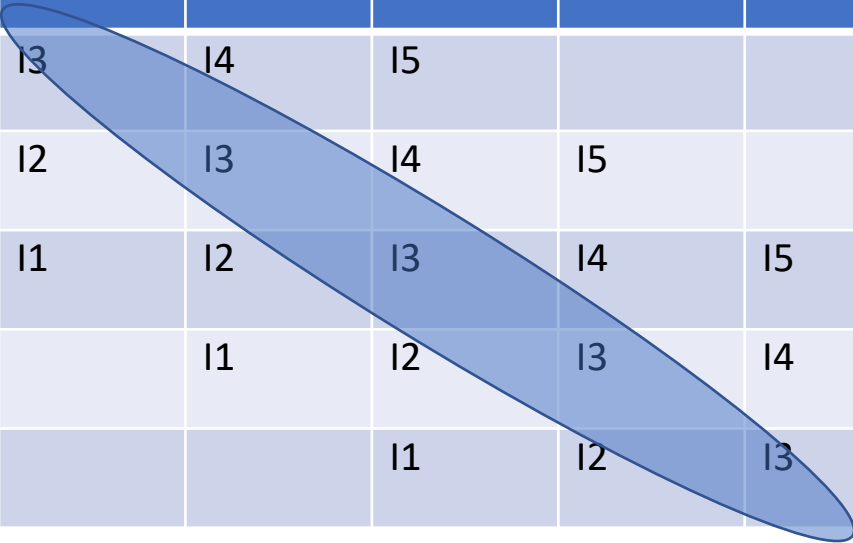
K stage	T1	T2	T3	T4	T5	T6	T7	T8	T9
IF	I1	I2	I3	I4	I5				
ID		I1	I2	I3	I4	I5			
EX			I1	I2	I3	I4	I5		
MB				I1	I2	I3	I4	I5	
WB					I1	I2	I3	I4	I5

Pipelined Execution

K stage	T1	T2	T3	T4	T5	T6	T7	T8	T9
IF	I1	I2	I3	I4	I5				
ID		I1	I2	I3	I4	I5			
EX			I1	I2	I3	I4	I5		
MB				I1	I2	I3	I4	I5	
WB					I1	I2	I3	I4	I5

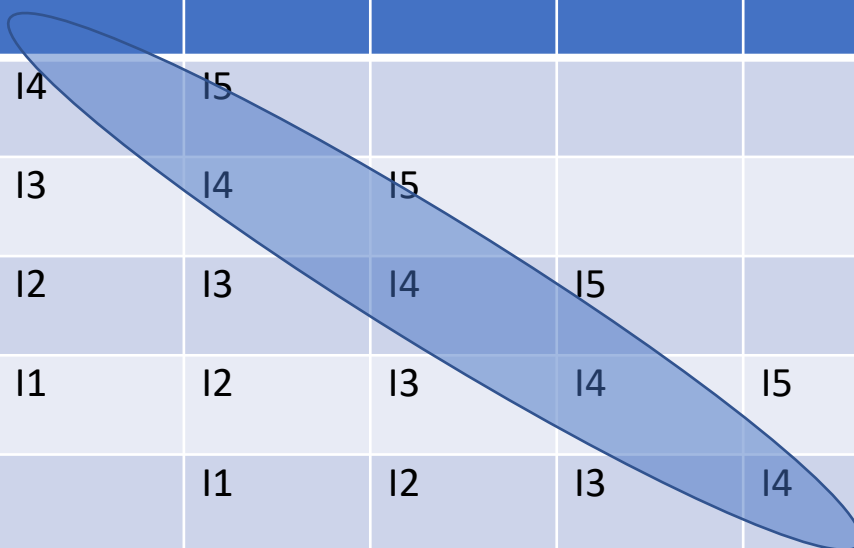
Pipelined Execution

K stage	T1	T2	T3	T4	T5	T6	T7	T8	T9
IF	I1	I2	I3	I4	I5				
ID		I1	I2	I3	I4	I5			
EX			I1	I2	I3	I4	I5		
MB				I1	I2	I3	I4	I5	
WB					I1	I2	I3	I4	I5



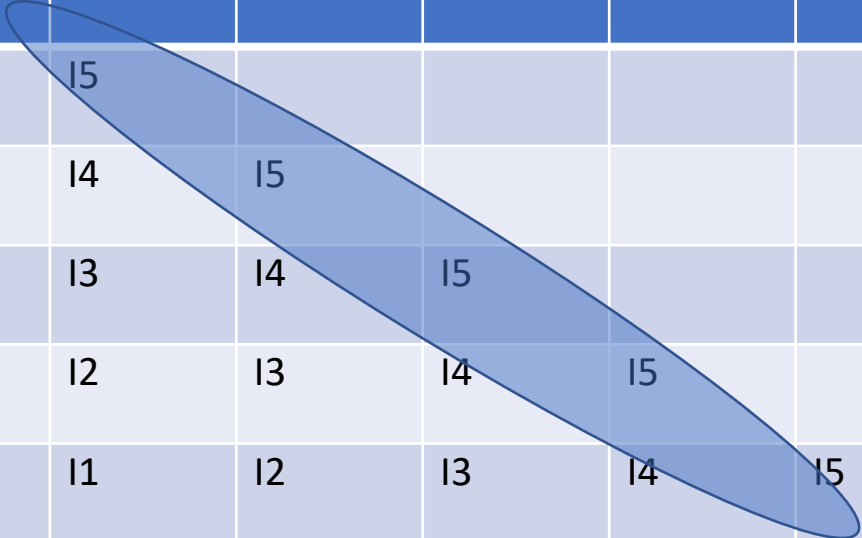
Pipelined Execution

K stage	T1	T2	T3	T4	T5	T6	T7	T8	T9
IF	I1	I2	I3	I4	I5				
ID		I1	I2	I3	I4	I5			
EX			I1	I2	I3	I4	I5		
MB				I1	I2	I3	I4	I5	
WB					I1	I2	I3	I4	I5



Pipelined Execution

K stage	T1	T2	T3	T4	T5	T6	T7	T8	T9
IF	I1	I2	I3	I4	I5				
ID		I1	I2	I3	I4	I5			
EX			I1	I2	I3	I4	I5		
MB				I1	I2	I3	I4	I5	
WB					I1	I2	I3	I4	I5



Pipelined Execution

K stage	T1	T2	T3	T4	T5	T6	T7	T8	T9
IF	I1	I2	I3	I4	I5				
ID		I1	I2	I3	I4	I5			
EX			I1	I2	I3	I4	I5		
MB				I1	I2	I3	I4	I5	
WB					I1	I2	I3	I4	I5

Pipelined Execution

K stage	T1	T2	T3	T4	T5	T6	T7	T8	T9
IF	I1	I2	I3	I4	I5				
ID		I1	I2	I3	I4	I5			
EX			I1	I2	I3	I4	I5		
MB				I1	I2	I3	I4	I5	
WB					I1	I2	I3	I4	I5

Pipelined Execution

K stage	T1	T2	T3	T4	T5	T6	T7	T8	T9
IF	I1	I2	I3	I4	I5				
ID		I1	I2	I3	I4	I5			
EX			I1	I2	I3	I4	I5		
MB				I1	I2	I3	I4	I5	
WB					I1	I2	I3	I4	I5

Pipelined Execution

K stage	T1	T2	T3	T4	T5	T6	T7	T8	T9
IF	I1	I2	I3	I4	I5				
ID		I1	I2	I3	I4	I5			
EX			I1	I2	I3	I4	I5		
MB				I1	I2	I3	I4	I5	
WB					I1	I2	I3	I4	I5

Pipelined Execution

K stage	T1	T2	T3	T4	T5	T6	T7	T8	T9
IF	I1	I2	I3	I4	I5				
ID		I1	I2	I3	I4	I5			
EX			I1	I2	I3	I4	I5		
MB				I1	I2	I3	I4	I5	
WB					I1	I2	I3	I4	I5

Diagram illustrating the execution of 5 instructions (I1 to I5) across 5 stages (IF, ID, EX, MB, WB) over 9 clock cycles (T1 to T9). The first instruction (I1) takes 5 clock cycles to complete. Subsequent instructions (I2 to I5) take 4 clock cycles each, as they can start at the next clock cycle after the previous instruction has completed its first stage (IF).

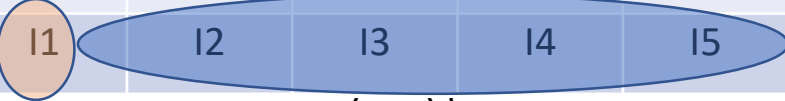
Annotations:

- A blue arrow points from the start of the first instruction (I1) at T1 to the start of the fifth instruction (I5) at T5, labeled $5*1*1$.
- A blue oval highlights the execution of the fifth instruction (I5) across stages T5 to T9, labeled $(5-1)*1$.

- 1st instruction takes 5 Clock time (Each stage, 1 clock time T_c)
- Rest will take 1 more than the previous instruction
- **Execution time on pipeline processor:** $= K \text{ stage} * 1 \text{ instruction} + (n-1) :$
 $= 5*1+4= 9 \text{ clocks}$
- **Execution on a non-pipeline processor:** $= K \text{ stage} * n \text{ Instructions}$
 $= 5*5= 25 \text{ clocks}$

Pipelined Execution

K stage	T1	T2	T3	T4	T5	T6	T7	T8	T9
IF	I1	I2	I3	I4	I5				
ID		I1	I2	I3	I4	I5			
EX			I1	I2	I3	I4	I5		
MB				I1	I2	I3	I4	I5	
WB					I1	I2	I3	I4	I5

$k * t_c * 1$


 $(n-1) * t_c$

Number of stages = k

Clock Cycle = t_c

Number of Instructions = n

■ Execution time_{pipeline} = $k * t_c * 1 + (n-1) * t_c = [k + (n-1)] t_c$

■ Execution time_{unpipeline} = $n * t_p = n * k * t_c$

Speedup vs # of Stages in Pipeline Processor

$$\blacksquare \text{Speedup}(S) = \frac{\text{Execution Time on Unpipelined}}{\text{Execution Time on Pipelined}}$$

$$\frac{nt_p}{(k+n-1)t_c} \rightarrow \frac{nt_p}{(k-1+n)t_c}$$

If n is too big or as number of instructions increases $n > k-1$ will tend to n

MEANING: Lengthy programs will have more number of instructions, hence number of stages of the pipeline is almost negligible.

$$\text{Speedup}(S) = \frac{nt_p}{nt_c}$$

$$\text{Speedup}(S) = \frac{t_p}{t_c}$$

$$\text{Speedup}(S) = \frac{k * t_c}{t_c}$$

$$\text{Speedup}(S) = k$$

Pipeline Execution

Compute the Execution time on 5 Stage processor (Pipeline vs Non Pipeline Processor):

Number of instructions = 100.

Cycle Time T_c = 60

Solution:

Execution on a non-pipeline processor: = $K \text{ stage} * T_c * n \text{ Instructions}$
= $5 * 60 * 100 = 30000$ clock cycles

Execution Time on Pipeline Processor: = $K \text{ stage} * 1 \text{ instruction} * T_c + (n-1) * T_c$:
= $(5 * 1 * 60) + (99 * 60) = 300 + 5940 = 6240$ clock cycles

Clock time per stage = Cycle time = T_c

$$\blacksquare \text{Speedup}(S) = \frac{\text{Execution Time on Unpipelined}}{\text{Execution Time on Pipelined}}$$

$$\blacksquare \text{Speedup}(S) = \frac{30000}{6240}$$

$$\blacksquare \text{Speedup}(S) = 4.8 \approx 5$$

Design issue 1

Clock Cycle Time of all the stages cannot be same!

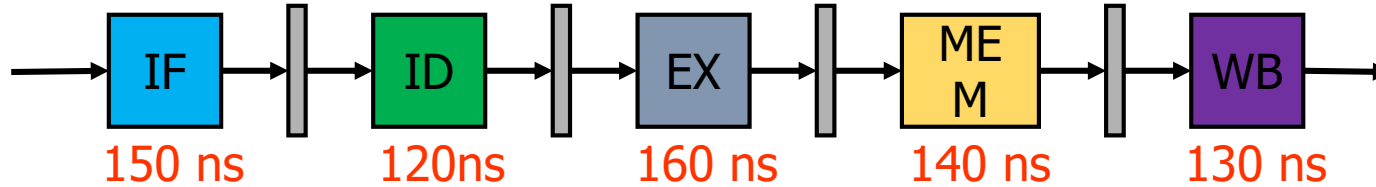
Example

IF	ID	EXE	MEM	WB
300 ps	400 ps	350 ps	550 ps	100 ps
200 ps	150 ps	100 ps	190 ps	140 ps

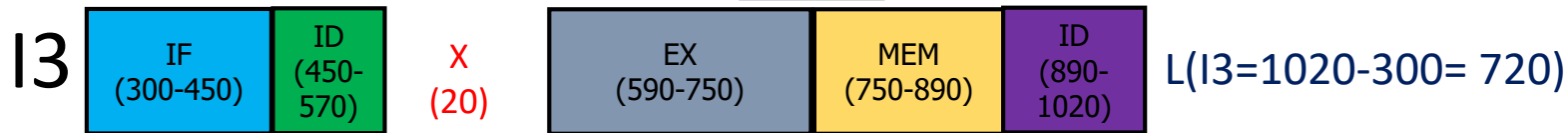
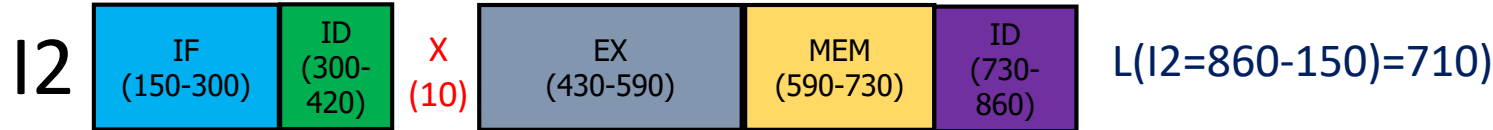
Leads to imbalance latency

Solution: Make length of each stage equal to length of longest stage or slowest stage

PIPELINE THROUGHPUT AND LATENCY



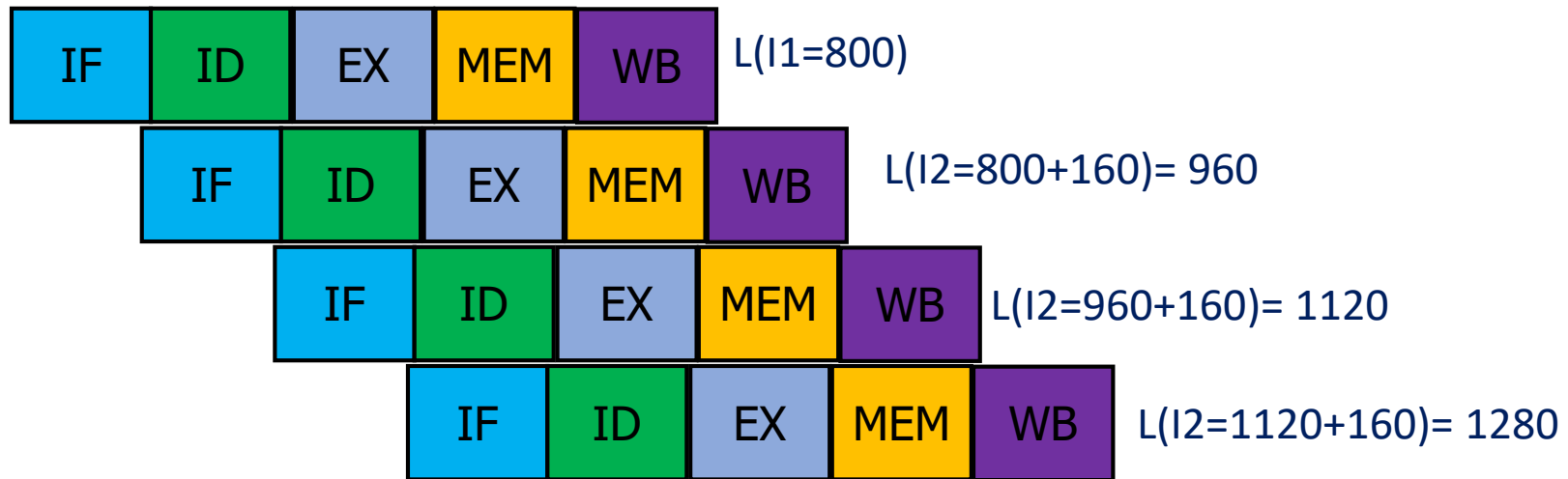
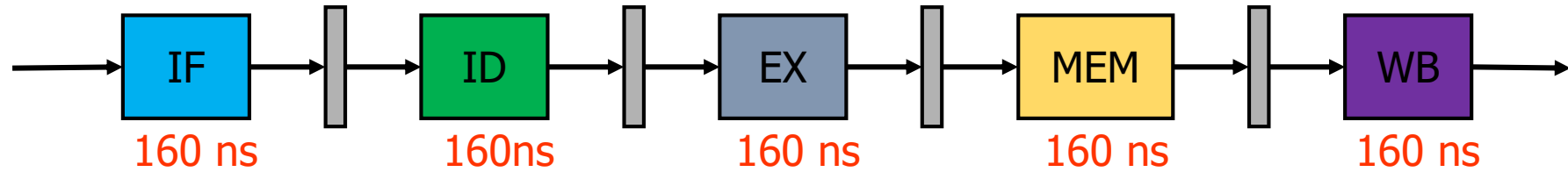
I1



Uneven latency and Throughput= 1180

PROCESSOR

PIPELINE THROUGHPUT AND LATENCY



Balanced latency and Throughput= 1280 instead of 1180

Evaluating 5-Stage Pipeline Processor

A 5 stage pipeline processor has stage delays as 150, 120, 160, 140 and 130 ns.

What is the time taken to execute 100 instructions.

What is the Speed up of pipeline processor?

Solution:

Execution on a non-pipeline processor: $(150+120+160+140+130) * 100 = 700 * 100 = 70000 \text{ ns}$

Execution on a pipeline processor:

Slowest stage: $\text{Max}(150, 120, 160, 140, 130) = 160 \text{ ns}$

Clock time T_c of each stage = 160 ns

$$\begin{aligned} &= (1 * T_c * \# \text{stages}) + ((IC - 1) * T_c) \\ &= (1 * 160 * 5) + (99 * 160) \\ &= 800 + 15840 \\ &= 16640 \text{ ns} \end{aligned}$$

Performance of 5-Stage Pipeline Processor

Execution on a non-pipeline processor

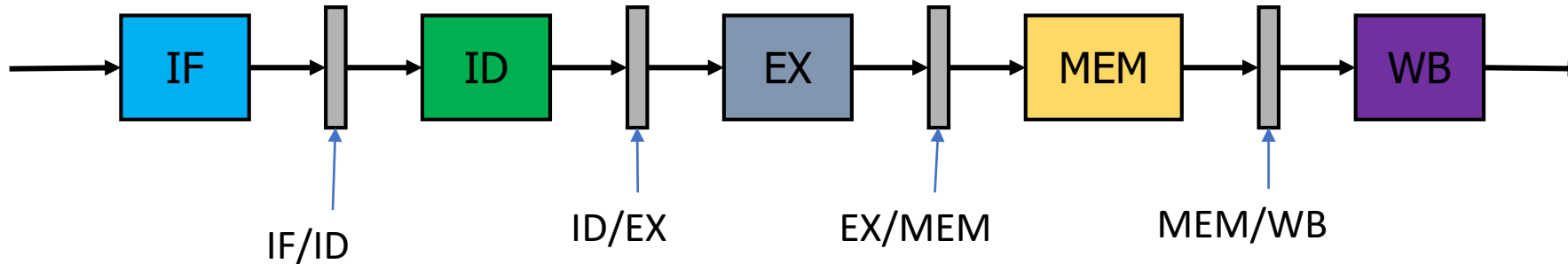
$$(150+120+160+140+130) * 100 = 700 * 100 = 70000\text{ns}$$

Execution on a pipeline processor:

$$(1 * 160 * 5) + (99 * 160) = 800 + 15840 = 16640\text{ns}$$

$$\text{Speedup}(S): 70000 / 16640 = 4.2$$

Design issue 2: Pipeline Register Overhead



- We shall refer to the pipeline registers that are set between two stages with the names of the stages. So, we will have IF/ID, ID/EX, EX/MEM and MEM/WB registers.
- They serve the purpose of transferring outputs produced in a phase to the subsequent phase in the multi-cycle implementation.
- These registers must be large enough to contain all data moving from one phase to the following one

Design issue 2: Pipeline Register Overhead

Pipeline overhead: combination of **pipeline register delay** and the **clock skew**.

- **Pipeline registers delay:** Setup time that triggers a write or when data input changes and propagation delay to the clock.
- **Clock skew:** Maximum delay between when the clock **arrives** at any two registers.

Evaluating 5-Stage Pipeline Processor With Register Overhead

A 5 stage pipeline processor has stage delays as 150, 120, 160, 140 and 130 ns. The register overhead is 5 ns each.

What is the time taken to execute 100 instructions.

What is the Speed up of pipeline processor?

Solution:

Execution on a non-pipeline processor: $(150+120+160+140+130) * 100 = 700 * 100 = 70\text{kns}$

Execution on a pipeline processor:

Slowest stage: $\text{Max}(150, 120, 160, 140, 130) = 160\text{ns}$

Clock time T_c of each stage = 160 ns

Register overhead of each stage = 5ns

Clock time T_c of each stage = 165 ns

Execution on a pipeline processor:

$$\begin{aligned} &= (1 * T_c * \# \text{stages}) + ((IC - 1) * T_c) \\ &= (1 * 165 * 5) + (99 * 165) \\ &= 825 + 16335 \\ &= 17160 \text{ ns} \end{aligned}$$

Performance of 5-Stage Pipeline Processor

Execution on a non-pipeline processor:

$$(150+120+160+140+130) * 100 = 362 * 100 = 70000\text{ns}$$

Execution on a pipeline processor:

$$(1 * 165 * 5) + (99 * 165) = 825 + 16335 = 17160\text{ns}$$

$$\text{Speedup(S): } 70000 / 17160 = 4.07$$

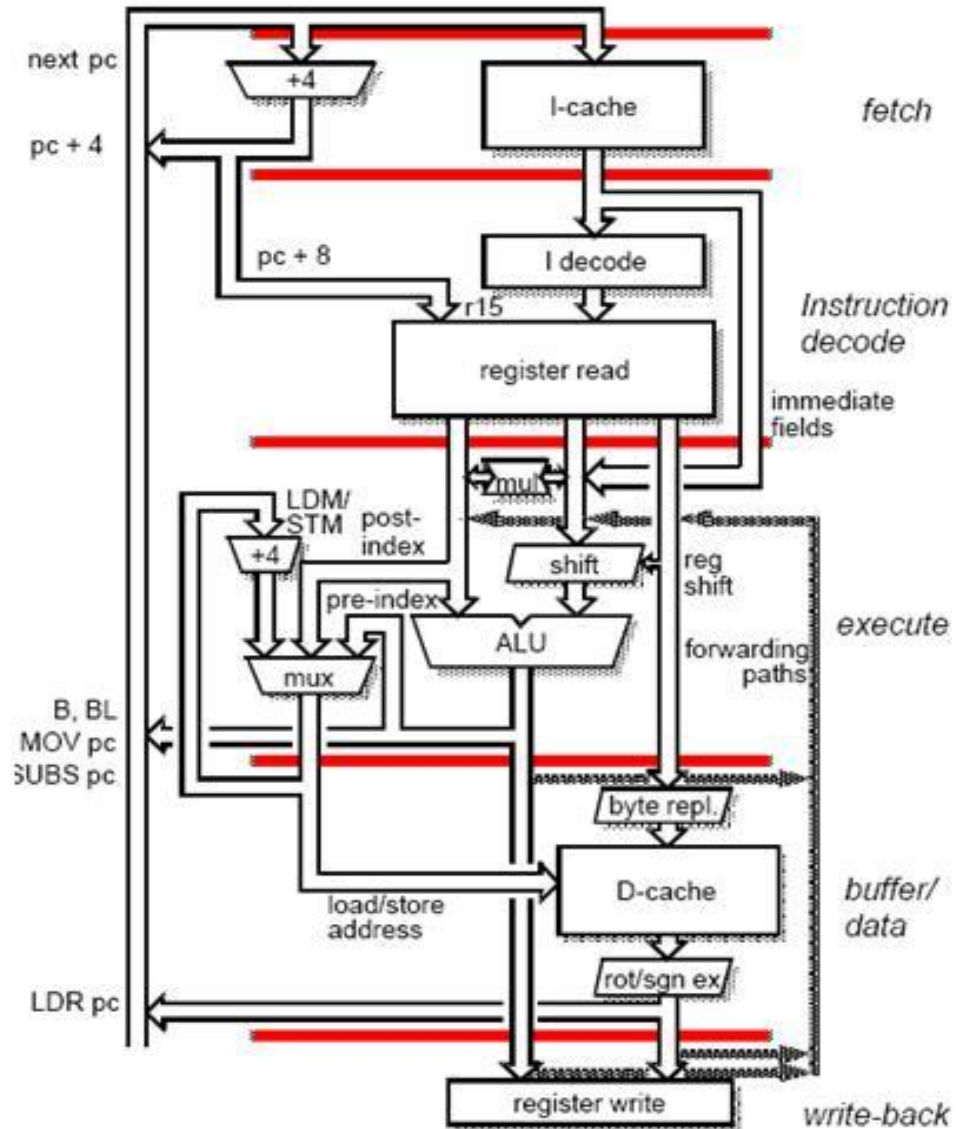


Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Session - 2.21

ARM 5-stage (ARM 9 Architecture)

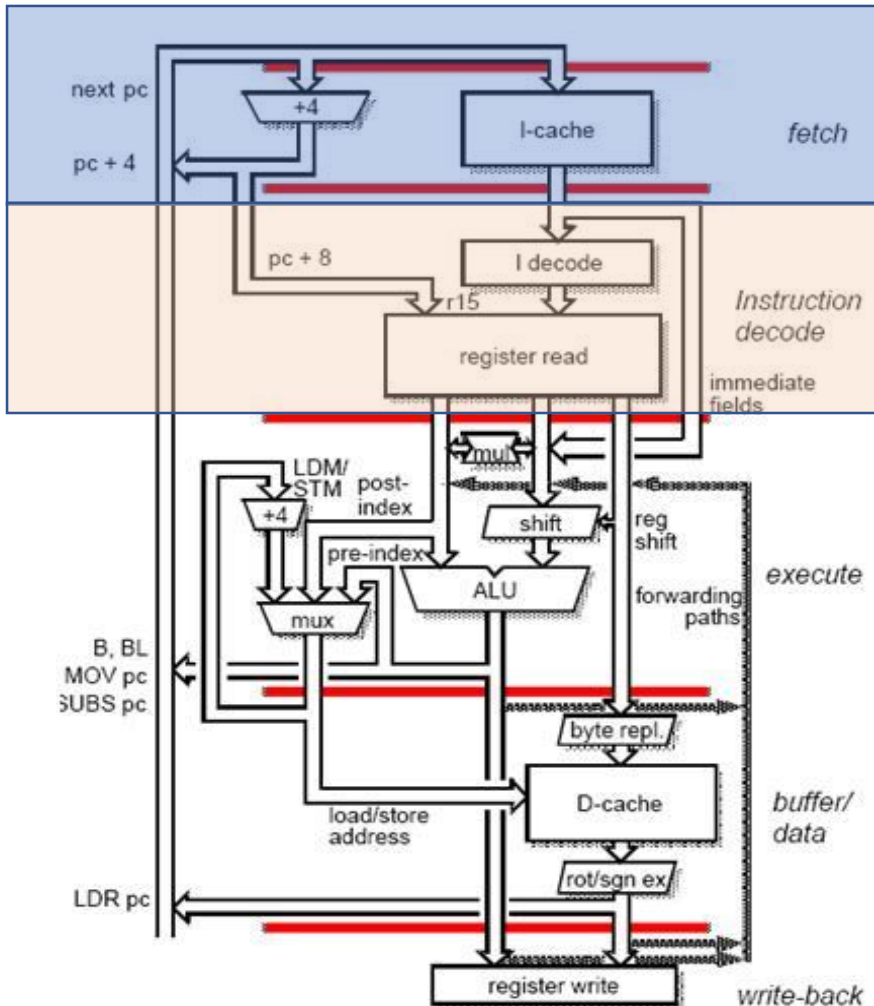


- Fetch - [IF]
- Decode - [ID]
- Execute - [EX]
- Buffer/Data or Memory Access - [MEM]
- Write back - [WB]

Instruction & Data Memory?
Split

Harvard Architecture

ARM ARCHITECTURE - 5 STAGE PIPELINING - Fetch - [IF], Decode – [ID]



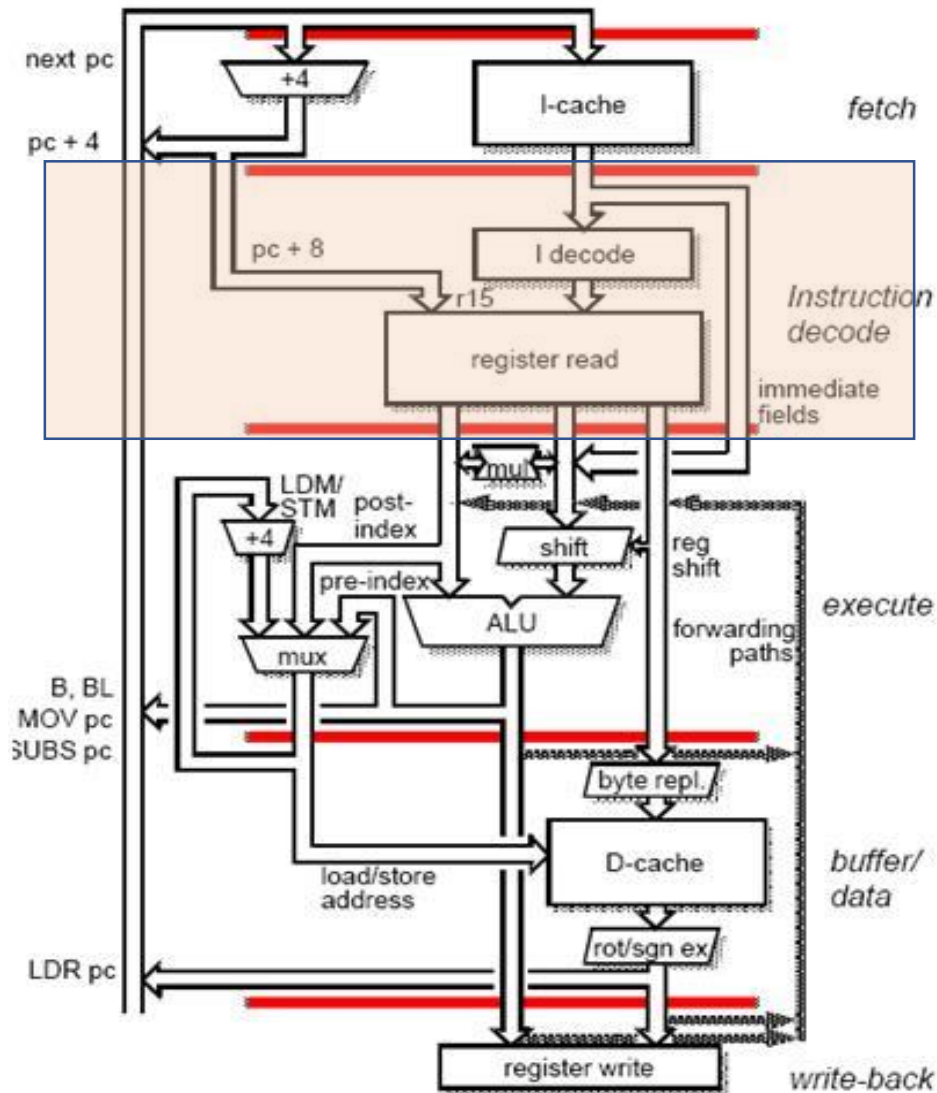
Fetch - [IF]

- The instruction is **fetched** from memory and placed in the instruction pipeline.
- Update the PC to the next sequential PC by adding 4 to PC.

Decode – [ID]

- The instruction is **decoded** and **register operands read** from the register files. There are 3 operand read ports in the register file so most ARM instructions can source all their operands in one cycle.
- Do the equality test on the registers as they are read, for a possible branch.
- Sign extend the offset field of the instruction in case it is needed.

ARM ARCHITECTURE - 5 STAGE PIPELINING- DECODE STAGE - [ID]



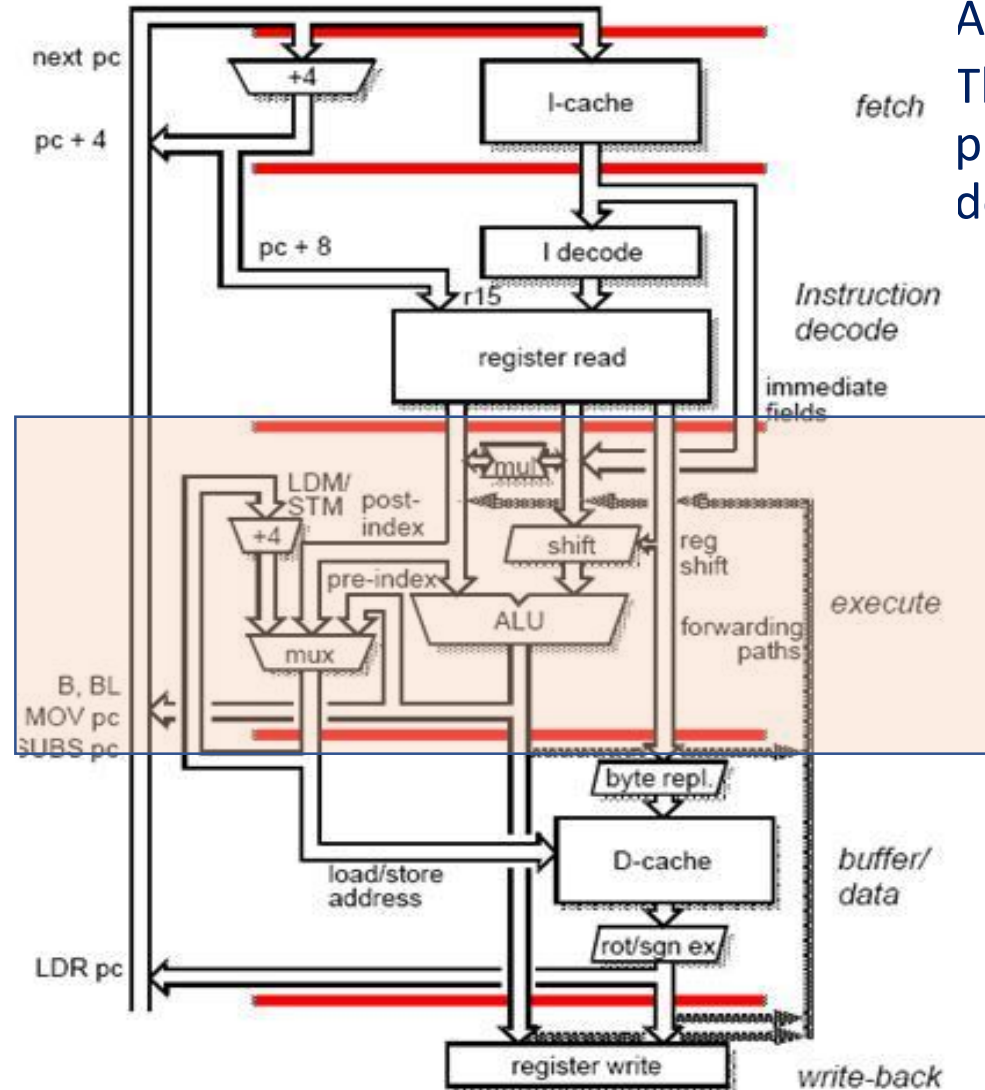
- Compute the possible branch target address by **adding the sign-extended offset** to the incremented PC.
- Further, the branch can be completed by the **end of this stage** by **storing** the branch target address into the **PC** if condition yielded true.
- Decoding is done in parallel with reading registers, as the register **specifiers** are at fixed location in a RISC architecture.

ARM ARCHITECTURE - 5 STAGE PIPELINING Execute – [EX]

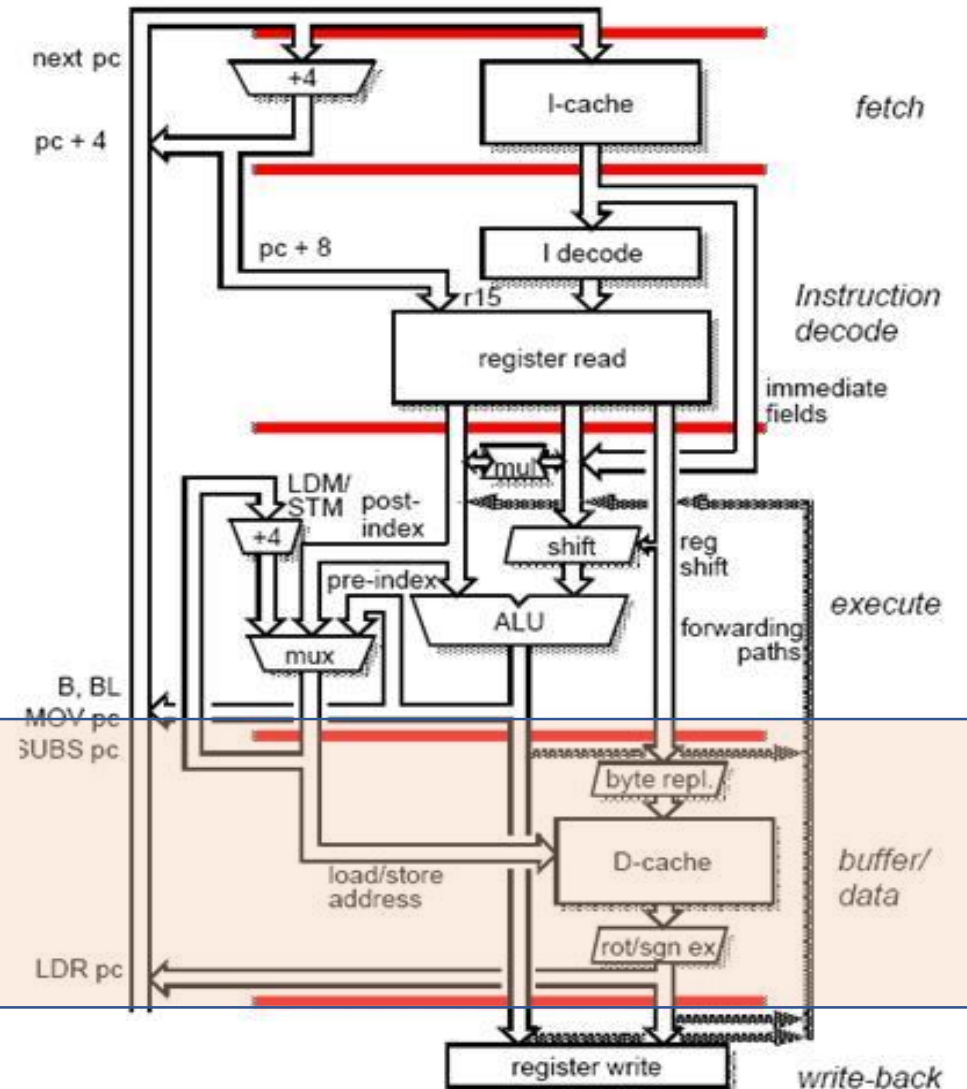
Also called as execute / effective address cycle.

The ALU operates on the operands prepared in the previous cycle, performing of the three functions depending on the instruction type.

- **Memory Reference:** the ALU adds the base register and the offset to form the effective address.
- **Register – Register ALU instruction:** The ALU performs the operation specified by the opcode on the values read from the register file.
- **Register – Immediate ALU instruction:** The ALU performs the operations specified by the opcode on the first value read from the register file and the sign extended immediate.



Buffer/Data or Memory Access-[MEM]



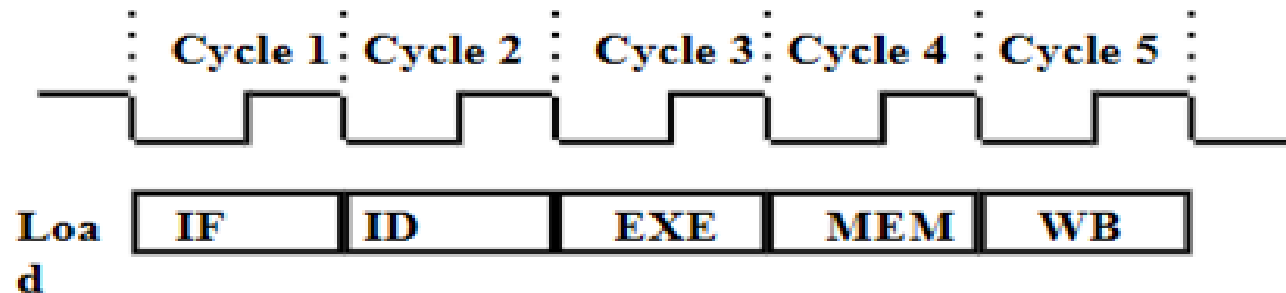
Data memory is accessed if required. Otherwise the ALU result is simply buffered for one cycle.

If the instruction is a LOAD, the memory does a read using effective address computed in the previous cycle.

If the instruction is a STORE, then the memory writes the data from the second register read using the effective address.

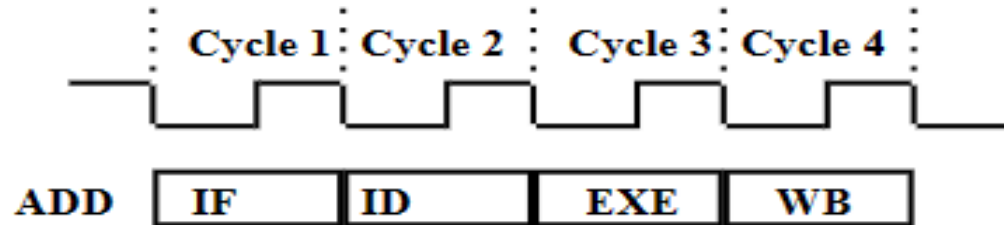
Buffer/Data or Memory Access-[MEM]

The Five Stages of Load



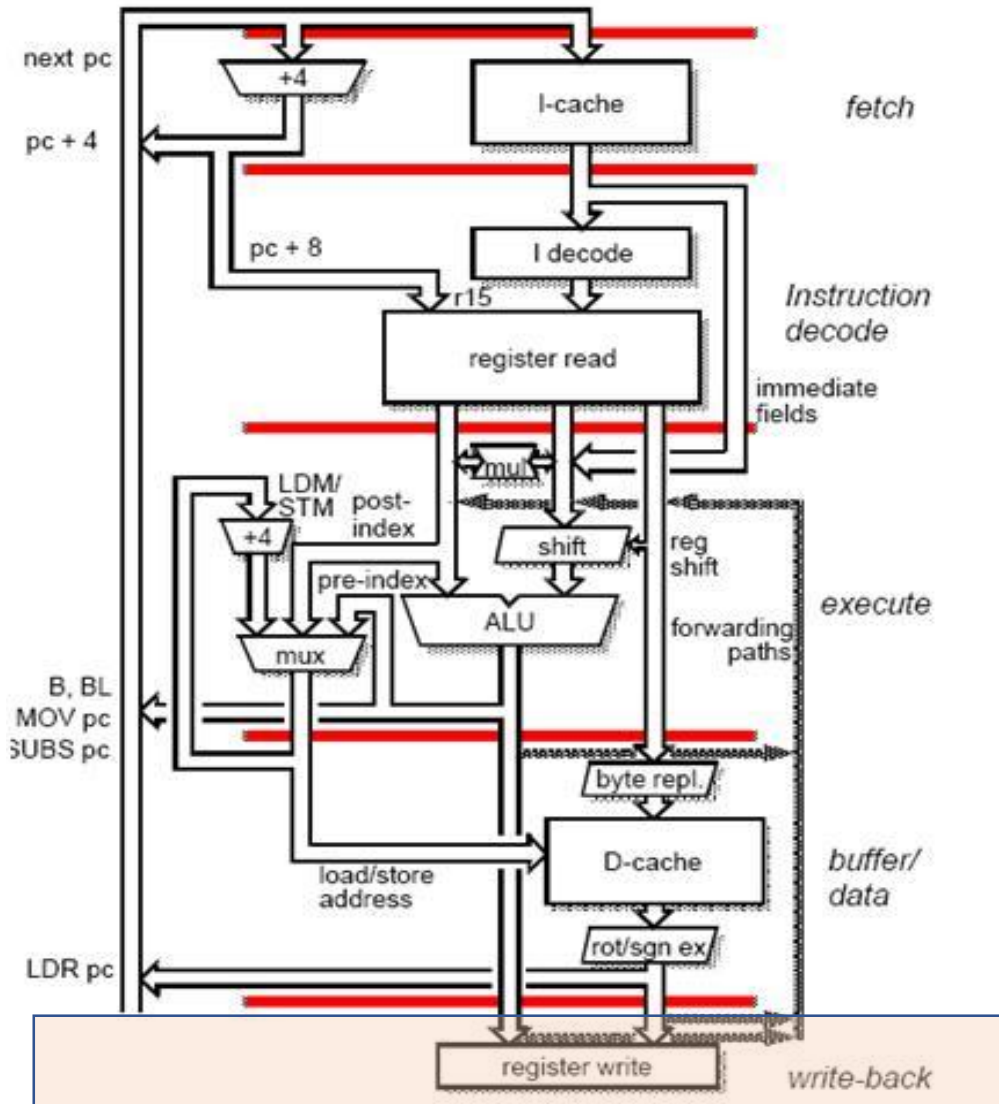
- IF: Instruction Fetch
 - Fetch the instruction from the Instruction Memory
- ID: Registers Fetch and Instruction Decode
- EXE: Calculate the memory address
- MEM: Read the data from the Data Memory
- WB: Write the data back to the register file

The Four Stages of Data Processing



- **IF: Instruction Fetch**
 - Fetch the instruction from the Instruction Memory
- **ID: Registers Fetch and Instruction Decode**
- **EXE:**
 - ALU operates on the two register operands
 - Update PC
- **WB: Write the ALU output back to the register file**

ARM ARCHITECTURE - 5 STAGE PIPELINING - Write back – [WB]



The result generated by the instruction is **written back to the register file**.

The data may come either from memory system [for LOAD], or from the ALU [for an ALU instruction].

1. Which pipeline stage determines the clock cycle time in a pipelined processor?

- A) First stage
- B) Last stage
- C) Average stage delay
- D) Slowest stage

2. Why does pipelining not reduce the execution time of the first instruction?

- A) Register overhead
- B) Instruction dependency
- C) Pipeline fill latency
- D) Memory access delay

1. Which pipeline stage determines the clock cycle time in a pipelined processor?

- A) First stage
- B) Last stage
- C) Average stage delay
- D) Slowest stage

ANS- D) Slowest stage

2. Why does pipelining not reduce the execution time of the first instruction?

- A) Register overhead
- B) Instruction dependency
- C) Pipeline fill latency
- D) Memory access delay

ANS- C) Pipeline fill latency

3. A 5-stage pipeline processor has stage delays of 150 ns, 120 ns, 160 ns, 140 ns, and 180 ns. No register overhead.

How much time is required to execute 100 instructions?

- A) 18720 ns
- B) 16640 ns
- C) 70000 ns
- D) 17500 ns

4. In the ARM 5-stage pipeline, during which stage is the effective address for LOAD/STORE instructions computed?

- A) Instruction Fetch (IF)
- B) Decode (ID)
- C) Execute (EX)
- D) Memory Access (MEM)

3. A 5-stage pipeline processor has stage delays of 150 ns, 120 ns, 160 ns, 140 ns, and 180 ns. No register overhead.

How much time is required to execute 100 instructions?

- A) 18720 ns
- B) 16640 ns
- C) 70000 ns
- D) 17500 ns

ANS- A) 18720 ns

4. In the ARM 5-stage pipeline, during which stage is the effective address for LOAD/STORE instructions computed?

- A) Instruction Fetch (IF)
- B) Decode (ID)
- C) Execute (EX)
- D) Memory Access (MEM)

ANS- C) Execute (EX)



THANK YOU



Team MPCA

Department of Computer Science and
Engineering



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Session - 2.3

Microprocessor & Computer Architecture (μ pCA)



UE24CS251B

Pipeline Processor- Hazards

Facts About Pipeline Processor

- Pipelining improves the CPU instruction throughput.
(Pipeline Throughput) Ideally, $CPI = 1$.
- Pipelining does not reduce the execution time of an instruction.
(Pipeline Latency)
- Infact, it slightly increases the execution time due to the increased control overhead of the pipeline stage register delays.
- All instructions that share a pipeline must have the same stages in the same order.
 - ✓ **Add** does nothing during Mem stage
 - ✓ **Store** does nothing during WB stage
 - ✓ **b{cond}** does nothing in EX, MEM & WB stage

Non-Pipeline Execution

sub R2, R1, R3
and R4, R4, R5
orr R8, R5, R6
add R9, R3, R5

sub R2, R1, R3



and R4, R4, R5



orr R8, R5, R6

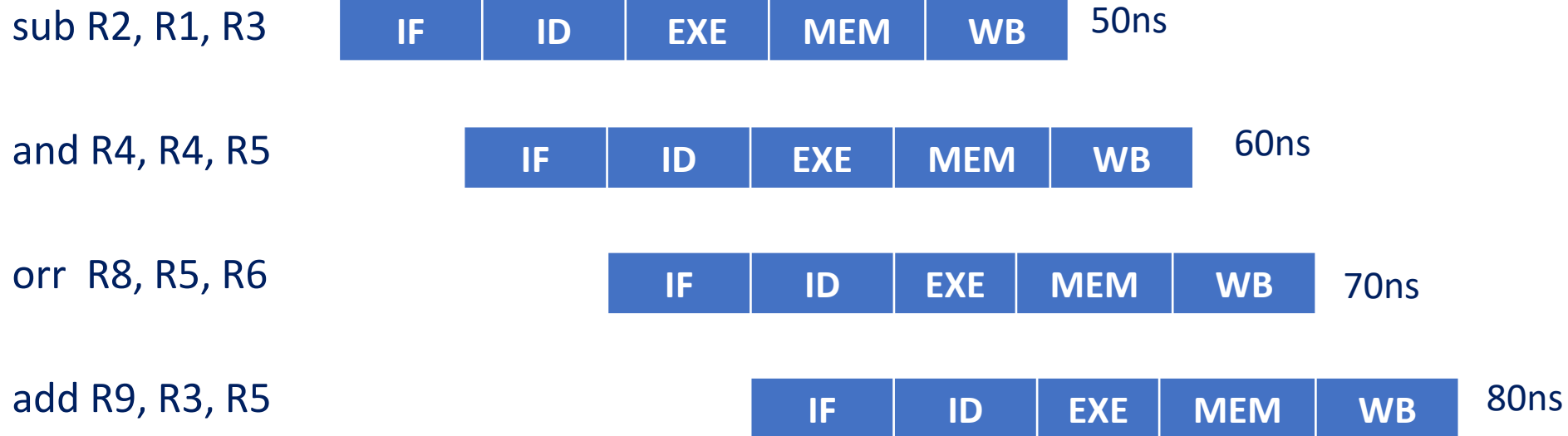


add R9, R3, R5



If Each stage takes 10ns, the latency will be 50 ns and the throughput is 200 ns

Pipelining is Good



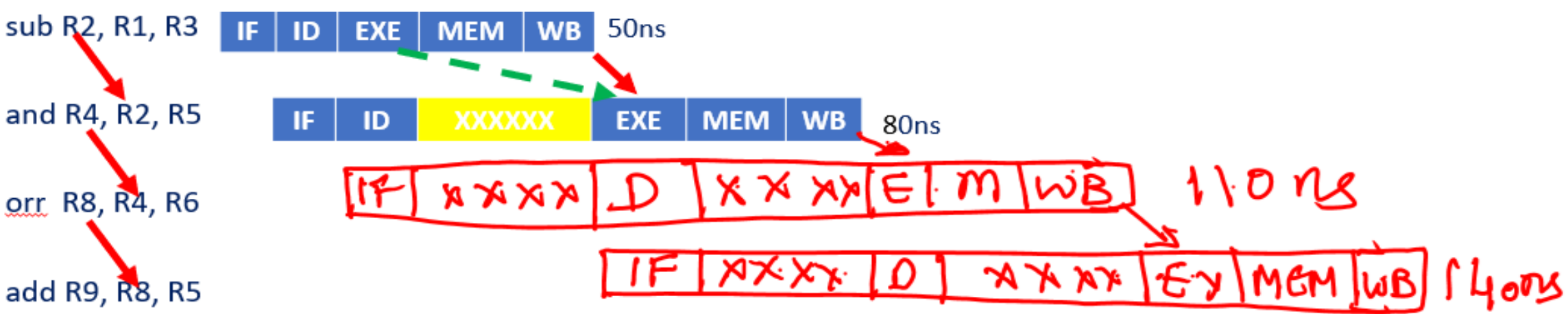
If Each stage take 10ns, the latency will be 50 ns

If Each stage take 10ns, the throughput is 80 ns instead of 200 ns

Pipelining, But What May Go Wrong?

- Is Good, When every instruction is independent of the other and can execute in the pipeline without any constraint
- But in application programs, most of the statements are dependent on each other
 - $C = A + B;$
 - $E = C * D;$
- This causes constraint on the pipeline to execute these dependent instructions in given 5 cycles.
 - Increases Latency

Pipelining, But What May Go Wrong?



If Each stage take 10ns, the latency will be 50 ns

If Each stage take 10ns, the throughput is 80 ns instead of 200 ns

With gap between stages, the throughput is 140 ns instead of 80ns

Pipelining, What May Go Wrong? Hazards

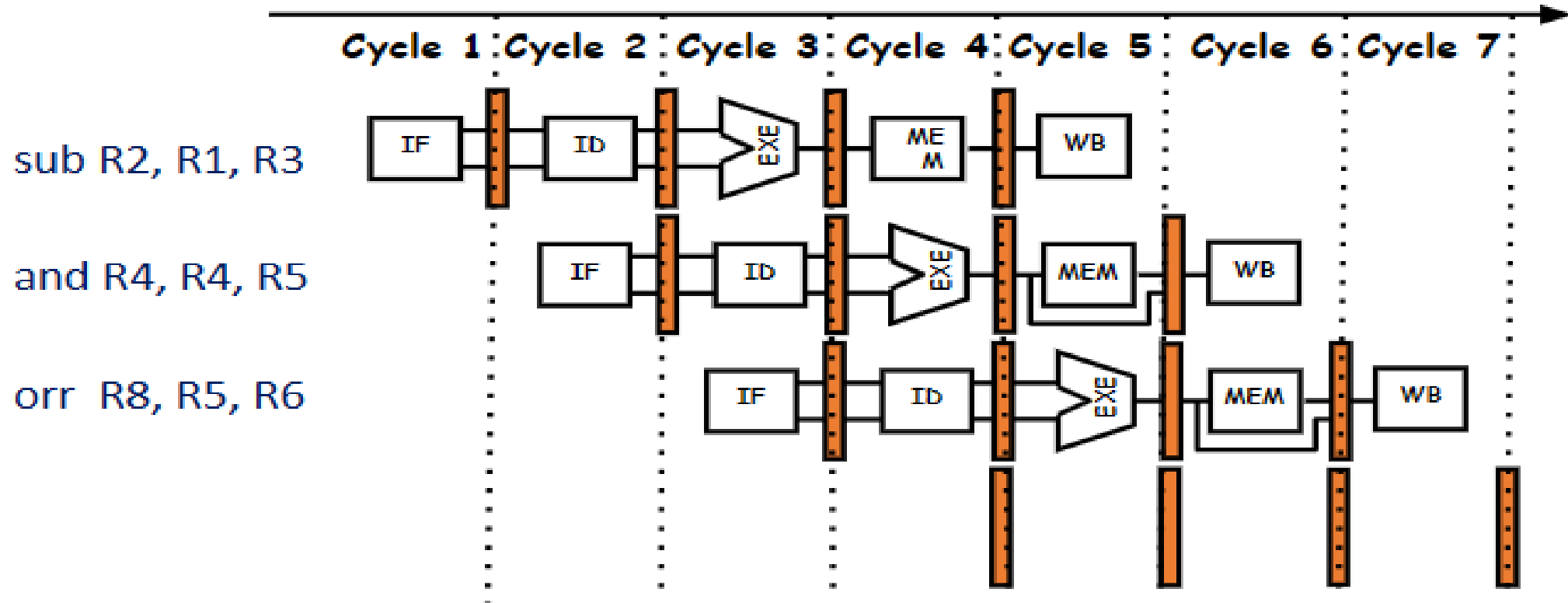
- Situations that prevent the next instruction in the instruction stream from executing during its designated cycle
- Condition that prevents an instruction in the pipe from executing its next scheduled pipe stage
- Where one instruction cannot immediately follow another
- Hazards reduce the performance from the ideal speedup gained by pipelining

Hazards: Limits to Pipelining

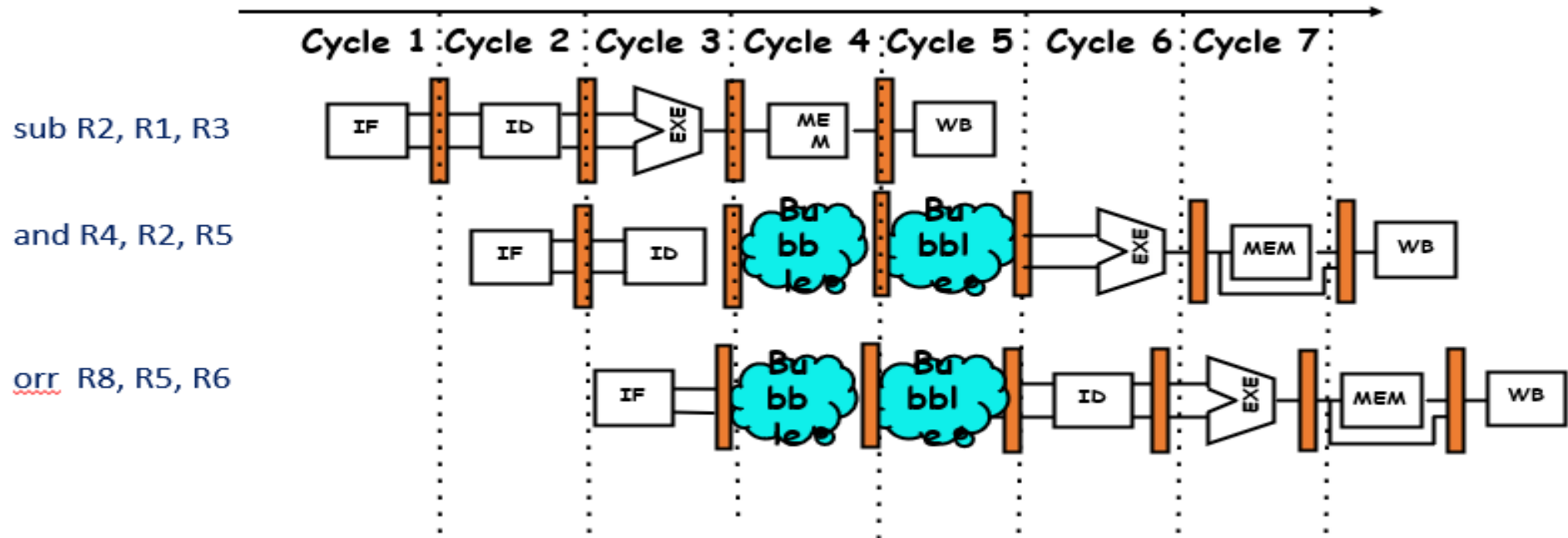
- **Structural Hazard:**
 - Due to structure of the pipeline
 - Hardware cannot support combination of instructions in the pipeline needing the same resources
- **Data Hazard:**
 - Due to data dependencies between instructions
 - Instruction depends on the result of prior instruction still in the pipeline
- **Control Hazard:**
 - Due to control instructions
 - Pipelining of branches & other instructions that change the PC

- Common solution for any type of hazard is to stall the pipeline until the hazard is resolved (Stall Cycles)
- This is achieved by inserting one or more “bubbles” in the pipeline
- To do this, hardware or software must detect that a hazard has occurred (Hazard Detection)

Stalls



Stalls



Stalls

Instruction Number	Clock number									
	1	2	3	4	5	6	7	8	9	10
sub R2, R1, R3	IF	ID	EX	MEM	WB					
and R4, R2, R5		IF	ID	stall	stall	EX	MEM	WB		
orr R8, R5, R6			IF	stall	stall	ID	EX	MEM	WB	

Disadvantage

- Stall cycle is a waste cycle
 - Opportunity wasted to execute one instruction
- Adds to pipeline delay and hence increases CPI
 - $(CPI > 1)$
 - Ex: If 30% are load/store, with only one memory; $CPI \geq 1.3$
- Not a preferred solution
- Affects performance of pipeline architecture

Performance of Pipelines with Stalls

A stall causes the pipeline performance to degrade the ideal performance.

$$\text{Speedup from pipelining} = \frac{\text{Average instruction time unpipelined}}{\text{Average instruction time pipelined}}$$

$$= \frac{\text{CPI}_{\text{unpipelined}} * \text{Clock Cycle Time}_{\text{unpipelined}}}{\text{CPI}_{\text{pipelined}} * \text{Clock Cycle Time}_{\text{pipelined}}}$$

$$= \frac{\text{CPI}_{\text{unpipelined}}}{1 + \text{Pipeline stall cycles per instruction}}$$

Note 1: Ignoring the cycle time overhead of pipelining and assume the stages are all perfectly balanced, then the cycle time of the two machines are equal

Note 2: Ideal CPI of pipeline processor is almost always =1

$$\begin{aligned} \text{CPI}_{\text{pipelined}} &= \text{Ideal CPI} + \text{Pipeline stall clock cycles per instruction} \\ &= 1 + \text{Pipeline stall clock cycles per instruction} \end{aligned}$$

Performance of Pipelines with Stalls

If all instructions take the same number of cycles, which must also equal the number of pipeline stages (the depth of the pipeline) then unpipelined CPI is equal to the depth of the pipeline

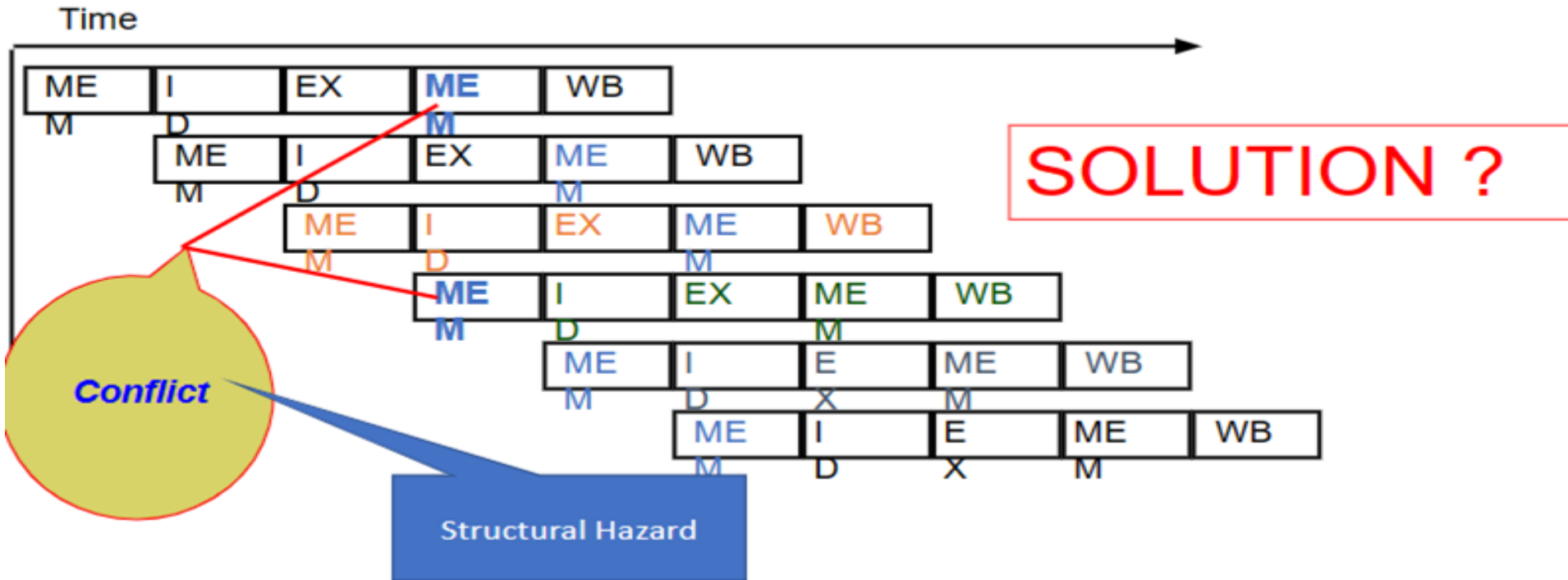
$$\text{Speedup} = \frac{\text{Pipeline depth}}{1 + \text{Pipeline stall cycles per instruction}}$$

If there are no pipeline stalls, this leads to the intuitive result that pipelining can improve performance by the depth of pipeline.

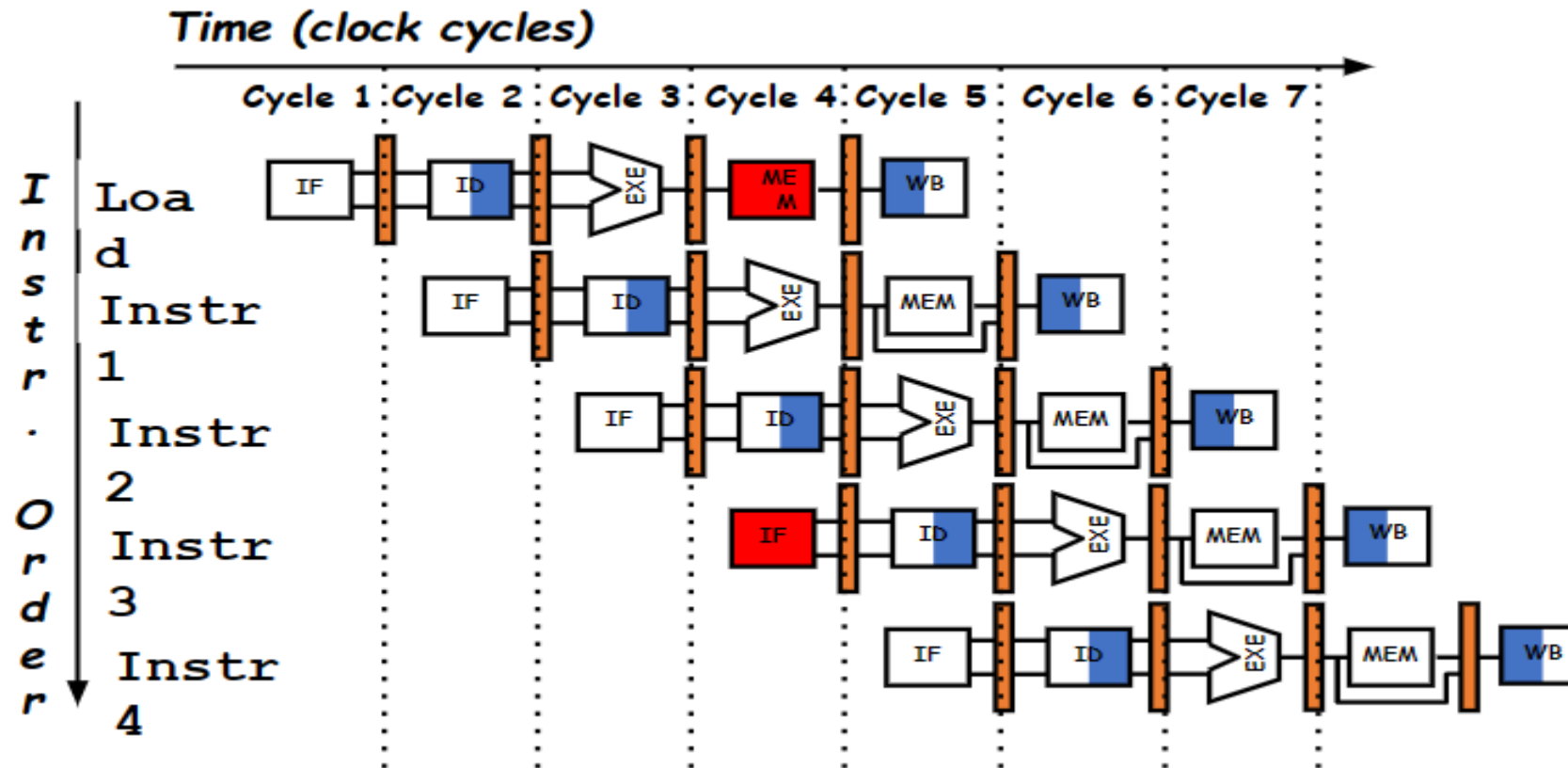
Structural Hazard

- If a **resource conflict** arises due to a hardware **resource** being **required by more than one instruction in a single cycle**, and one or more such instructions cannot be accommodated, then a structural hazard occurs, for example:
 - **when a pipelined machine has a shared single-memory for data and instructions.**
 - Unified Memory
 - **when a machine has only one register file write port**
 - Skip MEM stage for add/sub
 - **when a machine has overlapping of read and write**
 - Overlap ID & WB

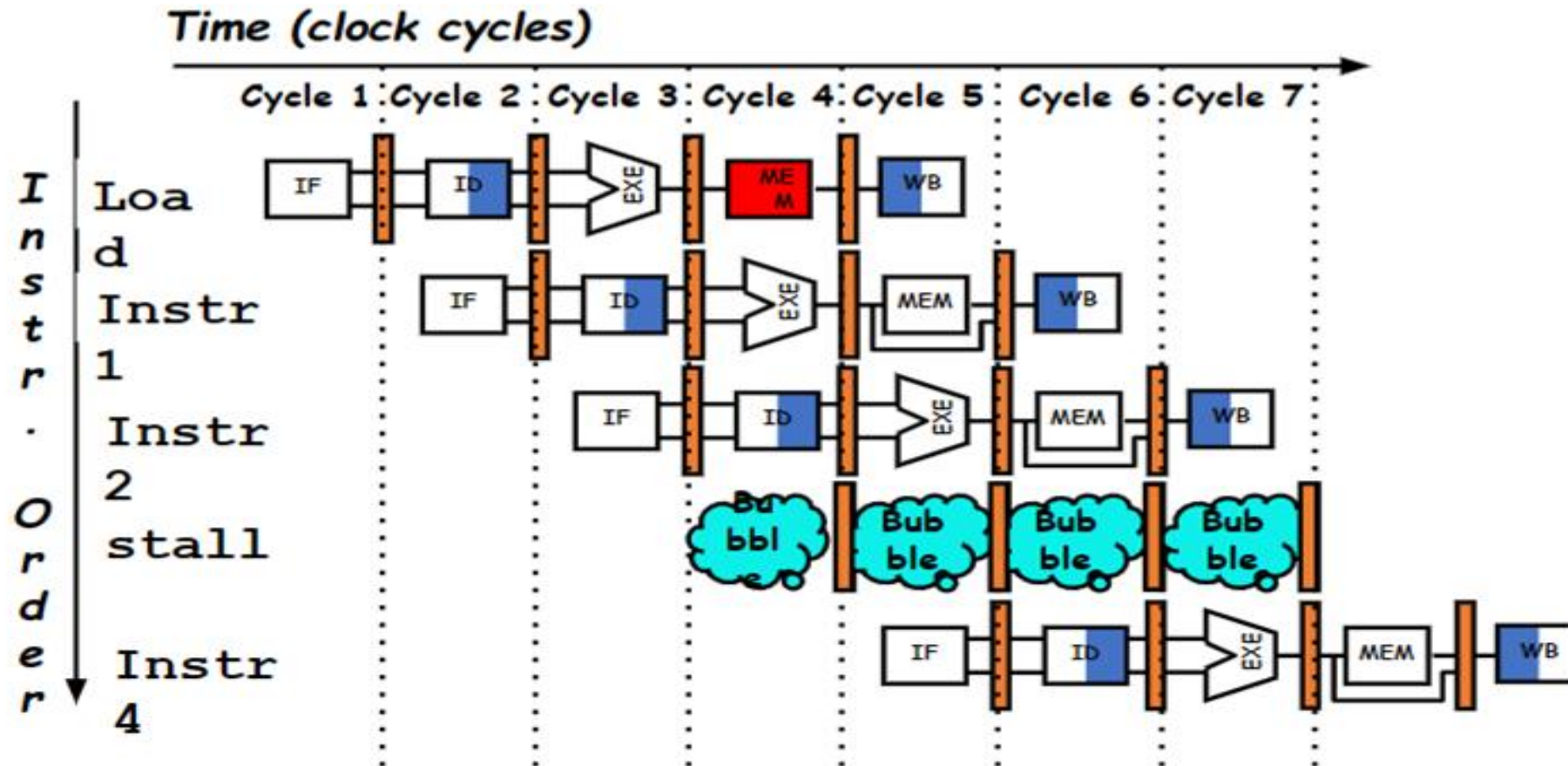
IF = MEM = Unified Mem operation



Structural Hazard: IF VS MEM:

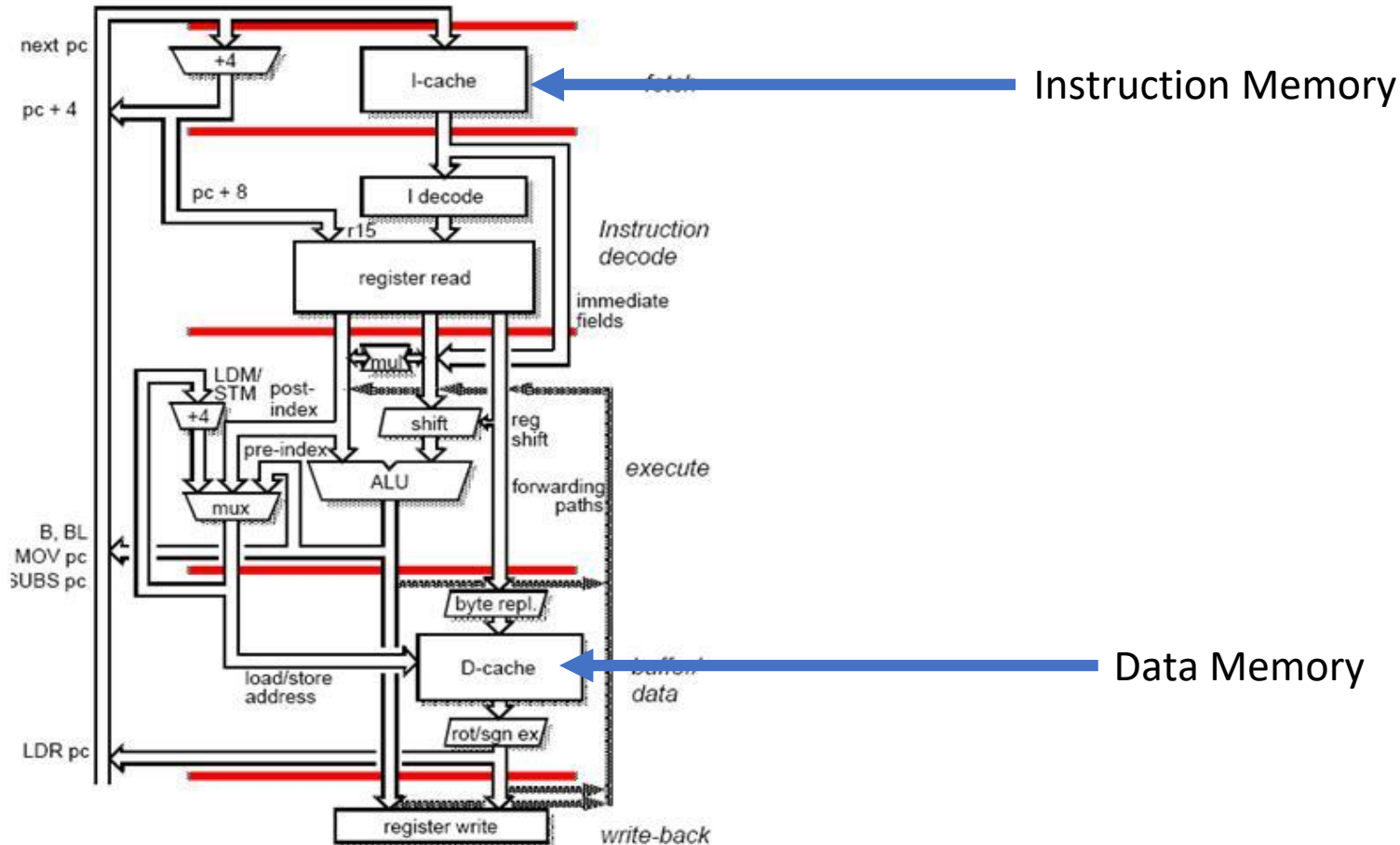


Structural Hazard: IF VS MEM:

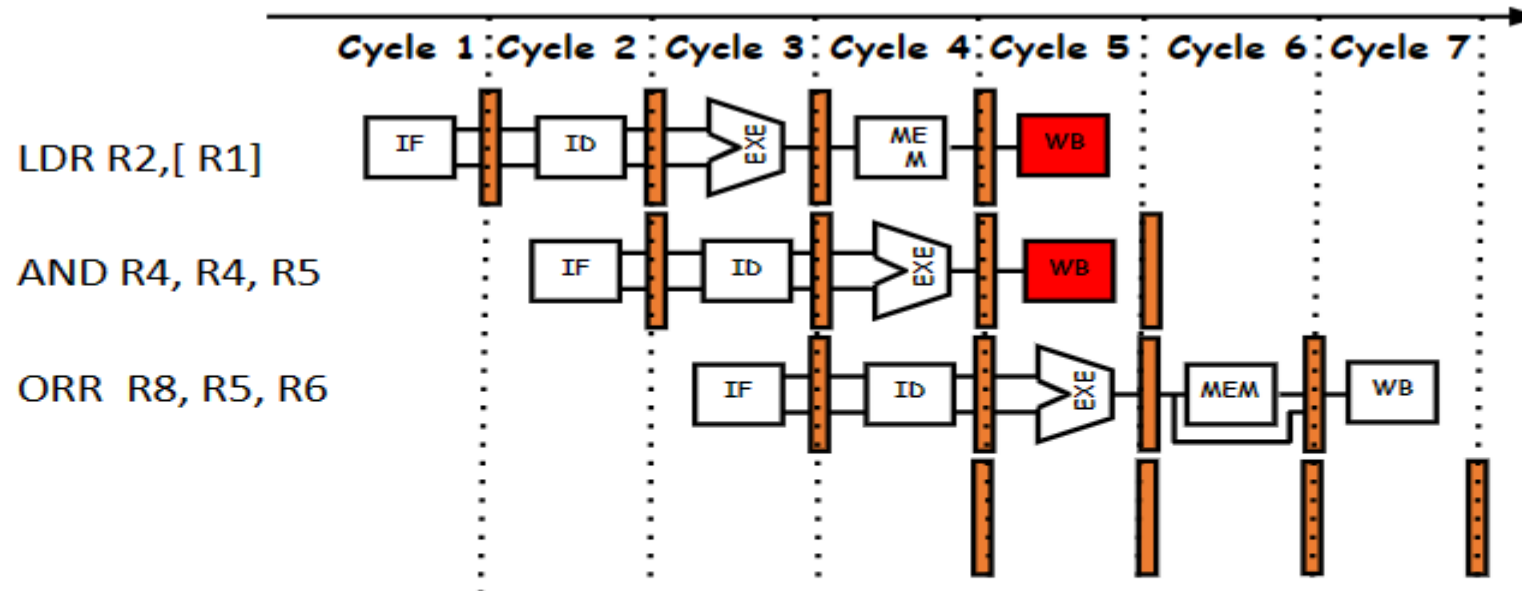


Structural Hazard: IF VS MEM: Solution

- Replication of resources
 - Split Memory (Instruction Memory & Data Memory)
 - Harvard Memory Architecture



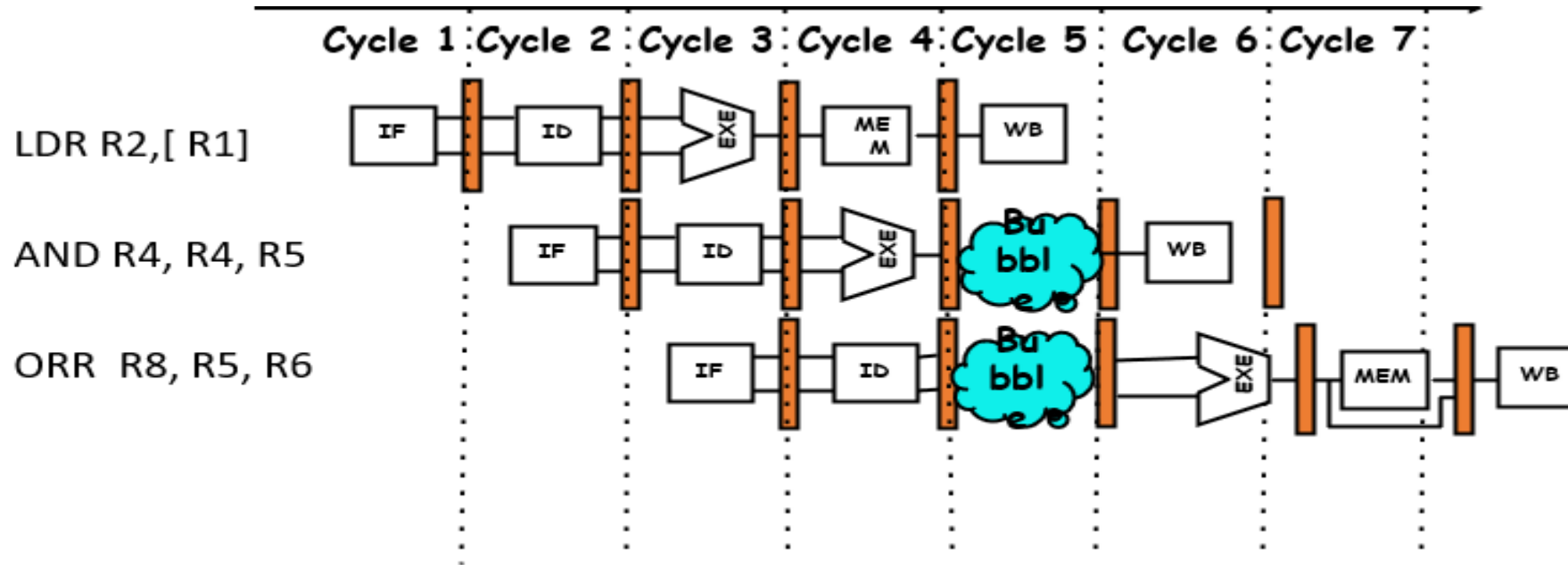
Structural Hazard: WB Vs WB:



2 ways to solve this pipeline hazard.

- Stall the stage
- Let all the instruction follow all stages, AND may take longer than usual

Structural Hazard: WB Vs WB: Solution 1

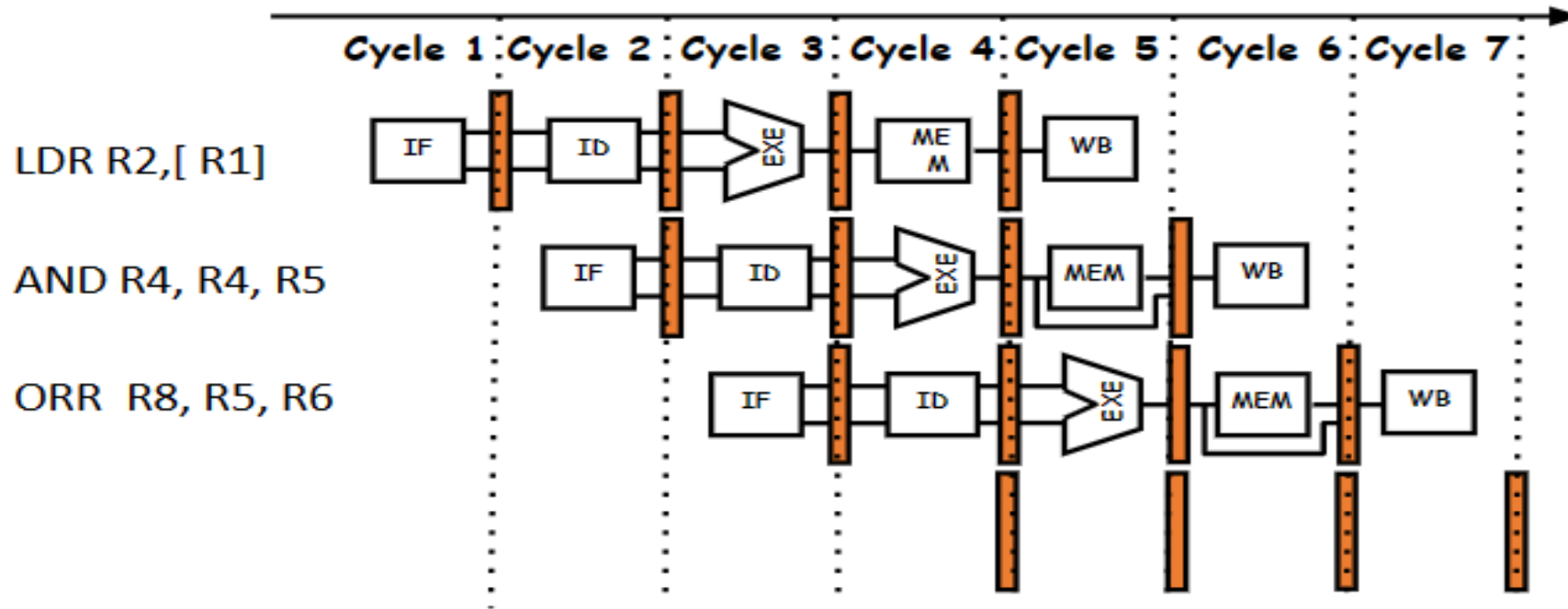


2 ways to solve this pipeline hazard.

°Stall the stage

°Let all the instruction follow all stages, AND may take longer than usual

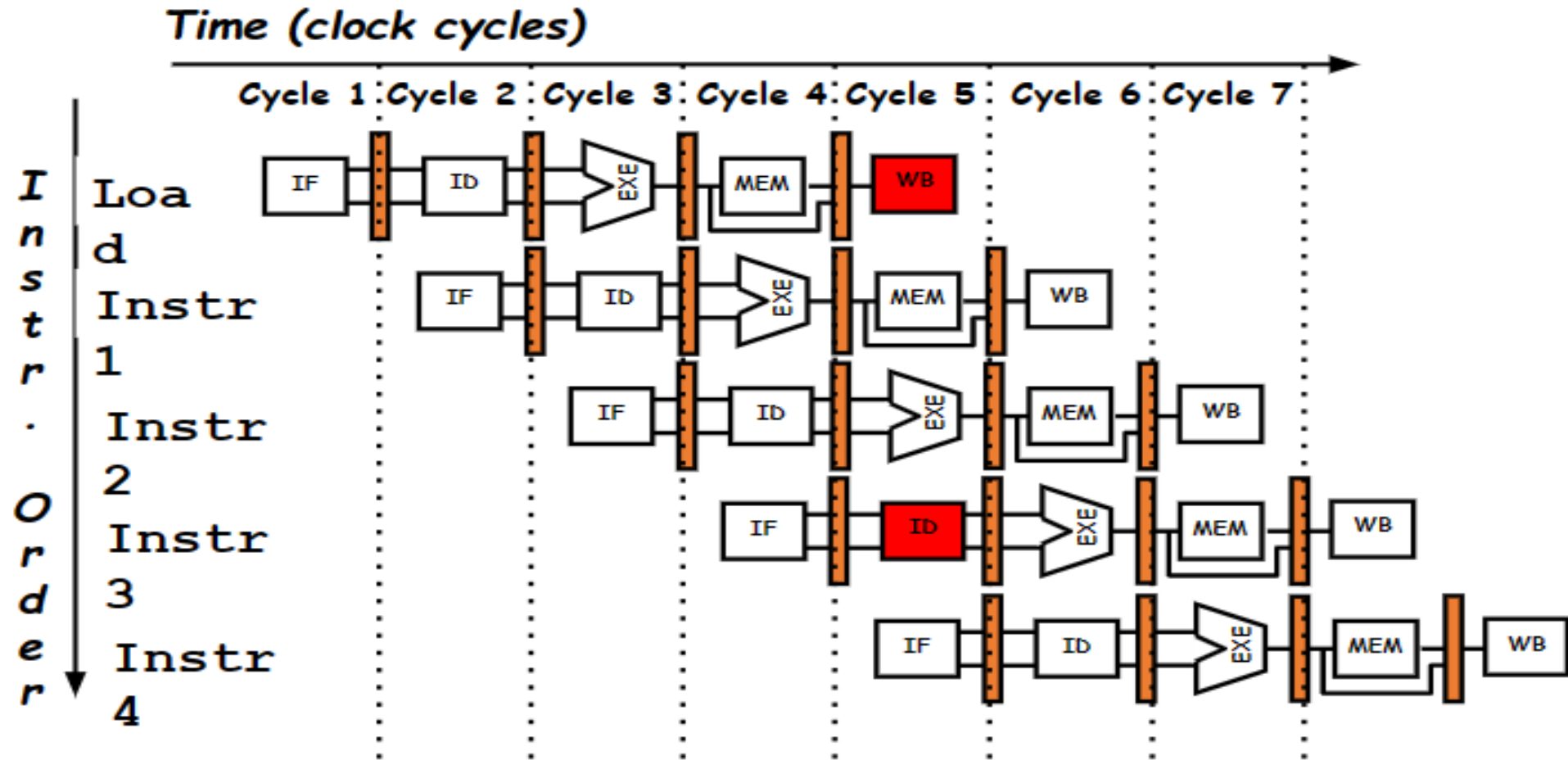
Structural Hazard: WB Vs WB: Solution 2



2 ways to solve this pipeline hazard.

- ° Stall the stage
- ° Let all the instruction follow all stages, AND may take longer than usual

Structural Hazard: ID VS WB



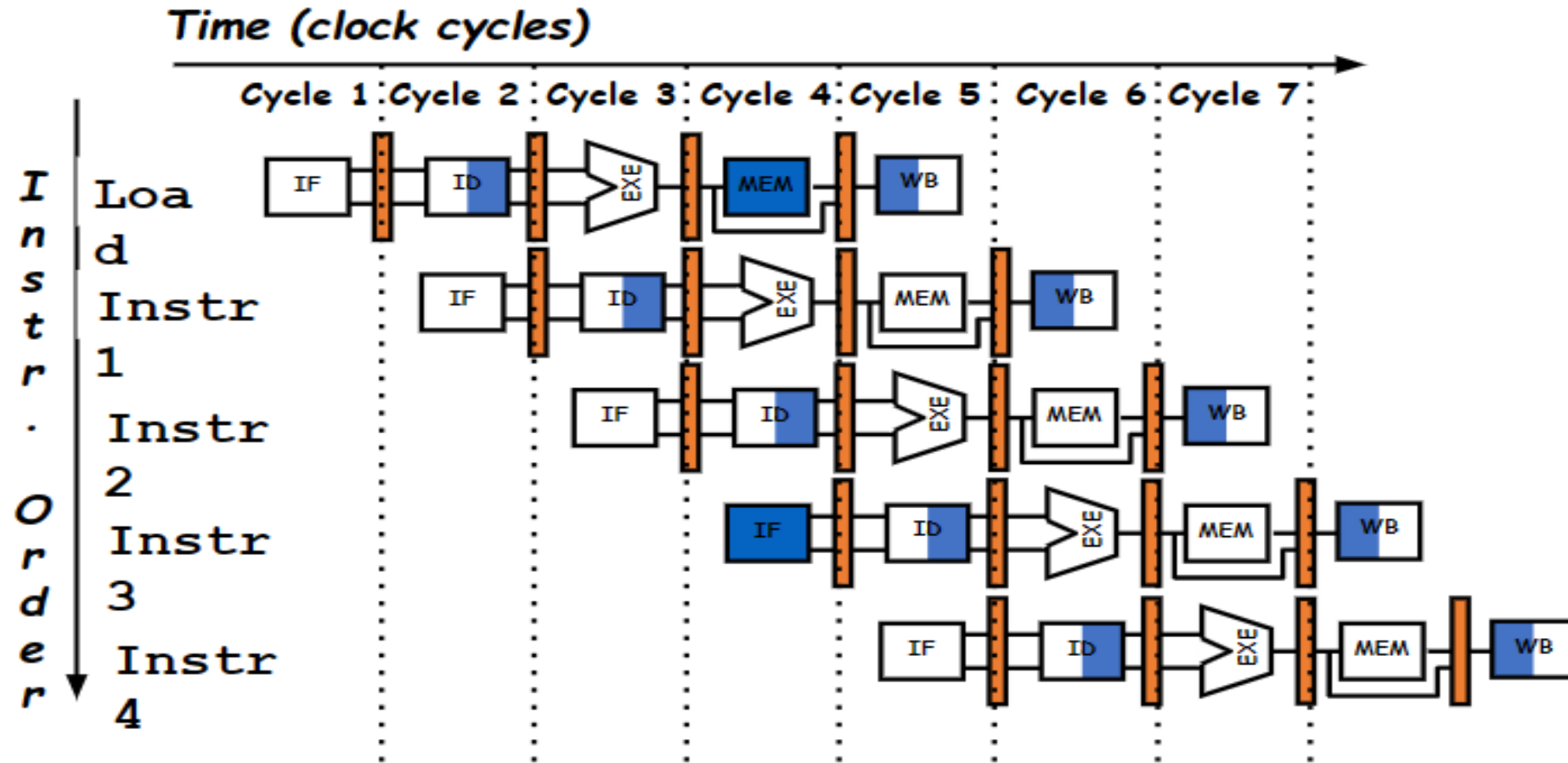
Structural Hazard: ID VS WB

Partitioning/Overlapping

1st half of the cycle Register Write

2nd half of the same cycle Register Read

Partitioning/Overlapping



Performance (with stalls)

- $n \gg k$; **Speedup = k**; no. of stages (also known as Depth of Pipeline)
- *For 5-stage pipeline with stalls:*
- $Speed\ up = \frac{Depth\ of\ Pipeline}{CPI} = 5$
- $Speed\ up = \frac{Depth\ of\ Pipeline}{1 + Pipeline\ Stall\ cycles\ per\ instruction}$
- If stalls are 0, then Speedup = k = 5
- If say, 30% are Load/Store in a unified memory pipeline, then every load/store overlap with IF will introduce 3 stalls. Thus: $CPI = 1 + (3 \times 0.3) = 1.9$
- $Speed\ up = \frac{5}{1 + (3 \times 0.3)} = 2.63$

1. A pipelined processor has 5 stages with an ideal $CPI = 1$. Due to data hazards, the pipeline incurs 0.4 stall cycles per instruction. Assuming perfectly balanced stages and no clock overhead, what is the speedup over a non-pipelined processor?

- A) 2.57
- B) 3.57
- C) 4.23
- D) 5.17

2. What is the most common generic solution used to handle any type of pipeline hazard?

- A) Increasing clock frequency
- B) Instruction reordering only by software
- C) Stalling the pipeline by inserting bubbles
- D) Increasing pipeline depth

1. A pipelined processor has 5 stages with an ideal CPI = 1. Due to data hazards, the pipeline incurs 0.4 stall cycles per instruction. Assuming perfectly balanced stages and no clock overhead, what is the speedup over a non-pipelined processor?

- A) 2.57
- B) 3.57
- C) 4.23
- D) 5.17

ANS- B) 3.57

2. What is the most common generic solution used to handle any type of pipeline hazard?

- A) Increasing clock frequency
- B) Instruction reordering only by software
- C) Stalling the pipeline by inserting bubbles
- D) Increasing pipeline depth

ANS- C) Stalling the pipeline by inserting bubbles

3. Which of the following is a valid solution to eliminate the IF vs MEM structural hazard in a pipelined processor?

- A) Increasing pipeline depth
- B) Introducing branch prediction
- C) Adding extra pipeline bubbles
- D) Splitting unified memory into instruction and data memory

4. Which of the following best defines a pipeline hazard?

- A) A condition that reduces the clock frequency of the processor
- B) A situation where an instruction finishes earlier than expected
- C) A condition that prevents the next instruction in the instruction stream from executing in its designated clock cycle
- D) A hardware failure in the pipeline stages

3. Which of the following is a valid solution to eliminate the IF vs MEM structural hazard in a pipelined processor?

- A) Increasing pipeline depth
- B) Introducing branch prediction
- C) Adding extra pipeline bubbles
- D) Splitting unified memory into instruction and data memory

ANS- D) Splitting unified memory into instruction and data memory

4. Which of the following best defines a pipeline hazard?

- A) A condition that reduces the clock frequency of the processor
- B) A situation where an instruction finishes earlier than expected
- C) A condition that prevents the next instruction in the instruction stream from executing in its designated clock cycle
- D) A hardware failure in the pipeline stages

ANS- C) A condition that prevents the next instruction in the instruction stream from executing in its designated clock cycle



THANK YOU



Team MPCA

Department of Computer Science and
Engineering



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Session - 2.4

Microprocessor & Computer Architecture (μ pCA)



UE24CS251B

Pipeline Processor: Data Hazard 1

Data Hazards

Consider a Scenario

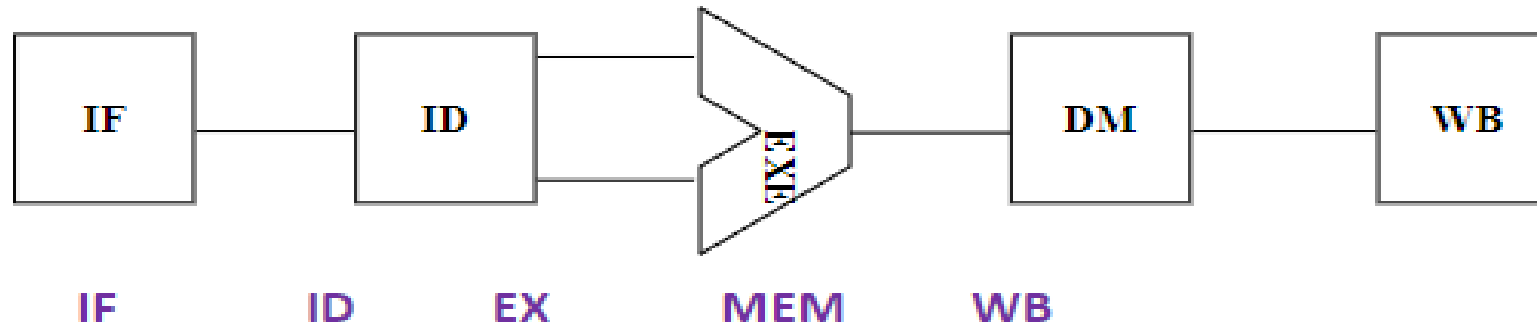
Suppose initially, register *i* holds the number 2i
(ie. R3 = 6; R6=12; R8 = 16...)

What happens when the following sequence is executed in 5-stage pipeline?

- | | |
|--------------------------|--|
| add R3, R10, R11 | - this should add 20 + 22, putting result 42 into r3 |
| ldr R8, [R3, #50] | - this should load r8 from memory location 42+50 = 92 |
| sub R11, R8, R7 | - this should subtract 14 from that just-loaded value from Mem[92] |

Cycle 1

add R3, R10, R11



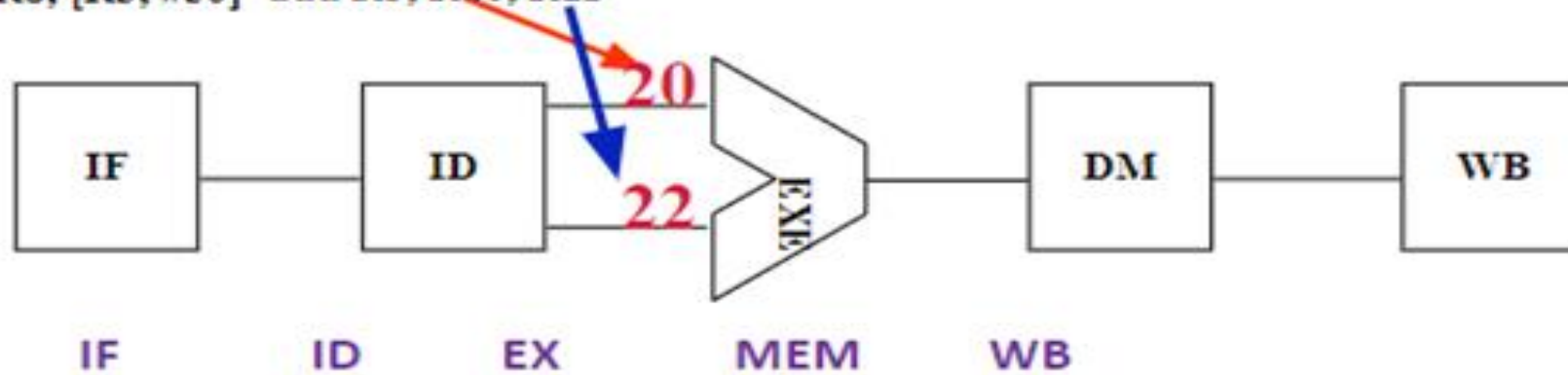
add R3, R10, R11 - this should add 20 + 22, putting result 42 into r3

ldr R8, [R3, #50] - this should load r8 from memory location $42 + 50 = 92$

sub R11, R8, R7 - this should subtract 14 from that just-loaded value from Mem[92]

Cycle 2

ldr R8, [R3, #50] add R3, R10, R11



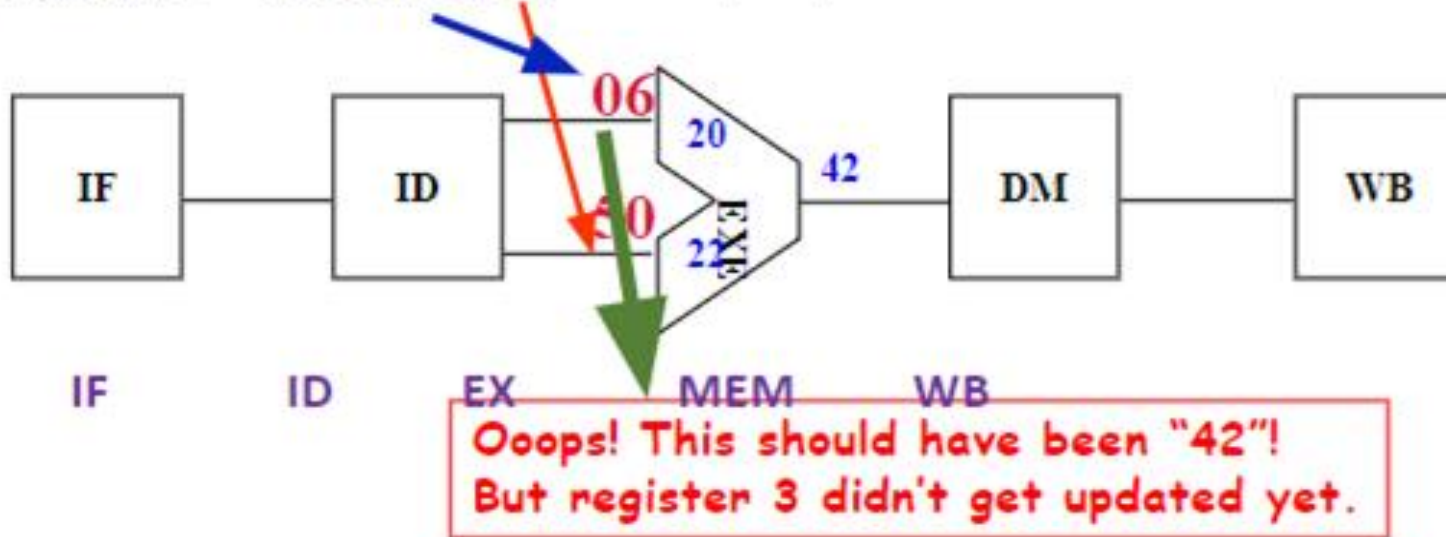
add R3, R10, R11 - this should add 20 + 22, putting result 42 into r3

ldr R8, [R3, #50] - this should load r8 from memory location $42 + 50 = 92$

sub R11, R8, R7 - this should subtract 14 from that just-loaded value
from Mem[92]

Cycle 3

sub R11, R8, R7 ldr R8, [R3, #50] add R3, R10, R11

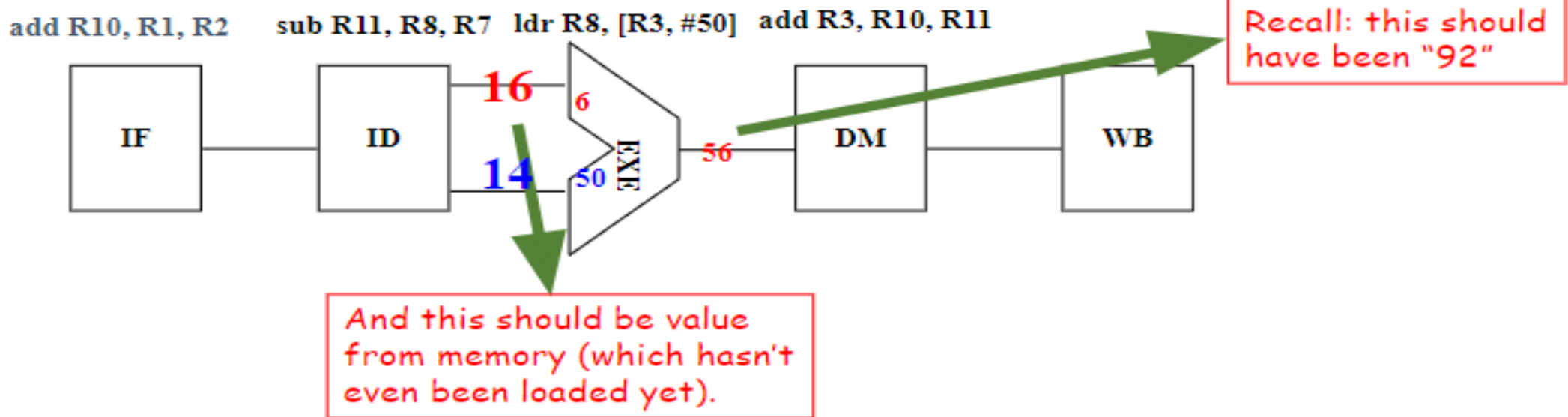


add R3, R10, R11 - this should add 20 + 22, putting result 42 into r3

ldr R8, [R3, #50] - this should load r8 from memory location $42 + 50 = 92$

sub R11, R8, R7 - this should subtract 14 from that just-loaded value from Mem[92]

Cycle 4



add R3, R10, R11 - this should add 20 + 22, putting result 42 into r3

ldr R8, [R3, #50] - this should load r8 from memory location 42+50 = 92

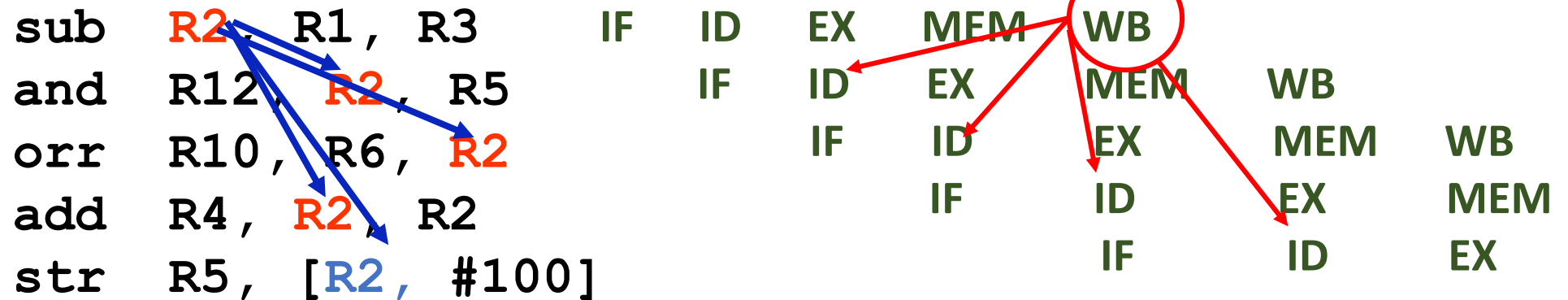
sub R11, R8, R7 - this should subtract 14 from that just-loaded value from Mem[92]

Problem with starting next instruction before first is finished

- Data dependencies here, that "go backward in time" create data hazards.

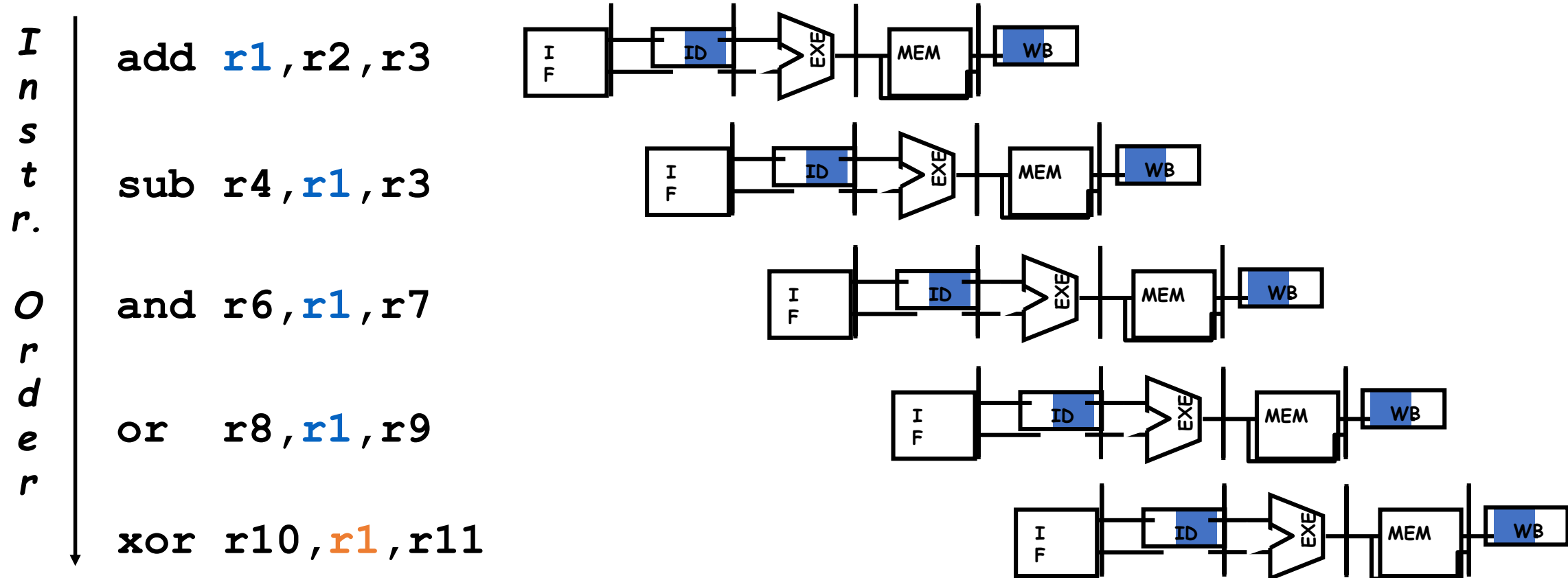
EXAMPLE

- Consider the following sequence:



DEPTH OF HAZARD = 3 cycles

Data Dependency



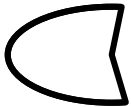
Data Dependencies

- Three types of data dependencies defined in terms of how succeeding instruction depends on preceding instruction
 - **RAW:** Read after Write or Flow dependency
(True Dependency)
 - **WAR:** Write after Read or anti-dependency
(Anti Dependency)
 - **WAW:** Write after Write
(Output Dependency)

Data Dependencies

- Read After Write (RAW)

Instr_j tries to read operand before Instr_i writes it

 I: add r1, r2, r3
J: sub r4, r1, r3

- Caused by a “**Dependence**” (in compiler nomenclature). This hazard results from an actual need for communication.

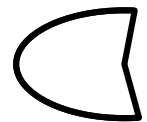
Data Dependencies

- Example program (a) with two instructions
 - i1: load **r1**, a;
 - i2: add r2, **r1**, r1;
- Program (b) with two instructions
 - i1: mul **r1**, r4, r5;
 - i2: add r2, **r1**, r1;
- Both cases we cannot read in i2 until i1 has completed writing the result
 - In **(a)** this is due to load-use dependency
 - In **(b)** this is due to define-use dependency

Data Dependencies

- Write After Read (WAR)

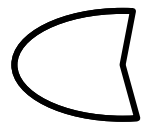
Instr_j writes operand before Instr_i reads it

 I: sub r4, r1, r3
J: add r1, r2, r3
K: mul r6, r1, r7

- Called an “anti-dependence” by compiler writers. This results from reuse of the name “r1”.
- Can’t happen if in pipeline:-
 - All instructions take 5 stages, and
 - Reads are always in stage 2, and
 - Writes are always in stage 5

Data Dependencies

- Write After Write (WAW)
Instr_j writes operand before Instr_i writes it.
- Called an “output dependence” by compiler writers
This also results from the reuse of name “r1”.
- Can’t happen if in pipeline:
- All instructions take 5 stages, and Writes are always in stage 5



```
I: sub r1, r4, r3  
J: add r1, r2, r3  
K: mul r6, r1, r7
```

If Executed Out of order by the Compiler, may lead to wrong results

WAR and WAW Dependency

- Example program (a):
 - i1: mul r1, r2, r3;
 - i2: add r2, r4, r5;
- Example program (b):
 - i1: mul r1, r2, r3;
 - i2: add r1, r4, r5;
- both cases we have dependence between i1 and i2
 - in (a) due to r2 must be read before it is written into
 - in (b) due to r1 must be written by i2 after it has been written into by i1

WAR and WAW Dependency

- Problem:
 - i1: mul r1, r2, r3;
 - i2: add r2, r4, r5;
- Is this really a dependence/hazard ?

WAR and WAW Dependency

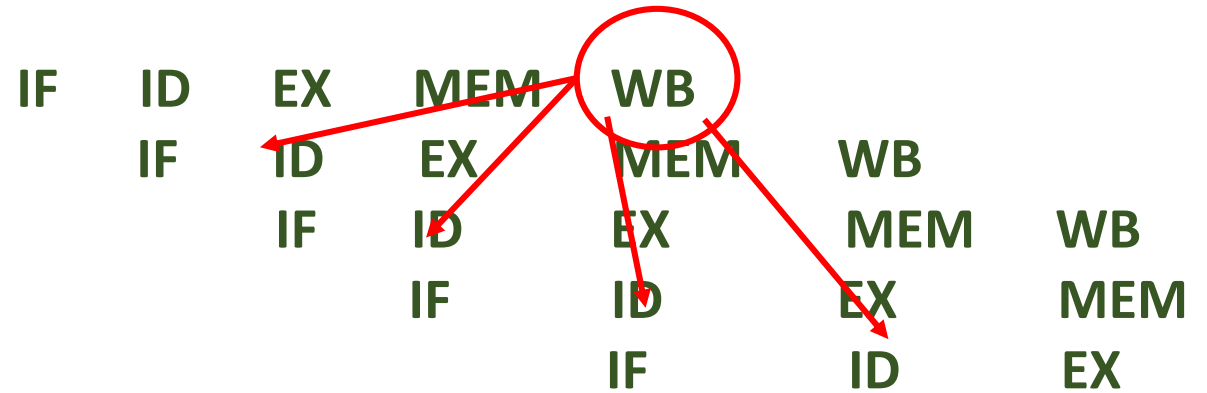
- Solution: Rename Registers
 - i1: mul r1, r2, r3;
 - i2: add r6, r4, r5;
- Register renaming can solve many of these false dependencies
 - note the role that the compiler plays in this
 - specifically, the register allocation process--i.e., the process that assigns registers to variables

EXAMPLE

- Consider the following sequence:

```

sub  R2, R1, R3
and  R12, R2, R5
orr  R10, R6, R2
add  R4, R2, R2
str  R5, [R2, #100]
  
```



DEPTH OF HAZARD = 3 cycles

1. Based on the 5-stage pipeline, why does the instruction sequence ADD R3, R10, R11 followed immediately by LDR R8, [R3, #50] cause a data hazard?
 - A. The LDR instruction tries to write to register R3 before the add instruction reads it.
 - B. The ADD instruction requires a value from memory that has not yet been loaded.
 - C. The LDR instruction tries to read register R3 in the pipeline before the add instruction has written the updated value to it.
 - D. The pipeline stalls because both instructions are trying to access the ALU at the same time.

2. Which type of data dependency is explicitly referred to as a "True Dependency" or "Flow Dependency" in the course material?
 - A. WAW – Write after Write
 - B. RAW – Read after Write
 - C. WAR – Write after Read
 - D. Load-Use Dependency

1. Based on the 5-stage pipeline, why does the instruction sequence ADD R3, R10, R11 followed immediately by LDR R8, [R3, #50] cause a data hazard?
 - A. The LDR instruction tries to write to register R3 before the add instruction reads it.
 - B. The ADD instruction requires a value from memory that has not yet been loaded.
 - C. The LDR instruction tries to read register R3 in the pipeline before the add instruction has written the updated value to it.**
 - D. The pipeline stalls because both instructions are trying to access the ALU at the same time.

2. Which type of data dependency is explicitly referred to as a "True Dependency" or "Flow Dependency"?
 - A. WAW – Write after Write
 - B. RAW – Read after Write**
 - C. WAR – Write after Read
 - D. Load-Use Dependency

3. Why do Write After Read (WAR) and Write After Write (WAW) hazards NOT happen in the specific pipeline described?

- A. Because the compiler automatically reorders all instructions to avoid them.
- B. Because instructions are executed out of order.
- C. Because all instructions take 5 stages, with reads always in stage 2 and writes always in stage 5.
- D. Because the hardware uses data forwarding to eliminate these specific hazards.

4. Consider the following instruction sequence:

i1: MUL r1, r2, r3

i2: ADD r1, r4, r5

What type of dependency exists between these two instructions according to the slides?

- A. RAW – Read after Write
- B. WAR – Write after Read
- C. WAW – Write after Write
- D. No Dependency

3. Why do Write After Read (WAR) and Write After Write (WAW) hazards NOT happen in the specific pipeline described?

A. Because the compiler automatically reorders all instructions to avoid them.

B. Because instructions are executed out of order.

C. Because all instructions take 5 stages, with reads always in stage 2 and writes always in stage 5.

D. Because the hardware uses data forwarding to eliminate these specific hazards.

4. Consider the following instruction sequence:

i1: MUL r1, r2, r3

i2: ADD r1, r4, r5

What type of dependency exists between these two instructions?

A. RAW – Read after Write

B. WAR – Write after Read

C. WAW – Write after Write

D. No Dependency



THANK YOU



Team MPCA

Department of Computer Science and
Engineering



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Session - 2.5

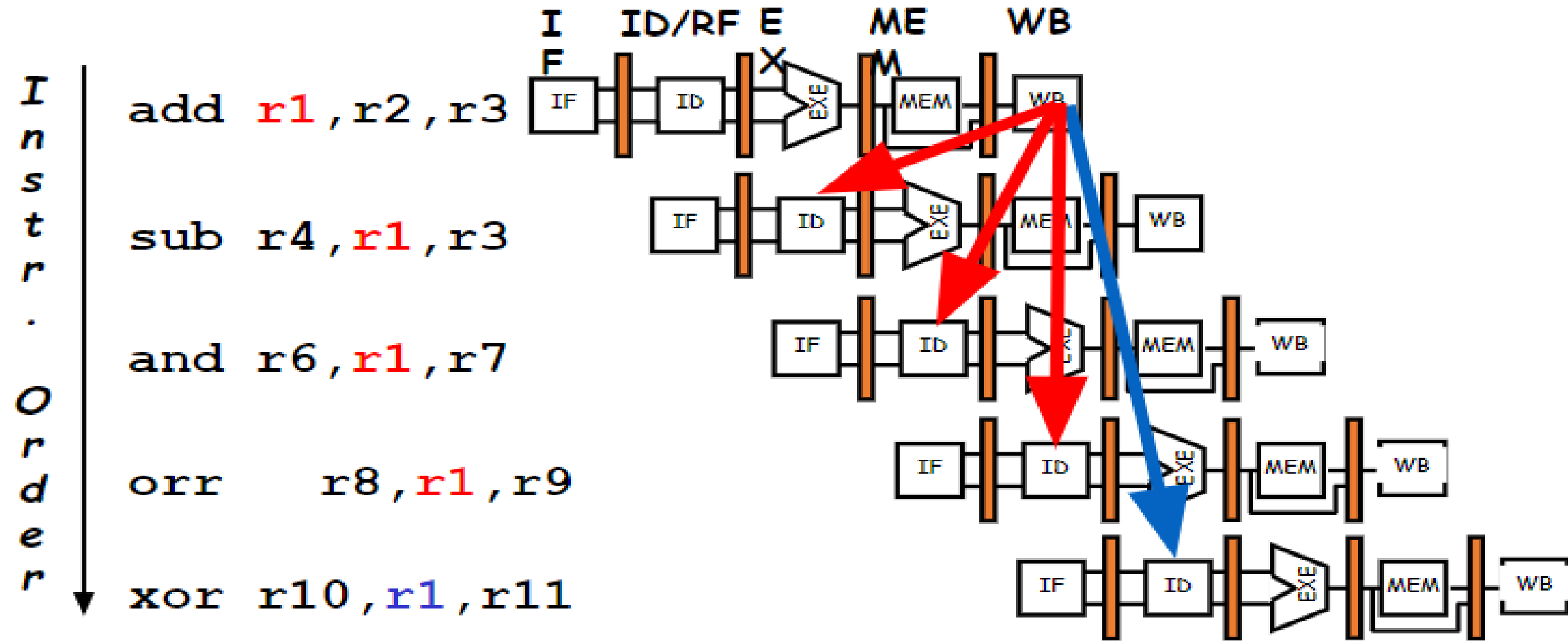
Microprocessor & Computer Architecture (μ pCA)



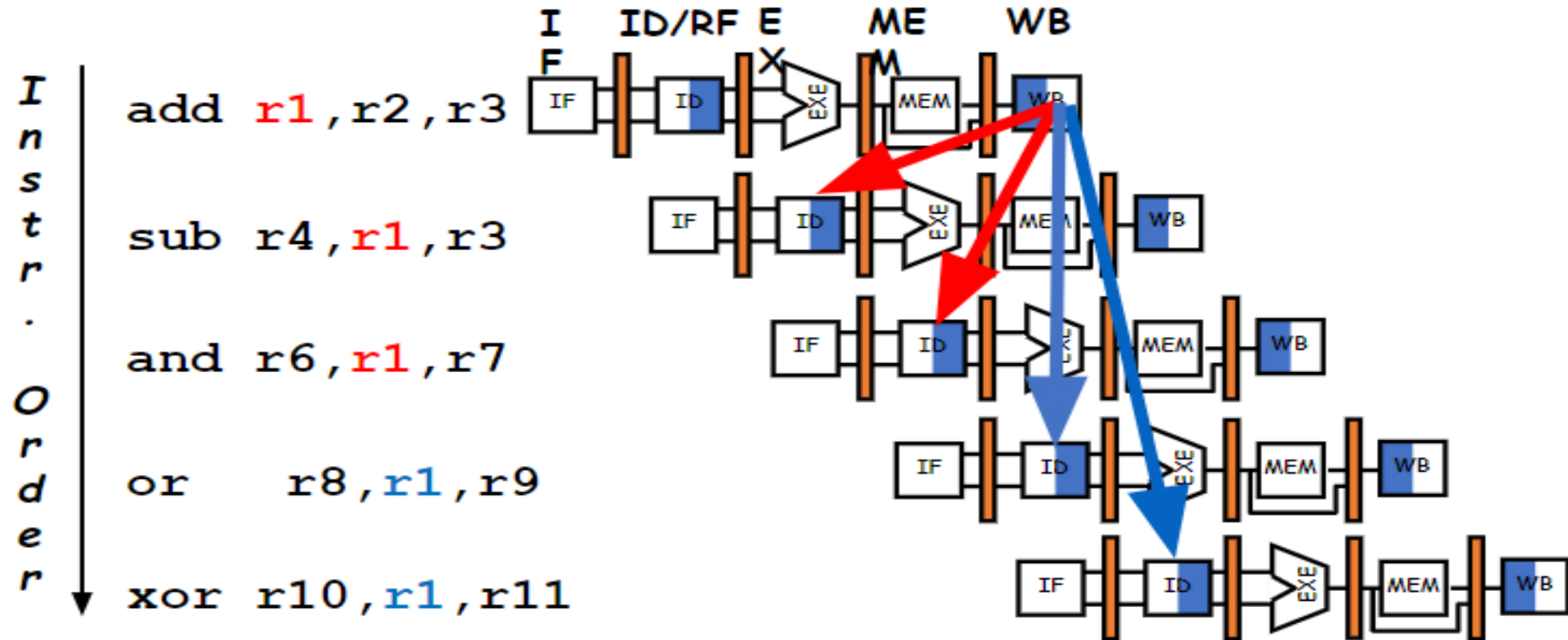
UE24CS251B

Pipeline Processor: Data Hazard 2

How to Attack RAW Dependency?



How to Attack RAW Dependency?



Attacking Data Hazard

- In Software
 - **Solution 1: Re-order instructions**
 - Solution 2: insert independent instructions (or no-ops) Ex: MOV R0, R0
- In Hardware
 - Solution 1: Insert bubbles (i.e. stall the pipeline)
 - Solution 2: Data Forwarding

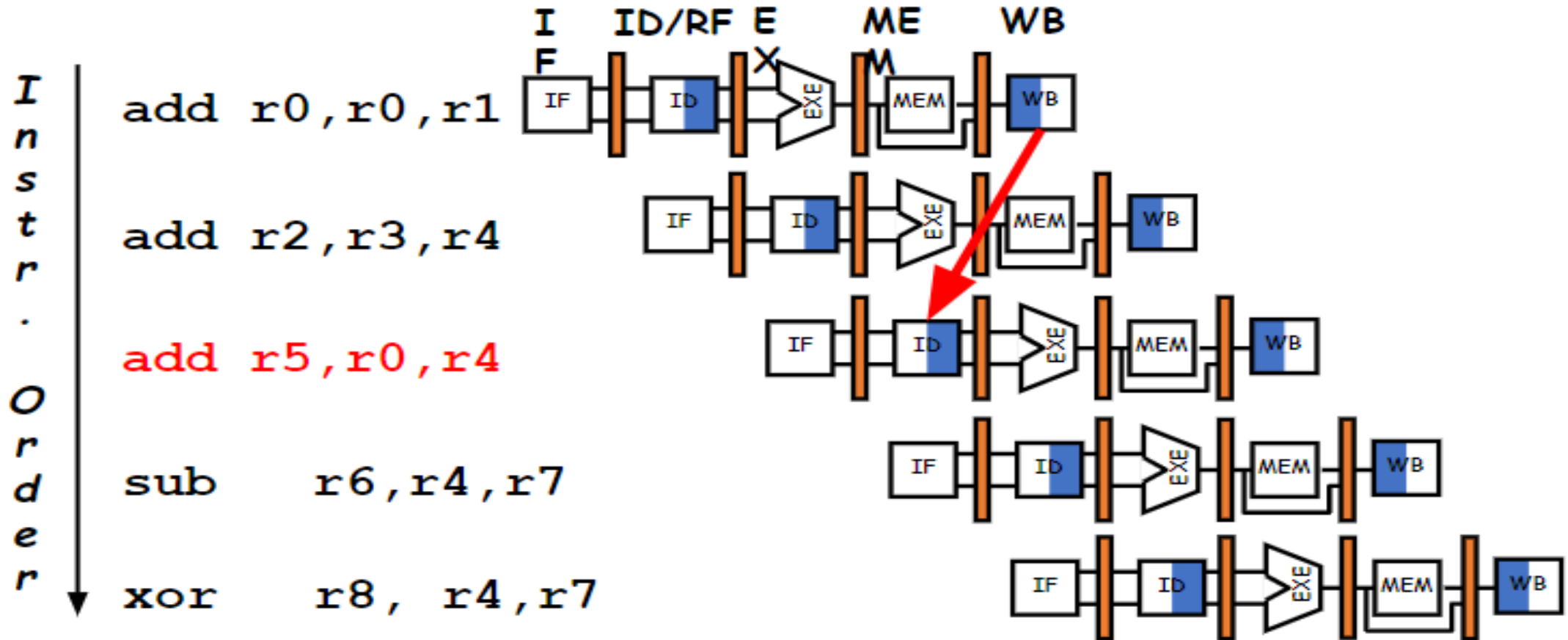
Software Solution 1: By Compiler

ADD R0,R0,R1
ADD R2,R3,R4
ADD R5,R0,R4
SUB R6,R4,R7
XOR R8,R4,R7

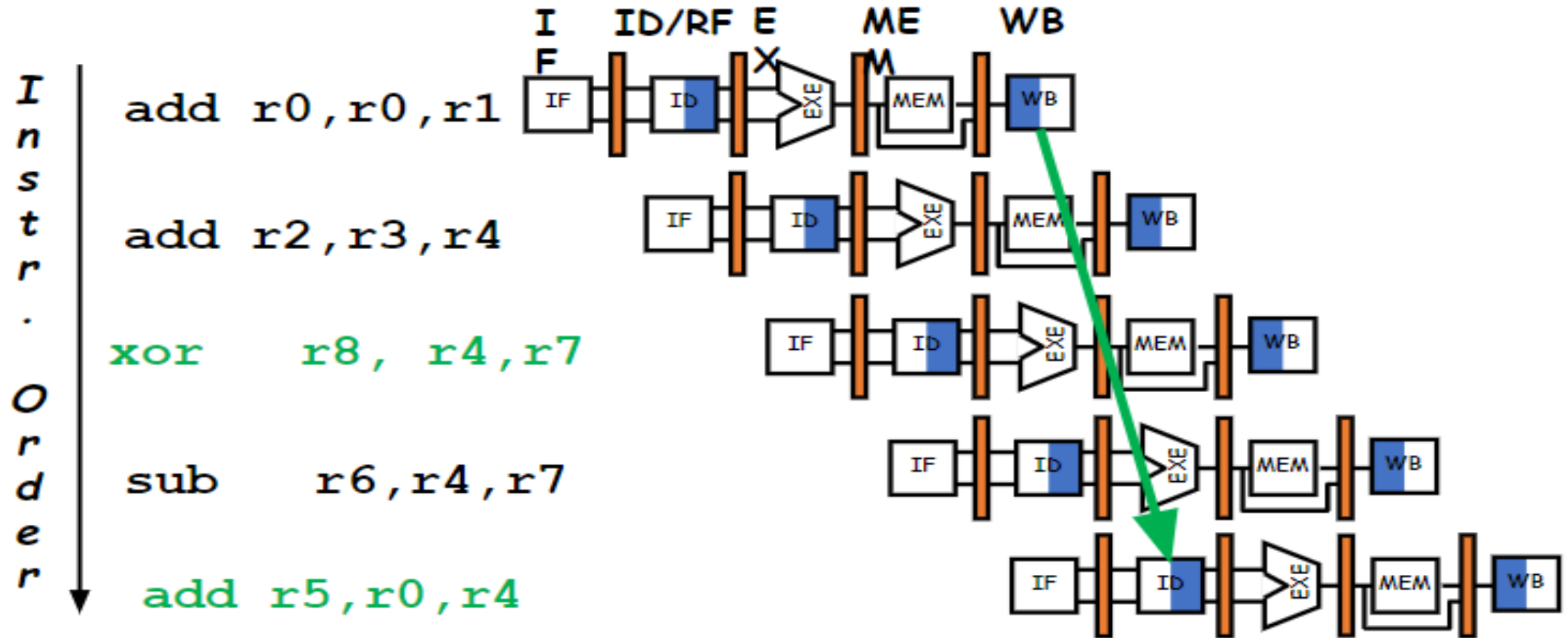
ADD R0,R0,R1
ADD R2,R3,R4
SUB R6,R4,R7
XOR R8,R4,R7
ADD R5,R0,R4

Out of Order Execution by Compiler

Software Solution 1: By Compiler



Software Solution 1: By Compiler



Attacking Data Hazard

- In Software
 - Solution 1: Re-order instructions
 - **Solution 2: Insert independent instructions (or no-ops) Ex: MOV R0, R0**
- In Hardware
 - Solution 1: Insert bubbles (i.e. stall the pipeline)
 - Solution 2: Data Forwarding

Software Solution 2: By Compiler

Insert NOP or MOV R0, R0

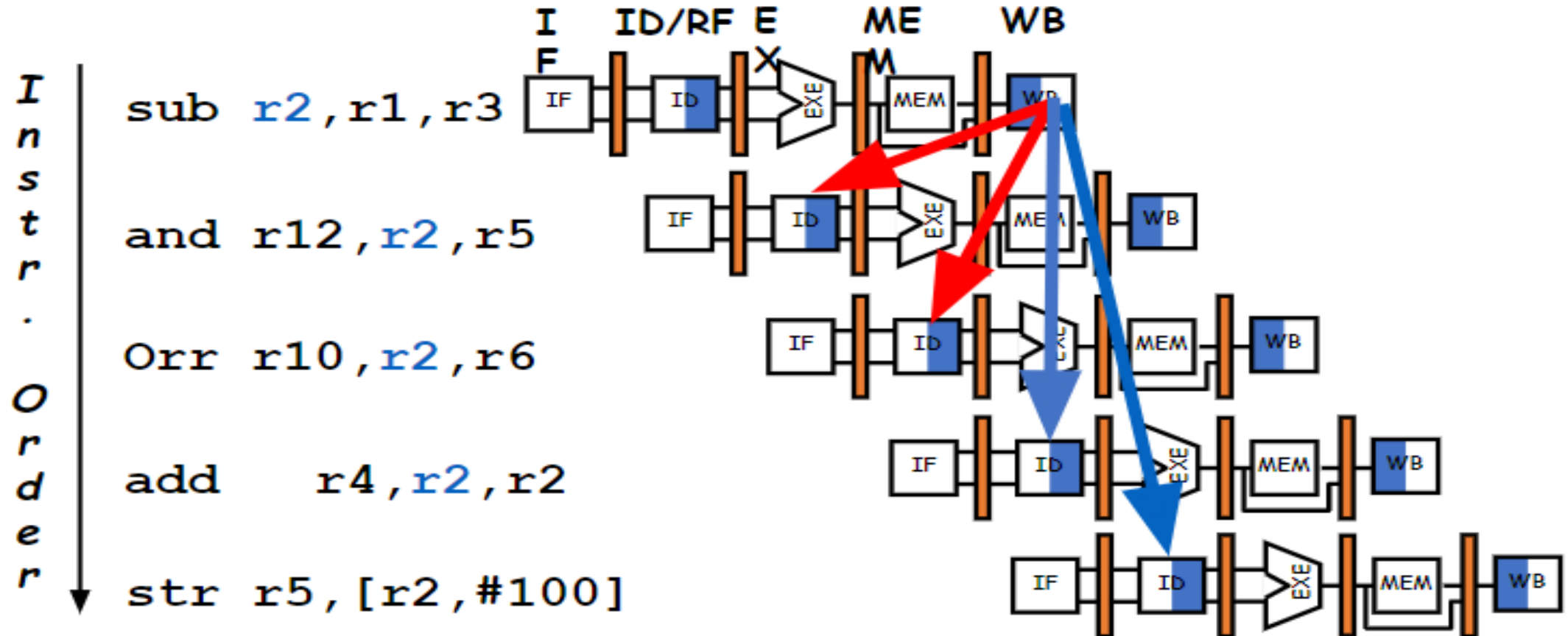
- The compiler can guarantee that no data hazards exist adding NOP or MOV instructions where needed. [Instruction Scheduling]

```
sub  R2, R1, R3
and  R12, R2, R5
orr  R10, R6, R2
add  R4, R2, R2
str  R5, [R2, #100]
```

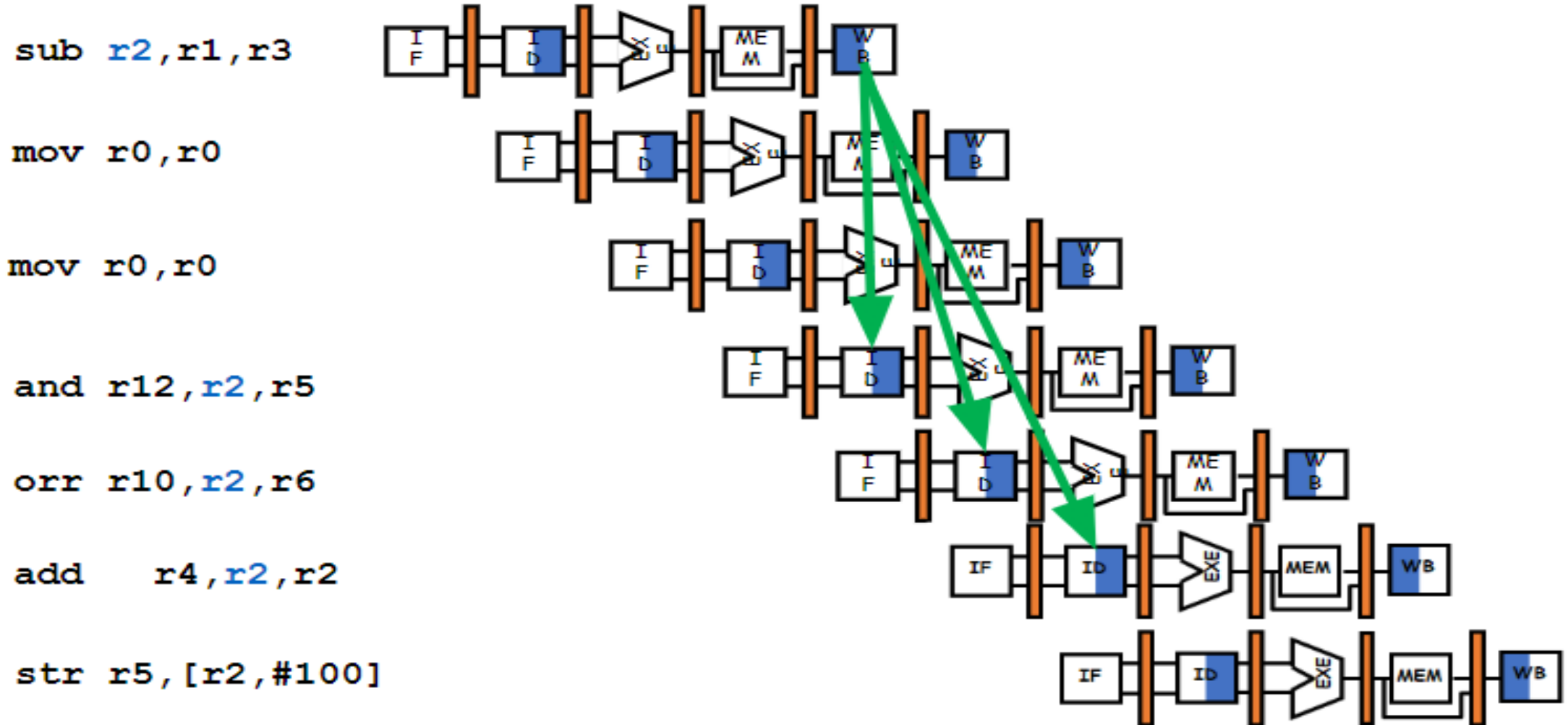
```
sub  R2, R1, R3
NOP or MOV R0, R0
NOP or MOV R0, R0

and  R12, R2, R5
orr  R10, R6, R2
add  R4, R2, R2
str  R5, [R2, #100]
```

Software Solution 2: By Compiler



Software Solution 2: By Compiler



Where are NOPs needed?

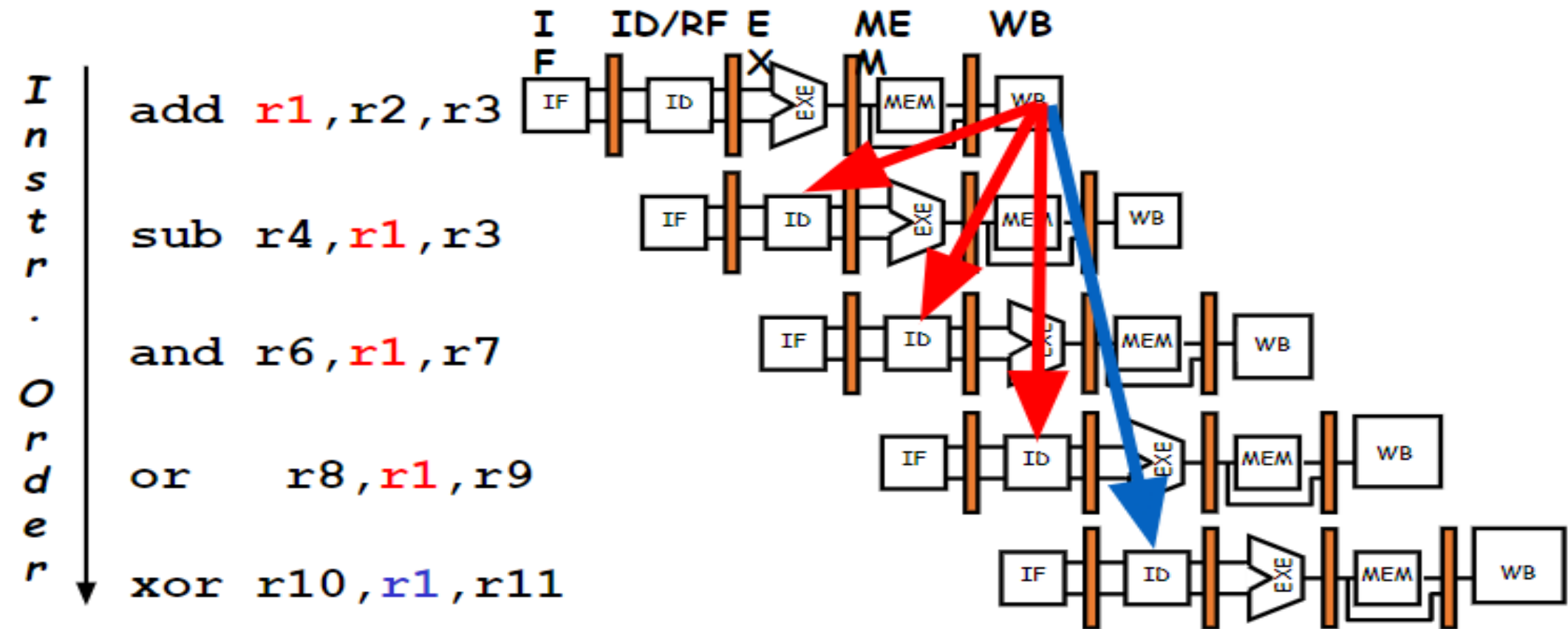
```
sub    R2, R1, R3
and    R4, R2, R5
orr    R8, R2, R6
add    R9, R4, R2
rsb    R1, R6, R7
```

```
sub    R2, R1, R3
NOP
NOP
and    R4, R2, R5
orr    R8, R2, R6
NOP
add    R9, R4, R2
rsb    R1, R6, R7
```

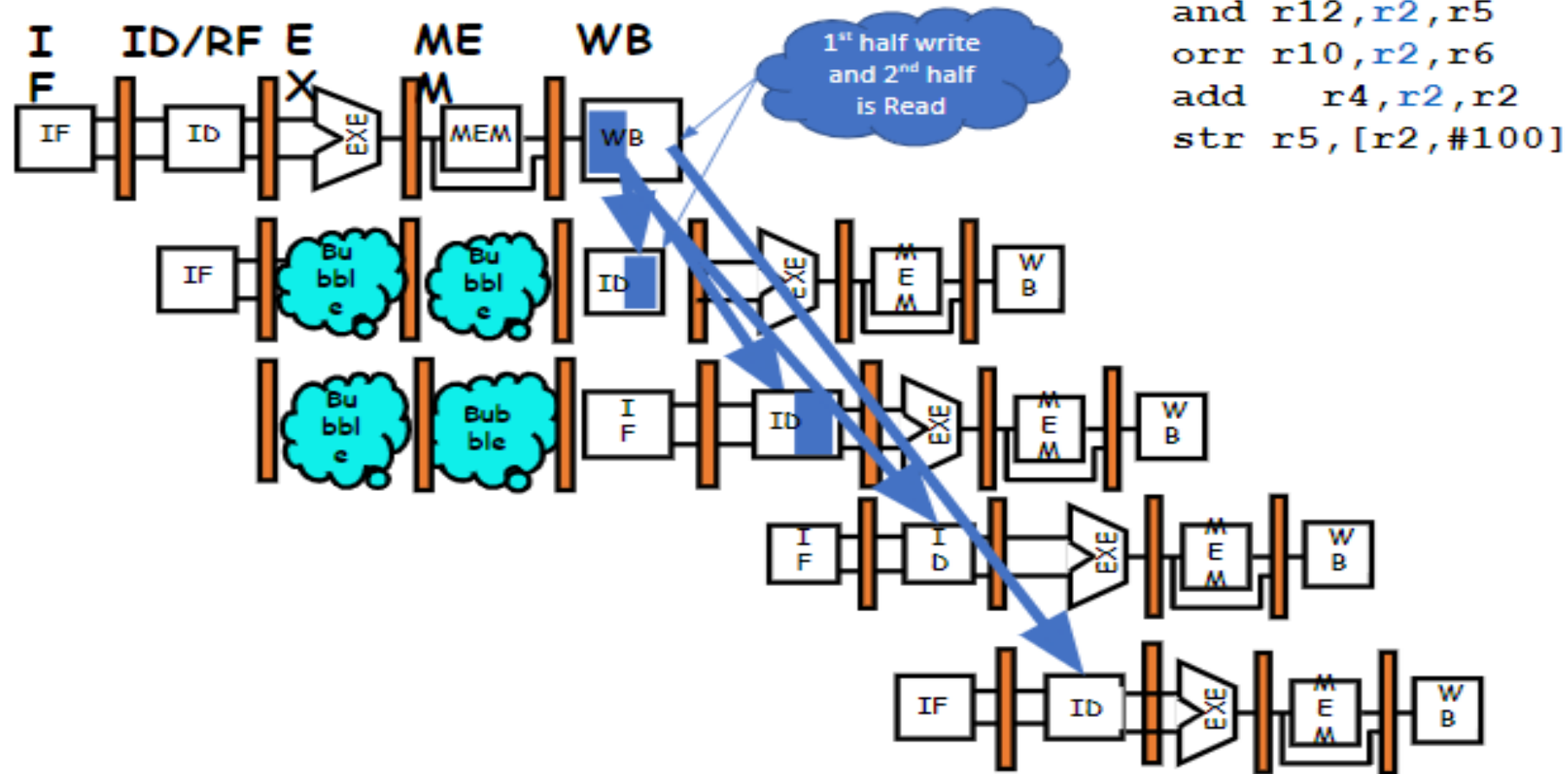
Attacking Data Hazard

- In Software
 - Solution 1: Re-order instructions
 - Solution 2: Insert independent instructions (or no-ops) Ex: MOV R0, R0
- In Hardware
 - **Solution 1: Insert bubbles (i.e. stall the pipeline)**
 - Solution 2: Data Forwarding

Hardware Solution 1



Hardware Solution 1



Attacking Data Hazard

- In Software

- Solution 1: Re-order instructions
- Solution 2: Insert independent instructions (or no-ops) Ex: MOV R0, R0

- In Hardware

- Solution 1: Insert bubbles (i.e. stall the pipeline)
- **Solution 2: Data Forwarding**

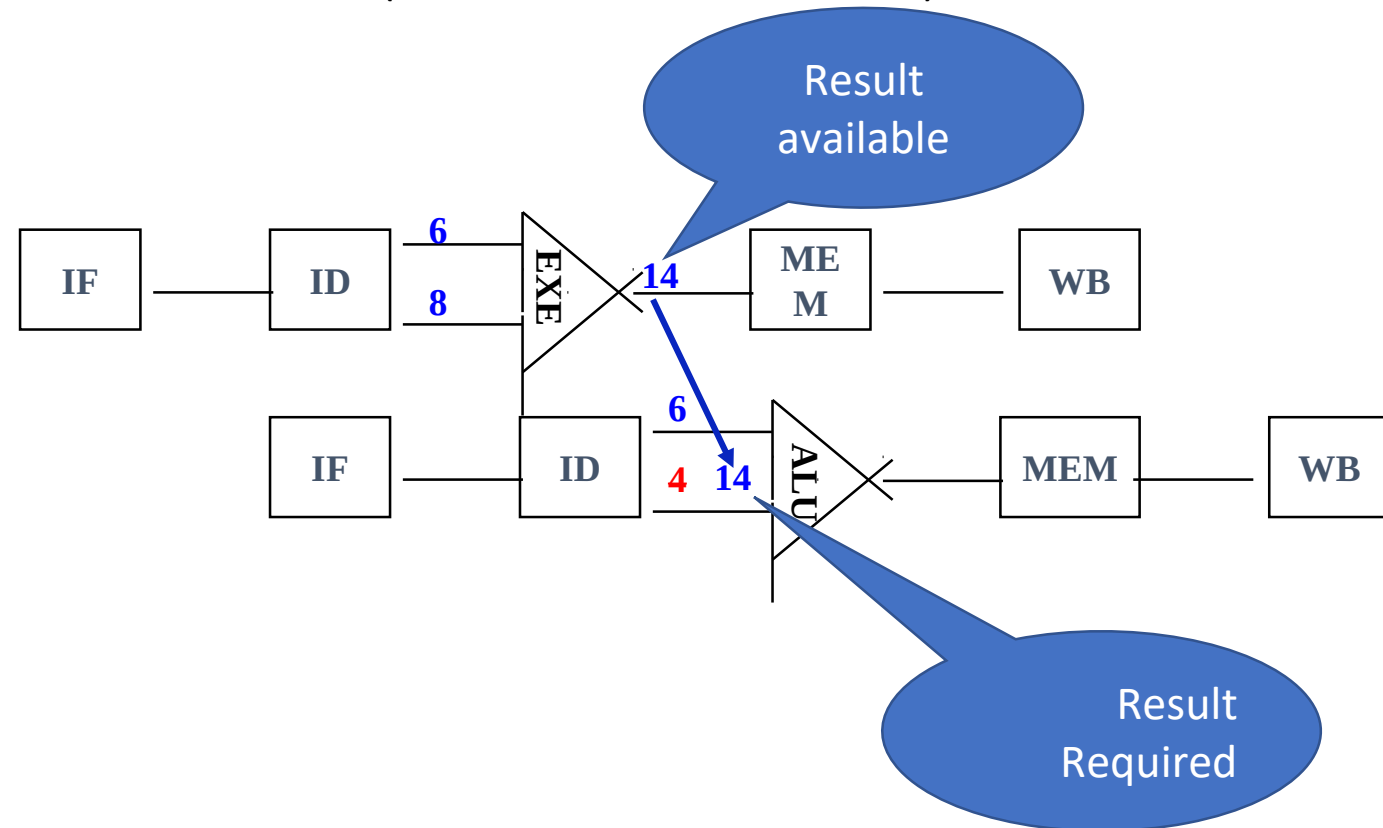
Hardware Solution 2: Data Forwarding

We could avoid stalling if we could get to EX stage the ALU output from “previous instruction” to ALU input for the “current instruction”

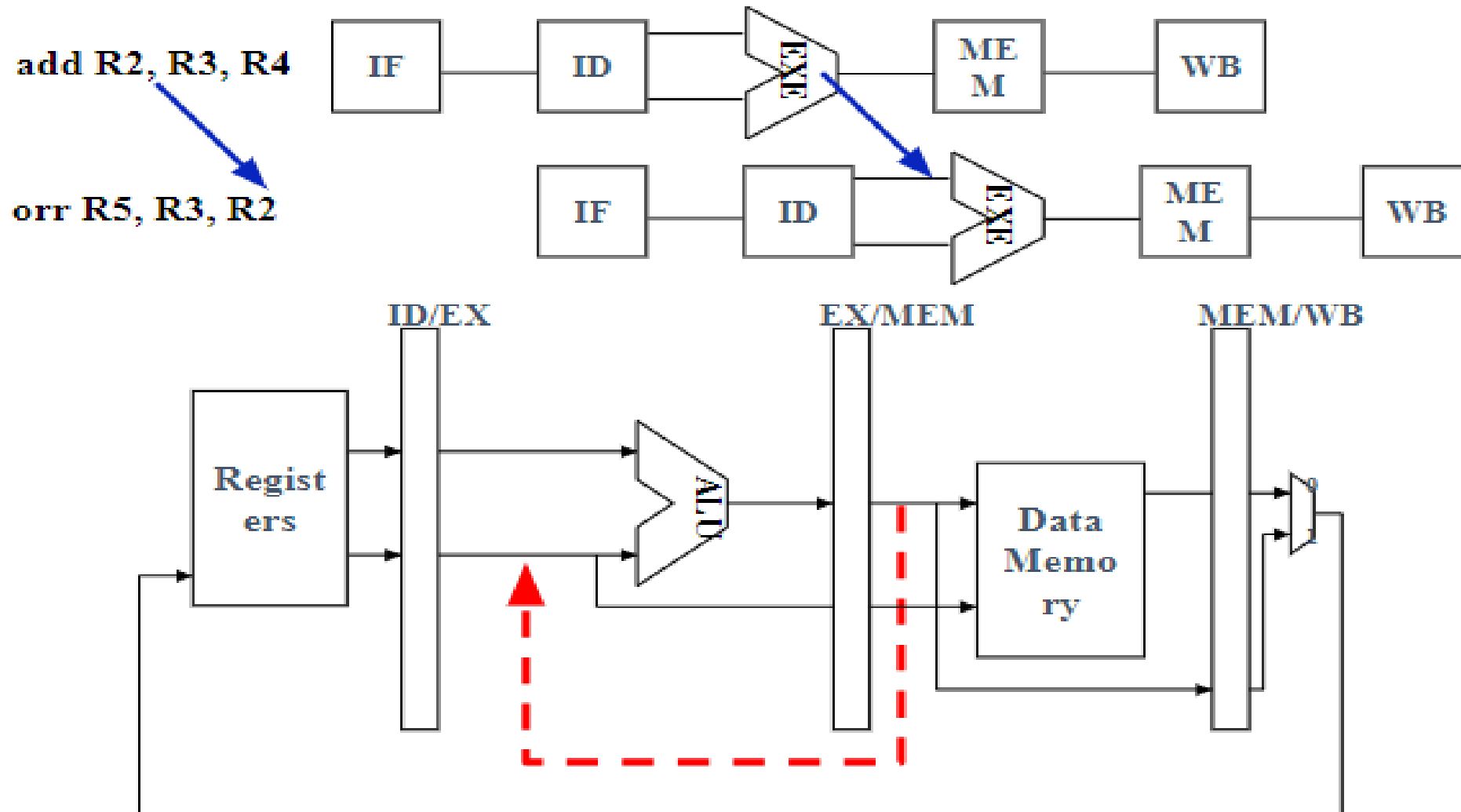
Suppose initially, register i holds the number 2i (ie. R3 = 6; R6=12; R8 = 16...)

add R2, R3, R4

orr R5, R3, R2

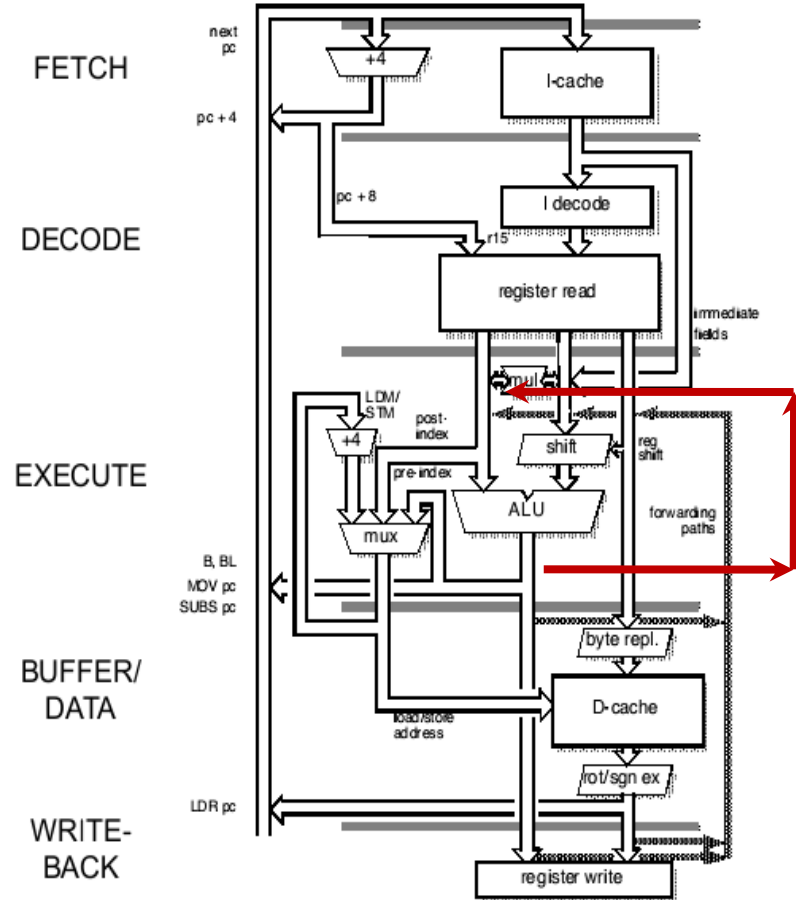


Hardware Solution 2: Data Forwarding



Hardware Solution 2: Data Forwarding

ARM9TDMI 5-stage pipeline organization



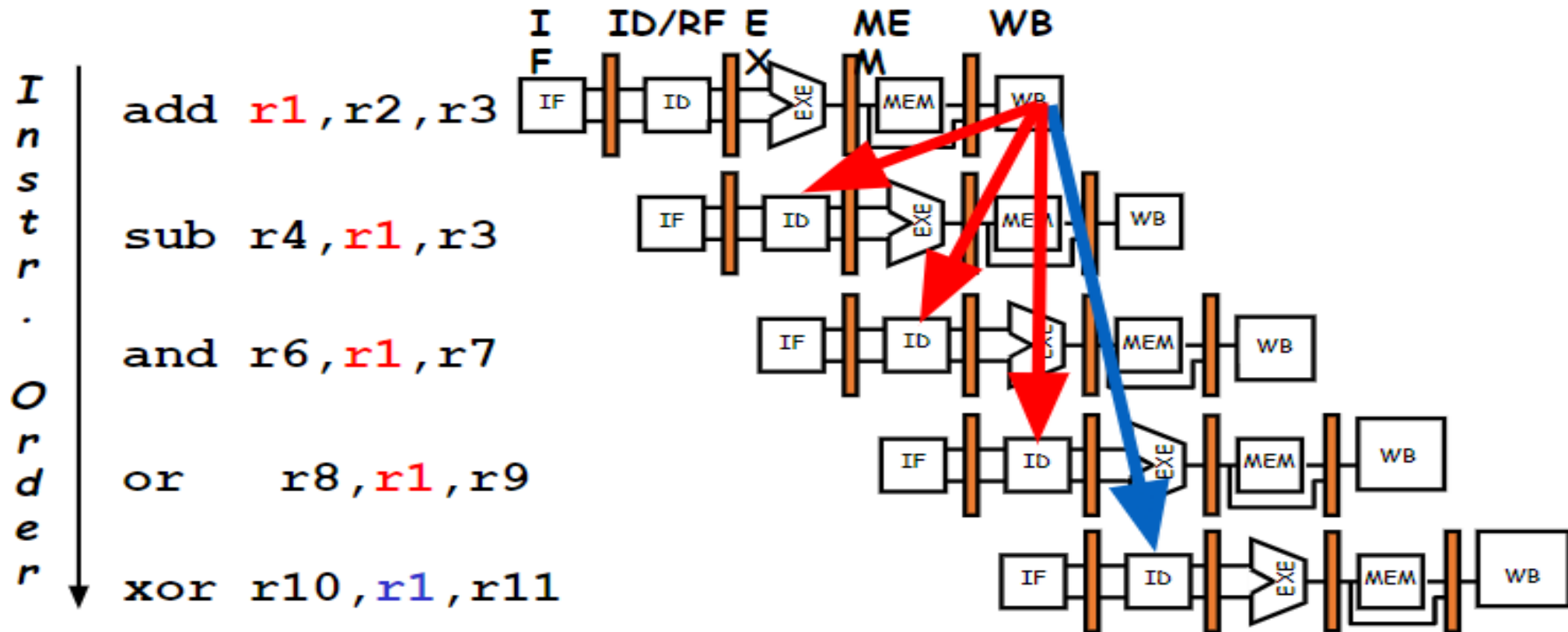
add R2, R3, R4

orr R5, R3, R2

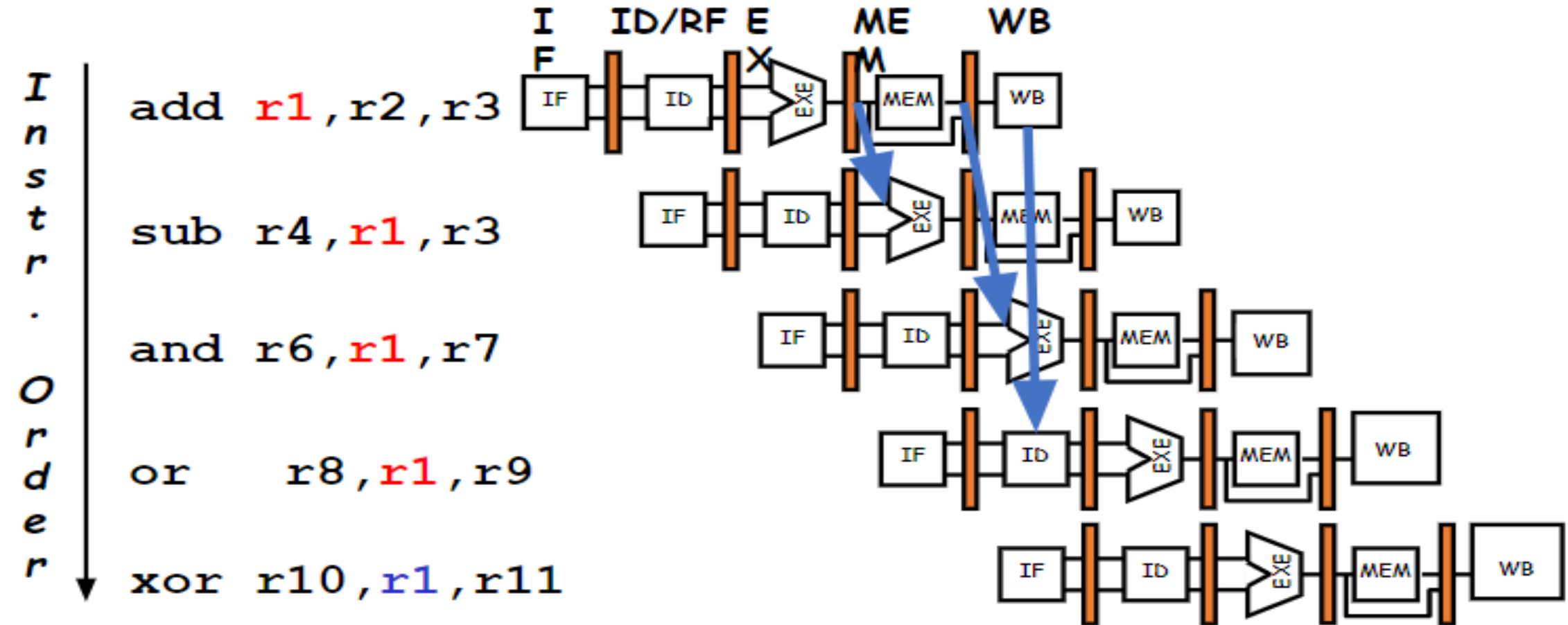
Hardware Solution 2: Data Forwarding

- Also known as:
 - Register – bypassing
 - Short-circuiting
- Forwarding handles hazards at both
 - EX stage
 - MEM stage

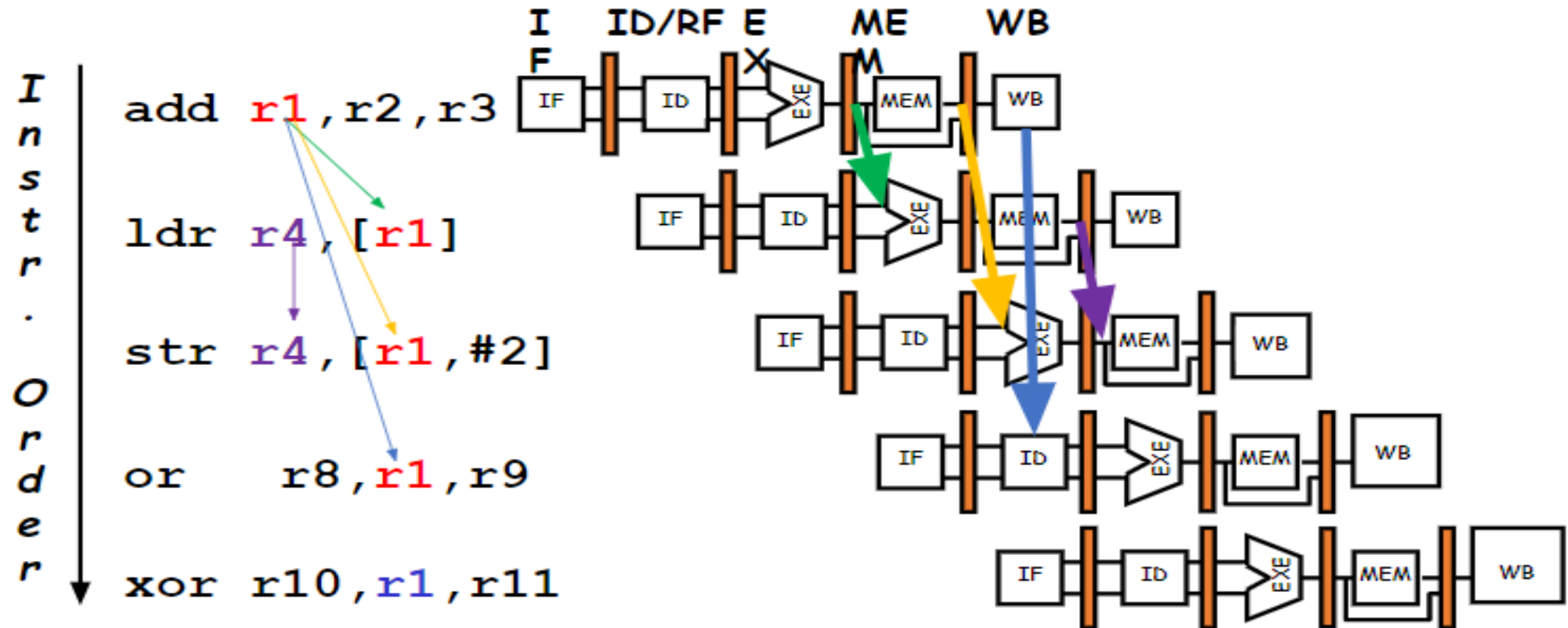
Hardware Solution 2: Data Forwarding



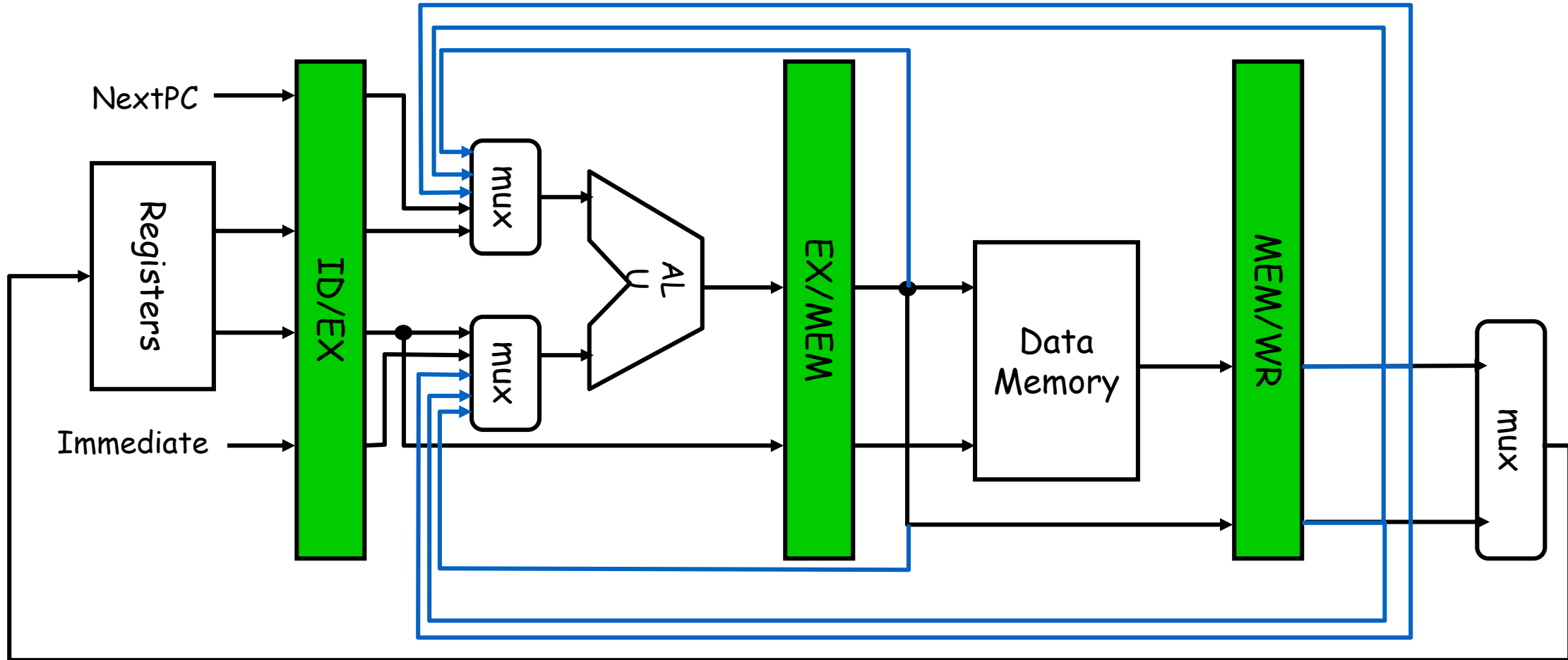
Hardware Solution 2: Data Forwarding



Hardware Solution 2: Data Forwarding



Hardware change for Forwarding



What circuit detects and resolves this hazard?

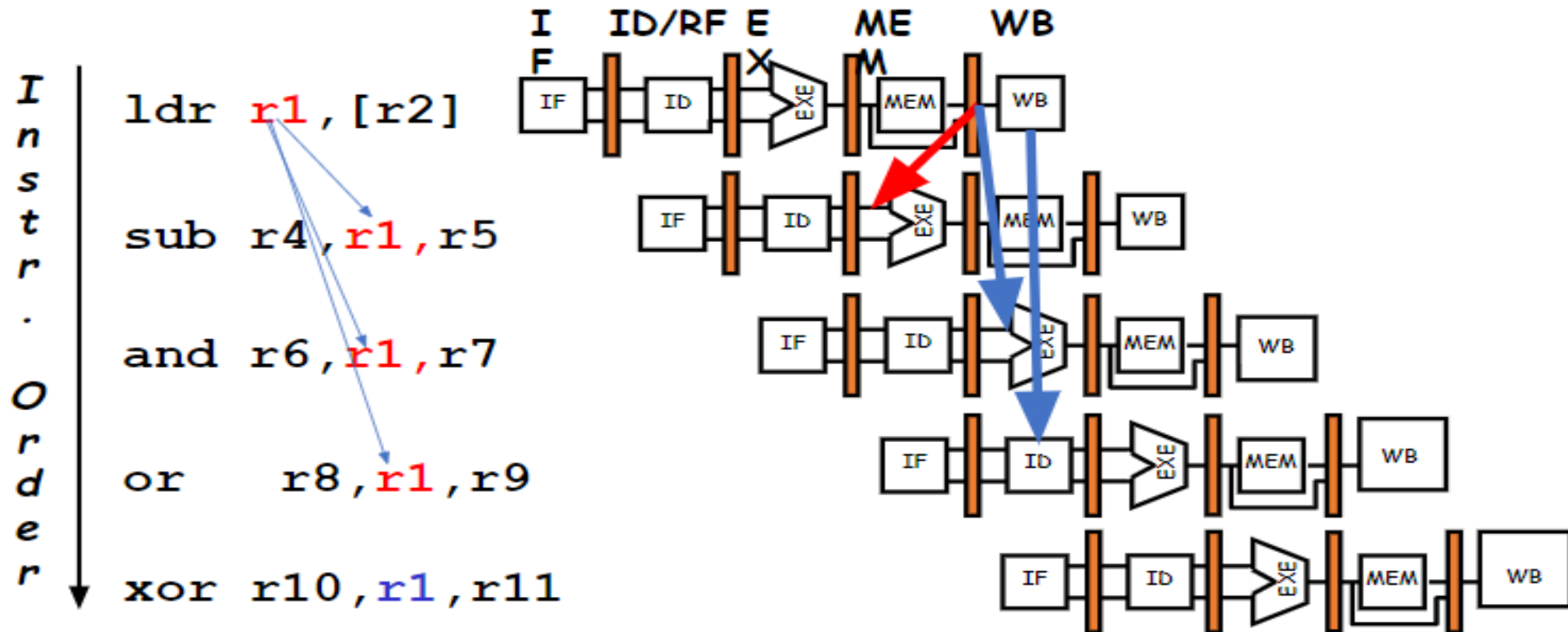
Does Forwarding eliminate all hazards??

Consider this example:

```
LDR    R0, [R1, #60]  
ADD    R2, R0, R4
```

NO! You may need to stall after loads

Hardware Solution 2: Data Forwarding



Think About It

```

ldr    R7, [R2]
ldr    R6, [R2, #4]
add    R4, R5, R6
str    R6, [R2, #4]

```

With forwarding we need to find only one independent instructions to place between them,

swapping the ldr instructions works:

ldr	R6, [R2, #4]	IF	ID	EXE	MEM	WB			
ldr	R7, [R2]		IF	ID	EXE	MEM	WB		
add	R4, R5, R6			IF	ID	EXE	MEM	WB	
str	R6, [R2, #4]				IF	ID	EXE	MEM	WB

Blue arrows indicate data forwarding from the MEM stage of the first ldr instruction to the EXE stage of the second ldr instruction, and from the MEM stage of the second ldr instruction to the EXE stage of the str instruction.

Without forwarding we need two independent instructions to place between them, so in addition a nop is added.

```

ldr    R6, [R2, #4]
ldr    R7, [R2]
nop
add    R4, R5, R6
str    R6, [R2, #4]

```

Why?

Think About It

$a = b + c;$

$d = e - f;$

Before:

Eliminate dependency
by renaming registers

After Reordering &
Assuming Data
Forwarding

```
ldr    R2, =b
ldr    R3, =c
add    R1, R2, R3
str    R1, =a
ldr    R2, =e
ldr    R3, =f
sub    R1, R2, R3
str    R1, =d
```

```
ldr    R2, =b
ldr    R3, =c
add    R1, R2, R3
str    R1, =a
ldr    R5, =e
ldr    R6, =f
sub    R4, R5, R6
str    R4, =d
```

```
ldr    R2, =b
ldr    R3, =c
ldr    R5, =e
add    R1, R2, R3
ldr    R6, =f
str    R1, =a
sub    R4, R5, R6
str    R4, =d
```

Draw the timing diagram check, if stalls are required?

MCQ's

1. In a standard 5-stage pipeline without data forwarding, how many NOP (No Operation) instructions must be inserted between a producer instruction (e.g., SUB R2, R1, R3) and a consumer instruction (e.g., AND R12, R2, R5) to resolve the hazard?

- A. 1 NOP B. 2 NOP C. 3 NOP D. 0 NOP

2. What is the technique called where the ALU output from a previous instruction is fed directly to the ALU input of the current instruction, bypassing the register file?

- A. Pipeline Stalling
B. Branch Prediction
C. Data Forwarding (Short Circuiting)
D. Instruction Re-ordering

MCQ's

1. In a standard 5-stage pipeline without data forwarding, how many NOP (No Operation) instructions must be inserted between a producer instruction (e.g., SUB R2, R1, R3) and a consumer instruction (e.g., AND R12, R2, R5) to resolve the hazard?

- A. 1 NOP **B. 2 NOP** C. 3 NOP D. 0 NOP

2. What is the technique called where the ALU output from a previous instruction is fed directly to the ALU input of the current instruction, bypassing the register file?

- A. Pipeline Stalling
B. Branch Prediction
C. Data Forwarding (Short Circuiting)
D. Instruction Re-ordering

MCQ's

3. What is the technique called where the ALU output from a previous instruction is fed directly to the ALU input of the current instruction, bypassing the register file?

- A. Data Forwarding
- B. Scheduling
- C. Stalling
- D. Re-ordering instructions

4. How can a compiler resolve a data hazard without increasing the code size (i.e., without adding NOPs) and without hardware changes?

- A. By increasing the clock speed
- B. By using Instruction Re-ordering (Scheduling)
- C. By disabling the pipeline
- D. By flushing the pipeline

3. What is the technique called where the ALU output from a previous instruction is fed directly to the ALU input of the current instruction, bypassing the register file?

- A. Data Forwarding**
- B. Scheduling
- C. Stalling
- D. Re-ordering instructions

4. How can a compiler resolve a data hazard without increasing the code size (i.e., without adding NOPs) and without hardware changes?

- A. By using Instruction Re-ordering (Scheduling)**
- B. By increasing the clock speed
- C. By disabling the pipeline
- D. By flushing the pipeline



THANK YOU



Team MPCA

Department of Computer Science and
Engineering



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Session - 2.6

Microprocessor & Computer Architecture (μ pCA)



UE24CS251B

**Pipeline Processor: Control
Hazard**

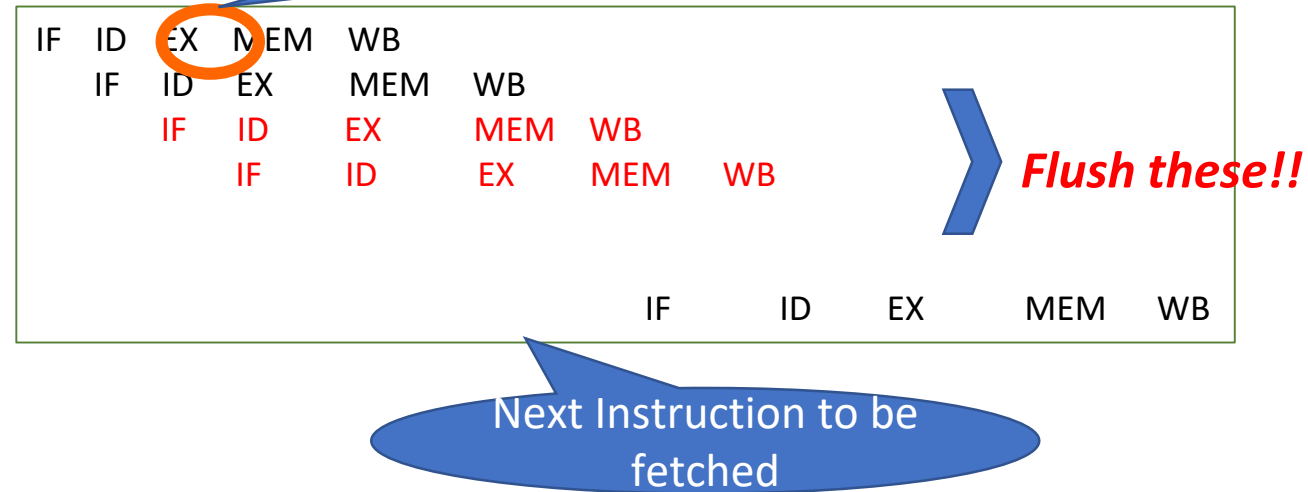
Control Hazards

- When the flow of instruction addresses is **not sequential** (i.e., $PC = PC + 4$); **incurred** by change of flow instructions
 - Conditional branches (**beq**, **bne**)
 - Unconditional branches (**b**, **bal**)
 - Exceptions
- Undesirable instructions get into the pipeline even before branch instruction is fetched, **if branch is taken (Compare and Change PC in EX Stage)**

Ex:

```
add  R1, R2, R3
beq  R1, R2, target
sub  R1, R4, R5
orr  R2, R7, R8
```

```
target:    ....
           add    R1, R4, R5
```



Control Hazard on Branches: Three Stage Stall

```

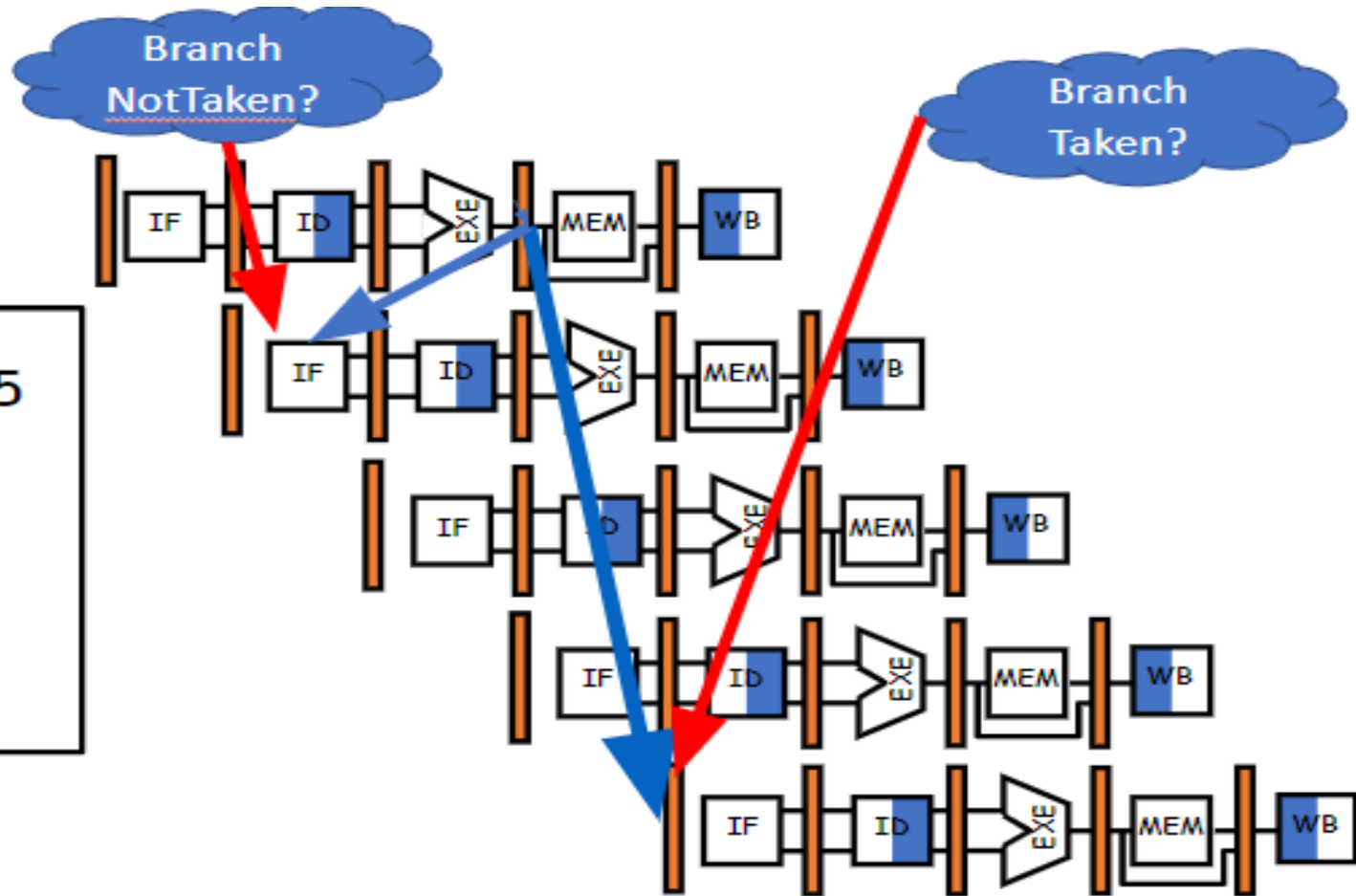
10: beq
r1,r3,36

14: and r2,r3,r5

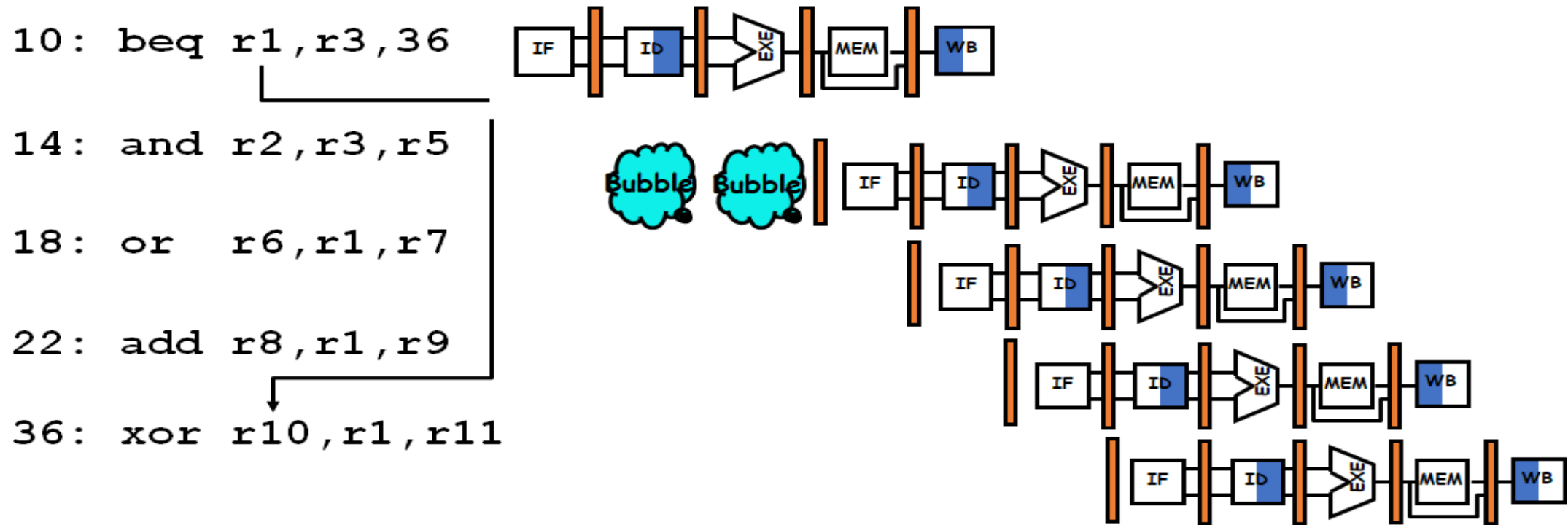
18: or
r6,r1,r7

22: add
r8,r1,r9

36: xor
r10,r1,r11
    
```



Control Hazard on Branches: Safe Decision



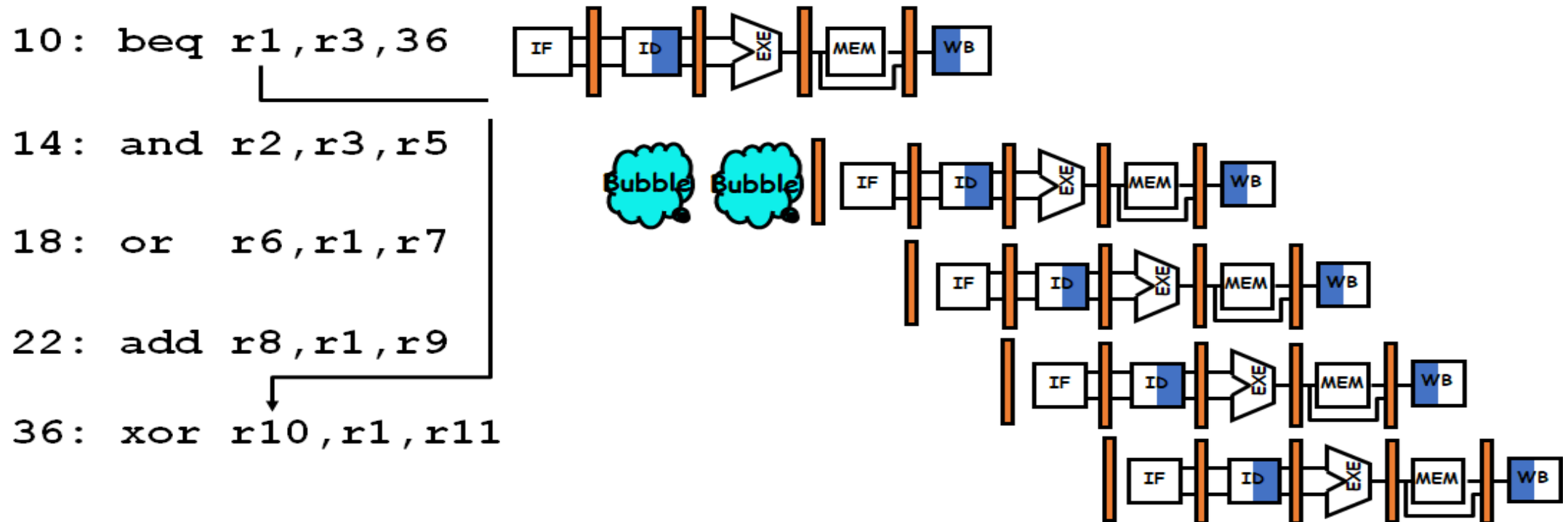
All branches waste 2 cycles irrespective of whether branch is taken or not.

Wasteful..

Better way???

Reduce stall cycles

Guess or Predict



Control Hazard on Branches

When Do I know the result of Branch?
When Do I update PC?

```

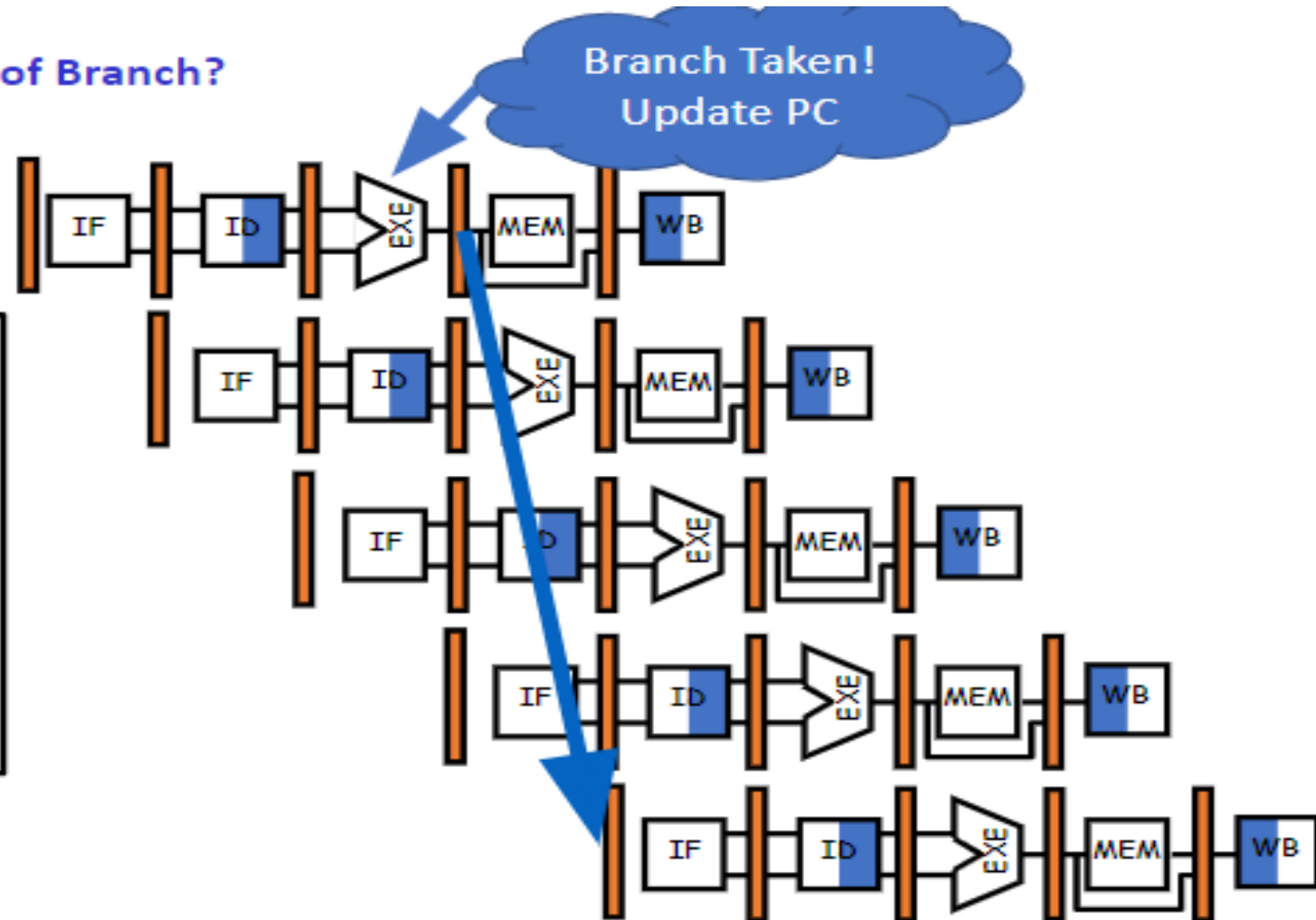
10: beq
r1, r3, 36

14: and r2, r3, r5

18: or
r6, r1, r7

22: add
r8, r1, r9

36: xor
r10, r1, r11
    
```



Control Hazard on Branches

Can I Reduce the Stalls by
Computing Result of Branch Early.
Update the PC Early.

```

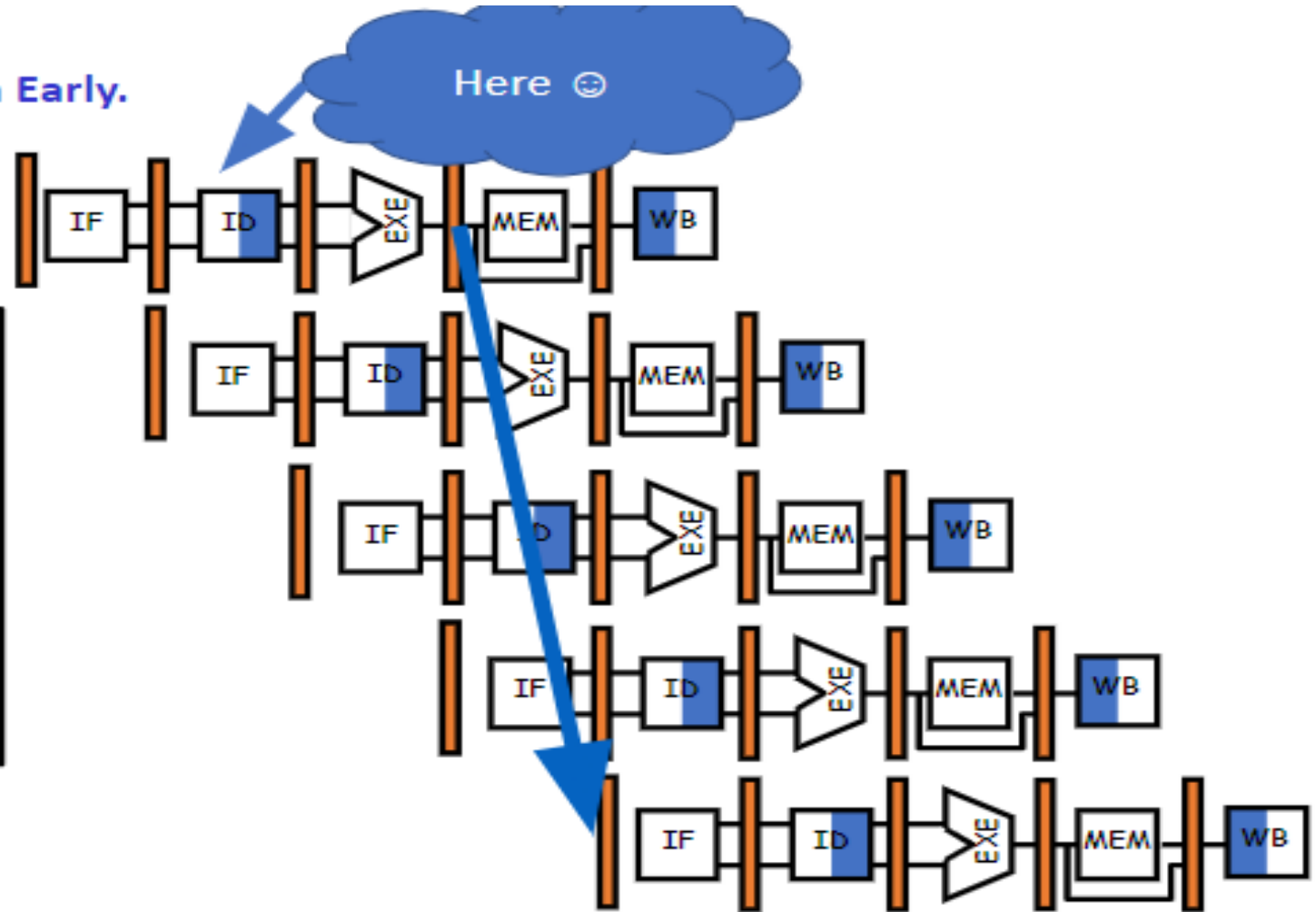
10: beq
r1, r3, 36

14: and r2, r3, r5

18: or
r6, r1, r7

22: add
r8, r1, r9

36: xor
r10, r1, r11
    
```

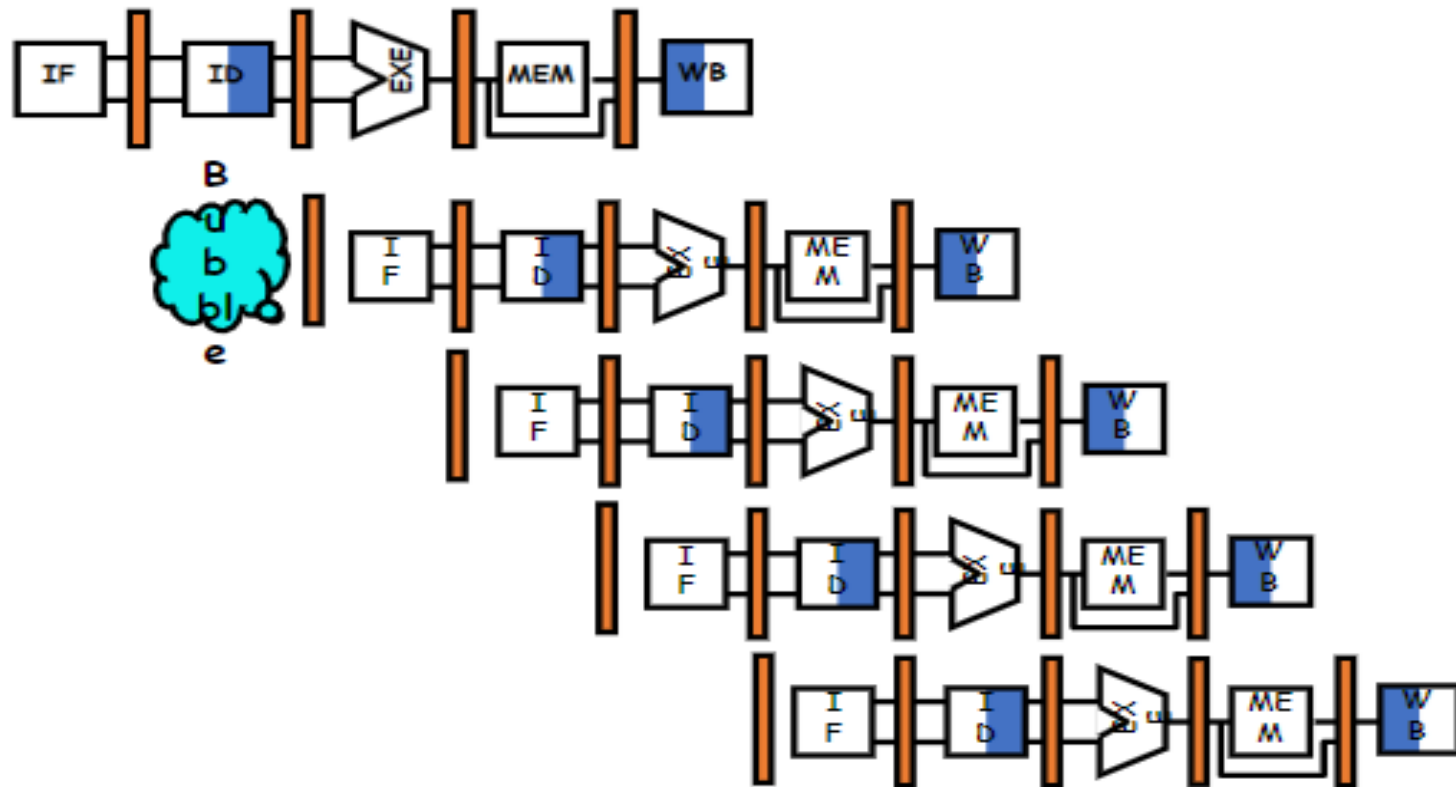


Control Hazard on Branches: Safe Decision

Can I Reduce the Stalls by
Computing Result of Branch Early.
Update the PC Early.

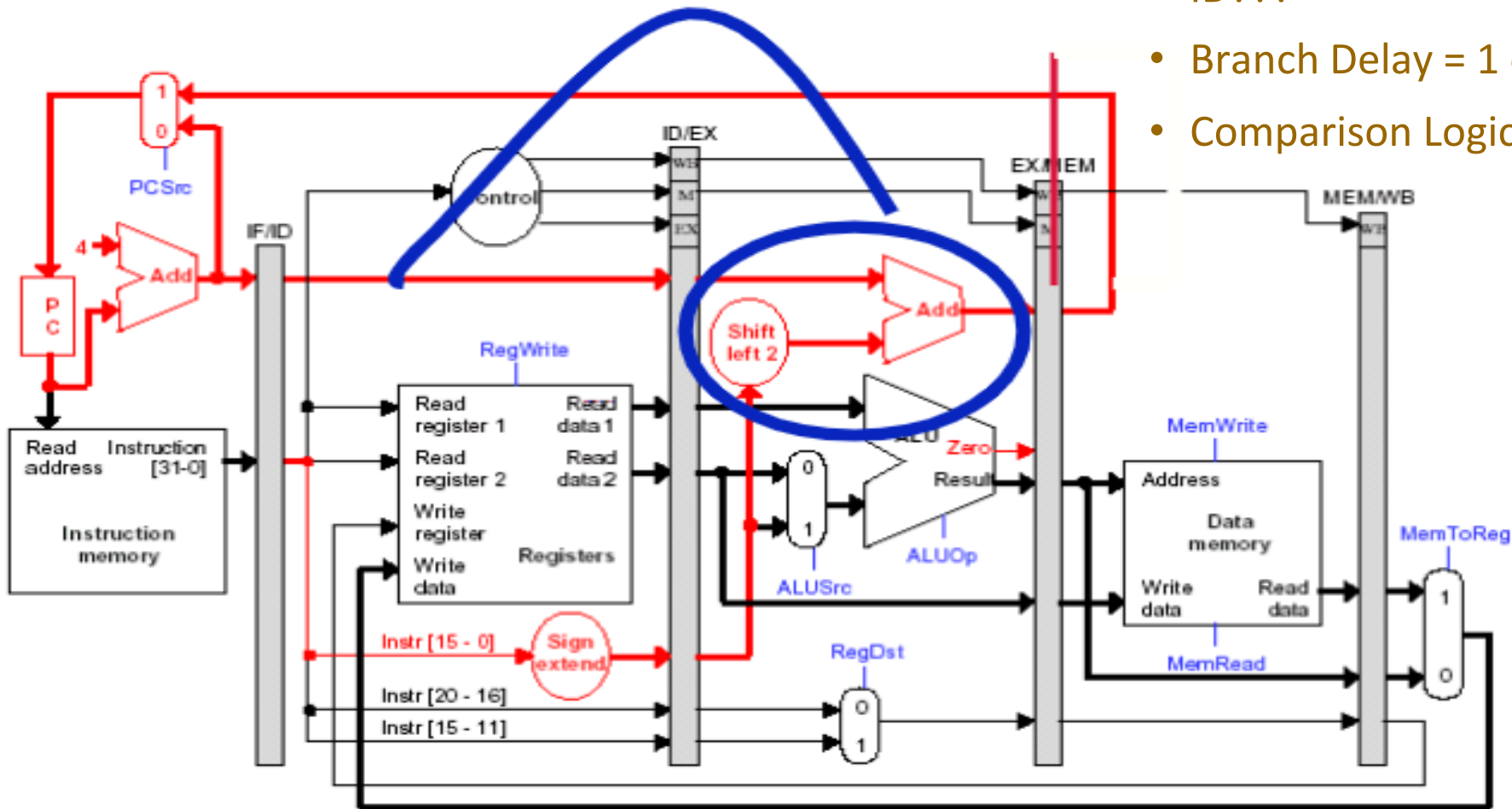
```

10: beq
r1,r3,36
14: and
r2,r3,r5
18: or
r6,r1,r7
22: add
r8,r1,r9
36: xor
r10,r1,r11
  
```

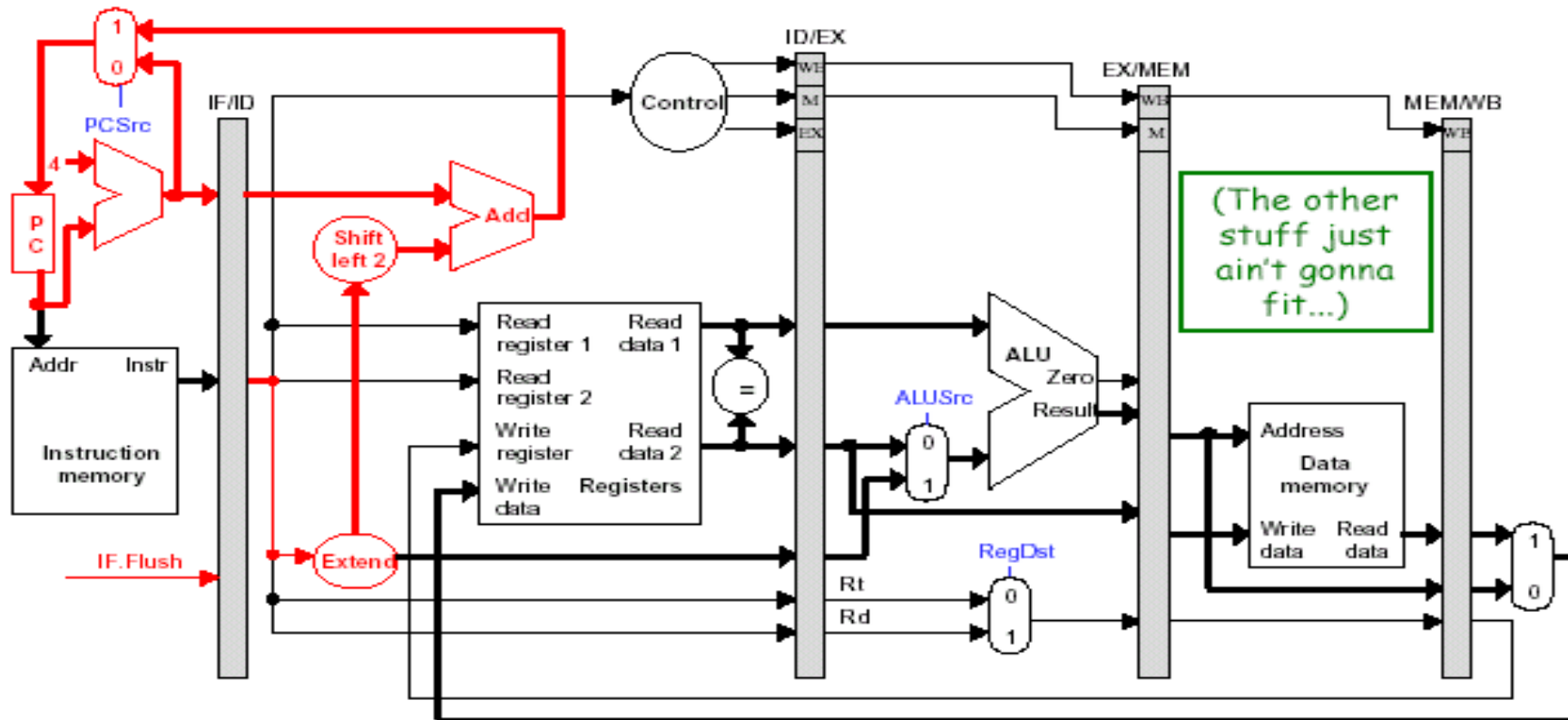


Reduce Stall Cycles?

- By moving updating of PC from EX to ID???
- Branch Delay = 1 cycle
- Comparison Logic is moved to ID stage



- Can we reduce it further by moving the updating of PC from ID to IF???



Example

- Assume the following instruction mix:

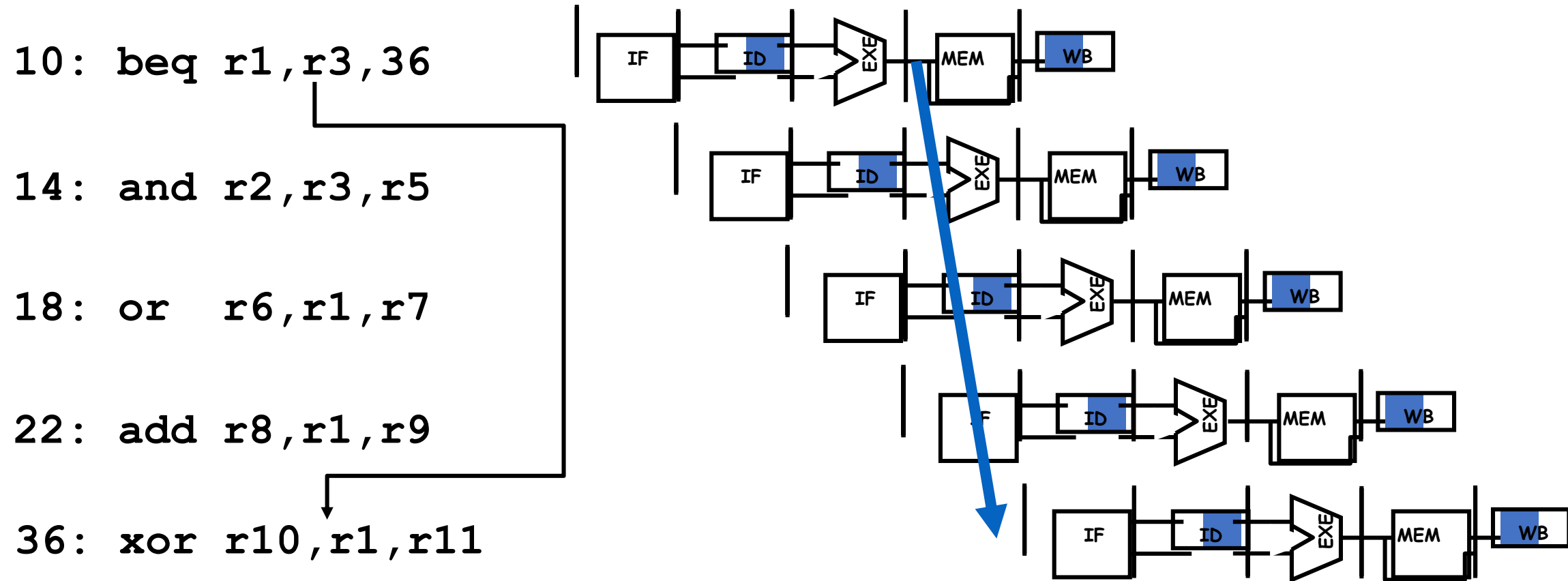
<u>Type</u>	<u>Frequency</u>	
Arith/Logic	40%	
Load	30%	of which 25% are followed immediately by an instruction using the loaded value
Store	10%	
branch	20%	of which 45% are taken

- What is the resulting CPI for the pipelined processor with forwarding and branch address calculation in ID stage?
- $$\begin{aligned}\text{CPI} &= \text{Ideal CPI} + \text{Pipeline stall clock cycles per instruction} \\ &= 1 + \text{stalls by loads} + \text{stalls by branches} \\ &= 1 + .3 \times .25 \times 1 + .2 \times .45 \times 1 \\ &= 1 + .075 + .09 \\ &= 1.165\end{aligned}$$

Delayed Branching

- Similar to *insert no-ops*
- Compiler inserts....??
- Used to eliminate branch stalls
- Instruction after branch is known as delay slot
- Instruction in delay slot is always executed
- Fill the slots with useful instructions

Control Hazard on Branches: Delay Slot



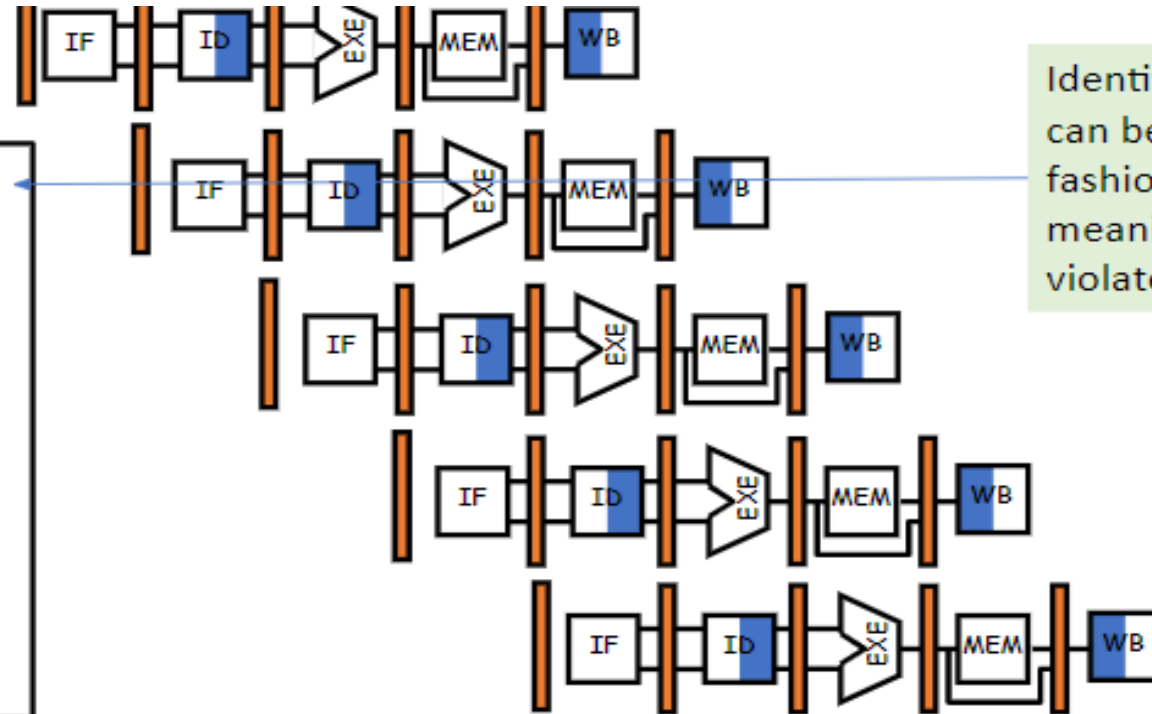
Identify the Instruction which can be executed out of order fashion i.e will not change the meaning of the program or not violate register write

48: sub r3, r1, r4

Control Hazard on Branches: Delay Slot

```

10: beq
r1,r3,36
48: sub r3,r1,r4
14: and r2,r3,r5
18: or
r6,r1,r7
22: add
r8,r1,r9
36: xor
r10,r1,r11
  
```

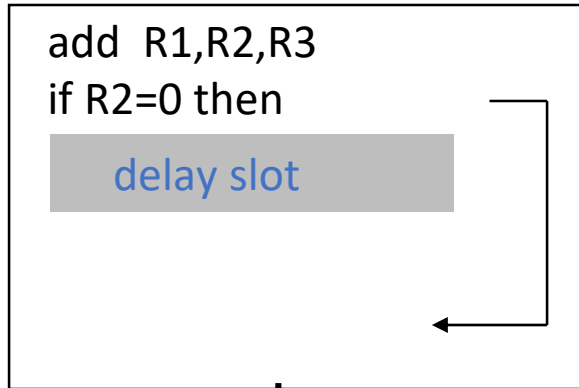


ONLINE

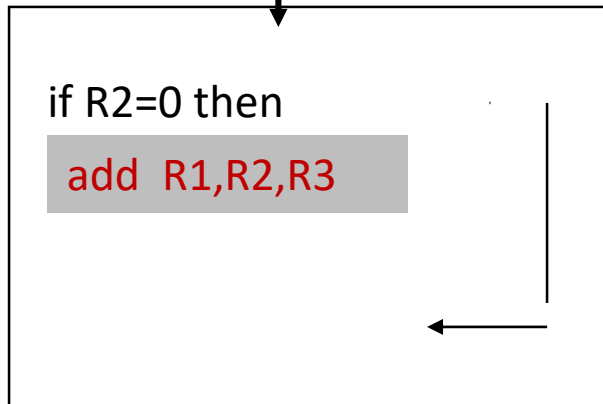
Identify the Instruction which can be executed out of order fashion i.e will not change the meaning of the program or not violate register write

Scheduling Branch Delay Slots

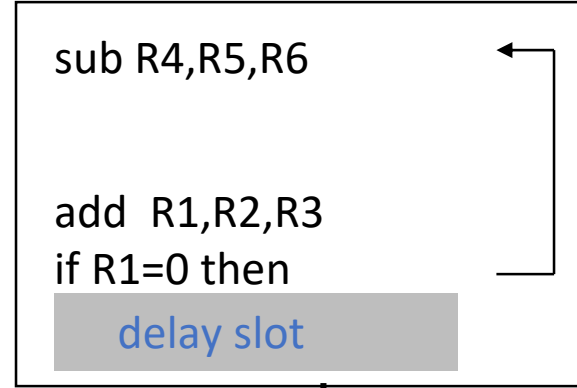
A. From before branch



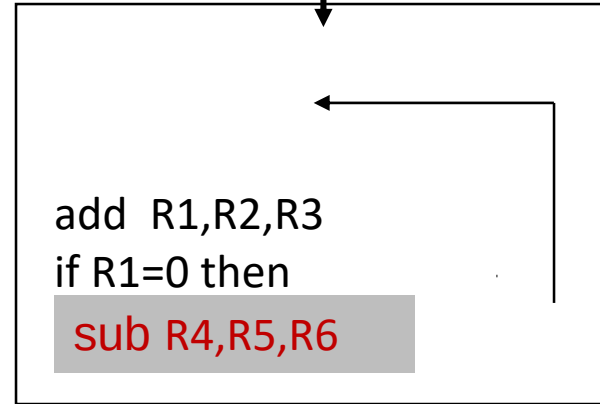
becomes



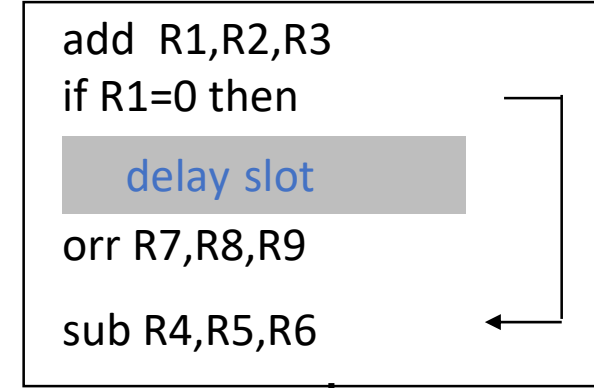
B. From branch target



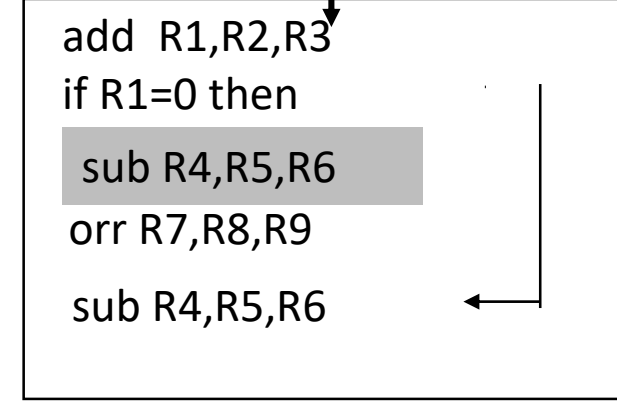
becomes



C. From fall through



becomes



- A is the best choice, fills delay slot and reduces IC
- In B and C, the **sub** instruction may need to be copied, increasing IC
- In B and C, must be okay to execute **sub** when branch fails

A: Filling the delay slot Before Branch

- **MUL R3, R4, R5**
- SUB R2, R1, R0
- ADD R1, R2, R2
- BEQZ R1, R7, there

Delay Slot

ADD R1, R4, R7

there:

- SUB R2, R1, R0
- ADD R1, R2, R2
- BEQZ R1, R7, there
- MUL R3, R4, R5**
- ADD R1, R4, R7

there:

- In case, the compiler is not able to find suitable instruction, then
- MUL R3, R4, R5
- SUB R2, R1, R0
- ADD R1, R2, R2
- BEQZ R1, R7, there
- NOP**
- ADD R1, R4, R7

there:

B: Filling the delay slot From Branch Target

SUB R2, R1, R0

ADD R1, R2, R2

BEQZ R1, R7, there

ADD R1, R4, R7

Branch Not Taken Part

there: MUL R3, R4, R5

Branch Taken Part

Delay Slot

- SUB R2, R1, R0
- ADD R1, R2, R2
- BEQZ R1, R7, there
- **MUL R3, R4, R5**
- ADD R1, R4, R7

there:

- Useful when it is predicted, 80 % Branch Taken & 20% Not Taken
- It should ensure no Register write violation in the Not Taken part of the instructions
- If Branch is not taken, then FLUSH i.e Waste of Cycle time

C: Filling the delay slot From Fall through

SUB R2, R1, R0

ADD R1, R2, R2

BEQZ R1, R7, there

MUL R3, R4, R5

ADD R1, R4, R7

Branch Not Taken Part

there:

Branch Taken Part

Delay Slot

- SUB R2, R1, R0
ADD R1, R2, R2
BEQZ R1, R7, there
MUL R3, R4, R5
ADD R1, R4, R7

there:

- Useful when it is predicted, 80 % Branch Not Taken & 20% Taken
- It should ensure no Register write violation in the Not Taken part of the instructions
- If Branch is not taken, then FLUSH i.e Waste of Cycle time

1. Which of the following scenarios triggers a Control Hazard (or Branch Hazard) in a pipelined processor?
 - A. When an instruction attempts to use data before it is available in the register file.
 - B. When two instructions compete for the same hardware resource at the same time.
 - C. When the flow of instruction addresses is not sequential (e.g., executing a beq or jump)
 - D. When the memory access stage takes longer than one clock cycle.

2. In the initial pipeline design discussed, a branch decision made in the EX (Execution) stage results in a 2-cycle penalty. How is this penalty reduced to just 1 cycle?
 - A. By deciding the branch direction in the IF (Instruction Fetch) stage
 - B. By moving the branch decision and address calculation logic to the ID (Instruction Decode) stage
 - C. By increasing the clock speed of the processor.
 - D. By removing the branch instruction entirely.

1. Which of the following scenarios triggers a Control Hazard (or Branch Hazard) in a pipelined processor?
 - A. When an instruction attempts to use data before it is available in the register file.
 - B. When two instructions compete for the same hardware resource at the same time.
 - C. When the flow of instruction addresses is not sequential (e.g., executing a beq or jump)**
 - D. When the memory access stage takes longer than one clock cycle.

2. In the initial pipeline design discussed, a branch decision made in the EX (Execution) stage results in a 2-cycle penalty. How is this penalty reduced to just 1 cycle?
 - A. By deciding the branch direction in the IF (Instruction Fetch) stage
 - B. By moving the branch decision and address calculation logic to the ID (Instruction Decode) stage**
 - C. By increasing the clock speed of the processor.
 - D. By removing the branch instruction entirely.

3. What is the defining characteristic of an instruction placed in the Branch Delay Slot?
- A. It is executed only if the branch is taken
 - B. It is executed only if the branch is not taken.
 - C. It is treated as a NOP (No Operation) by the processor.
 - D. It is always executed, regardless of whether the branch is taken or not.
4. A processor has an ideal CPI of 1. If 20% of the instructions are branches and each branch causes a 1-cycle stall, what is the resulting CPI? (Note: Assume no other hazards occur).
- A. 1.0
 - B. 1.2
 - C. 2.0
 - D. 1.02

3. What is the defining characteristic of an instruction placed in the Branch Delay Slot?

- A. It is executed only if the branch is taken
- B. It is executed only if the branch is not taken.
- C. It is treated as a NOP (No Operation) by the processor.
- D. It is always executed, regardless of whether the branch is taken or not.**

4. A processor has an ideal CPI of 1. If 20% of the instructions are branches and each branch causes a 1-cycle stall, what is the resulting CPI? (Note: Assume no other hazards occur).

- A. 1.0
- B. 1.2**
- C. 2.0
- D. 1.02



THANK YOU



Team MPCA

Department of Computer Science and
Engineering



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Session - 2.7

Microprocessor & Computer Architecture (μ pCA)



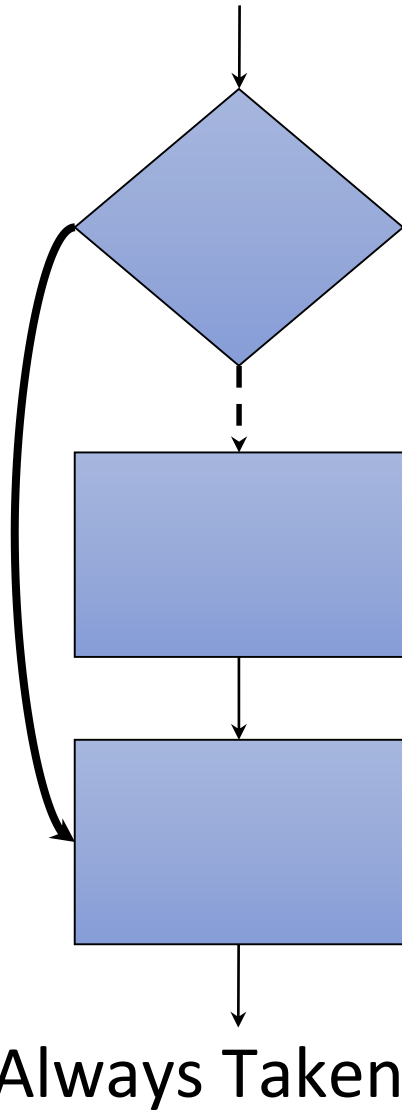
UE24CS251B

Pipeline Processor: Branch Prediction

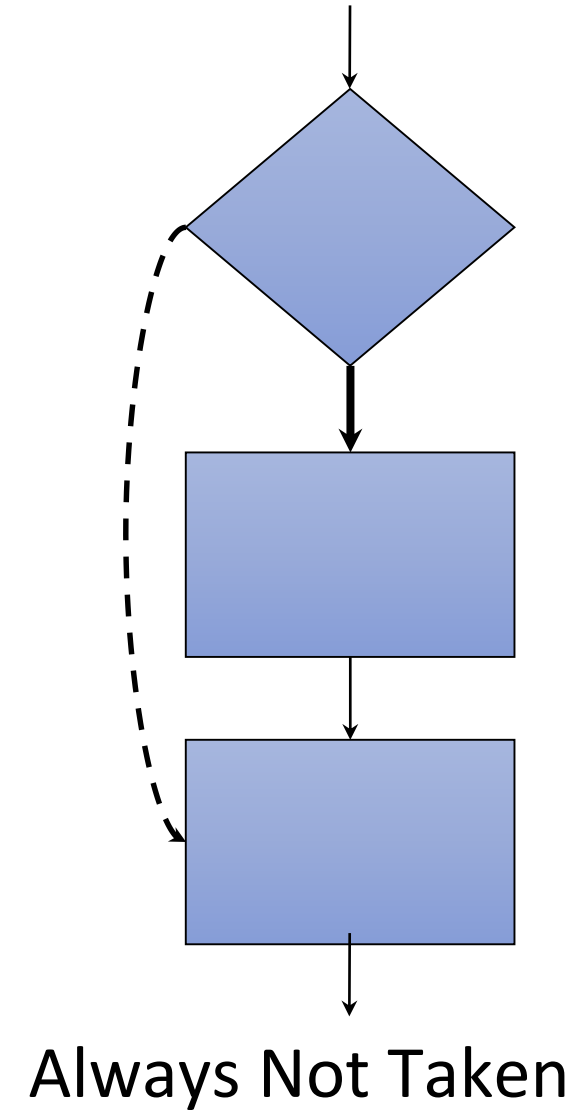
Branch Prediction

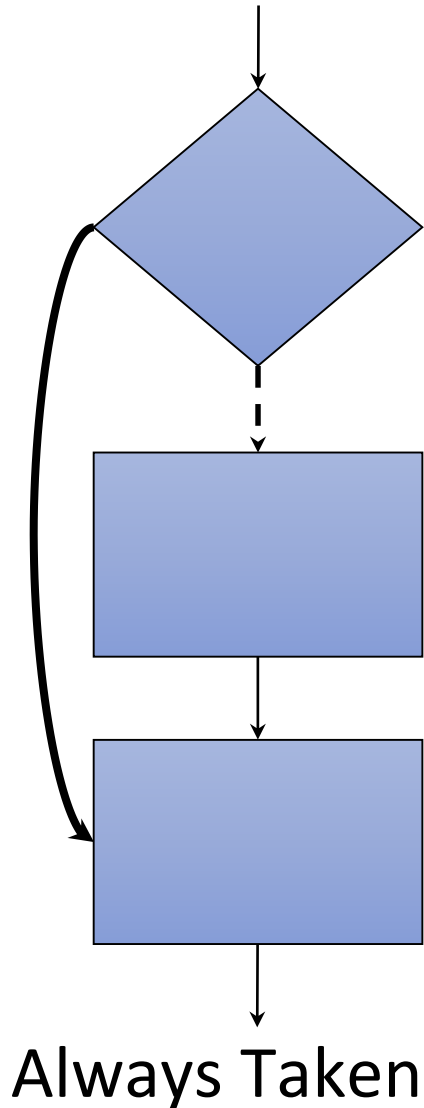
- Guessing the branch
 - One of the important problems in “Computer Architecture”
 - Static: prediction by compiler
 - Dynamic: prediction by hardware
- Static Prediction
 - Always Not Taken (easiest)
 - Always Taken
 - Taken / Not taken
- Dynamic Prediction
 - 1- bit
 - 2-bit
 - others

Static Predictions



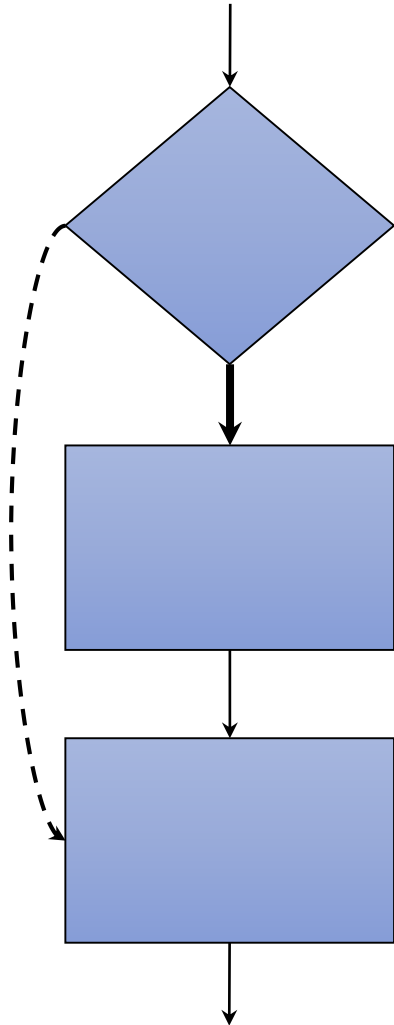
OR





- Early studies indicated that 2/3 of branches are taken
 - but 30% of those branches were unconditional!
- For conditional branches there appears to be no preferred direction.

Alternative Static Predictions

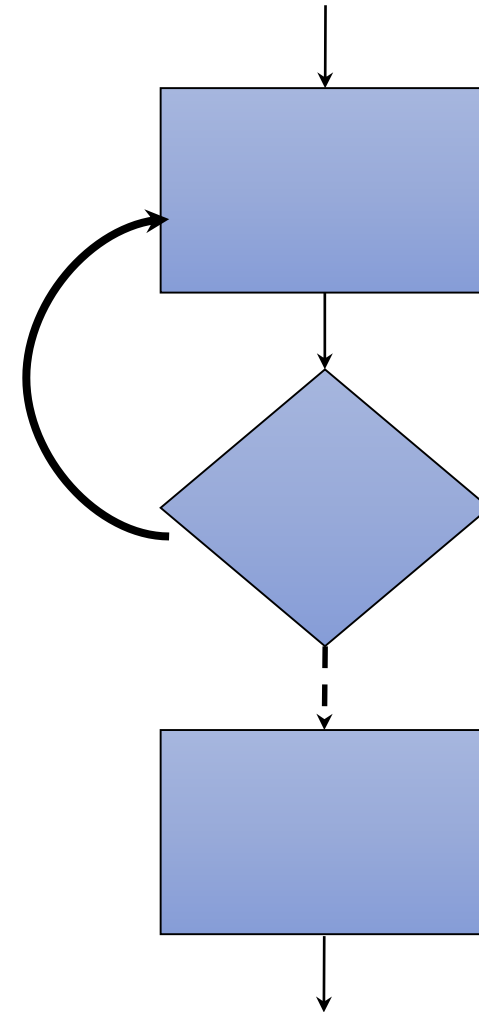


Forward Always Not Taken

Accuracy improvements are barely noticeable.

Static prediction based on profiling is slightly better.

Static branch-not-taken has no implementation cost on pipeline.



Backward Always Taken

Static Prediction: Taken

Consider a program with two branch statements with the following behavior

T T NT T NT T T T NT T T T T NT T T T T NT

How many Miss Prediction ?

5 miss predictions

T	T	NT	T	NT	T	T	T	NT	T	T	T	T	T	NT	T	T	T	T	NT
✓	✓	X	✓	X	✓	✓	✓	X	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	X

Static Prediction: Not Taken

Consider a program with two branch statements with the following behavior

T T NT T NT T T T NT T T T T NT T T T T NT

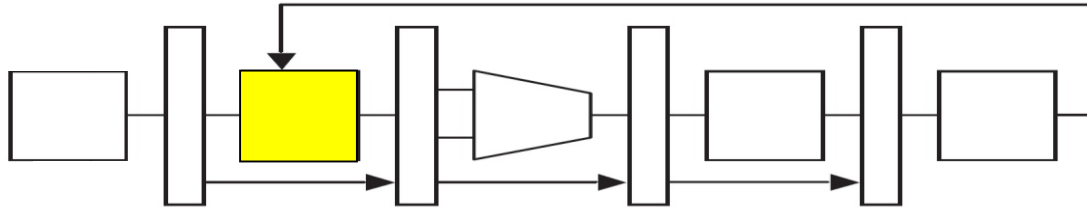
How many Miss Prediction?

15 miss predictions

T	T	NT	T	NT	T	T	T	NT	T	T	T	T	T	NT	T	T	T	T	NT
X	X	✓	X	✓	X	X	X	✓	X	X	X	X	X	✓	X	X	X	X	✓

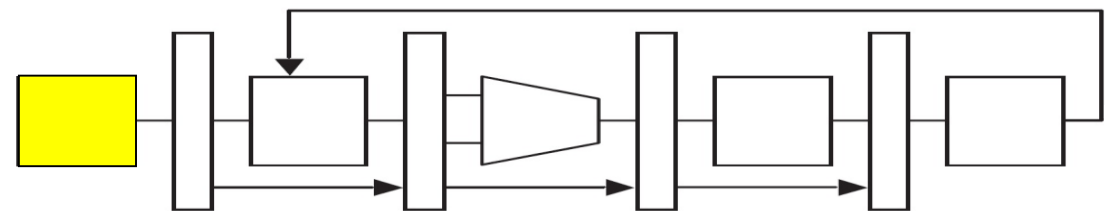
- Prediction of a given branch changes with the execution of the program.
 - **Simple**: a finite-state machine encodes the outcome of a few recent executions of the branch.
 - **Elaborate**: Not only early branch outcomes, but other correlated parts of the programs are considered.

When to Predict?



- Static prediction: at the Instruction Decode stage
 - Know that the instruction is a branch

- Dynamic prediction: at the Instruction Fetch stage
 - How to Know that the instruction is Branch
 - How to Know the target Address?
 - Should check Irrespective of branch instruction or not



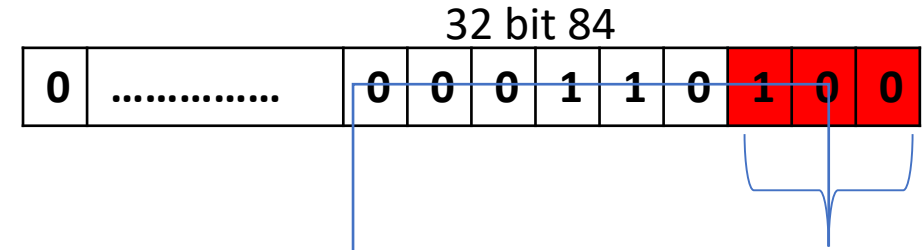
How?

One-bit Predictor

- Simplest method:
 - A branch prediction buffer or Branch History Table (BHT) indexed by low address bits of the branch instruction.
 - Each buffer location (or BHT entry) contains one bit indicating whether the branch was recently taken or not.
 - Change bit on misprediction
 - Always mispredicts in first and last loop iterations.

Branch History Table or Branch Prediction Buffer:-> One Bit

Address	Branch Address	Target Address	Prediction
000	432	456	1
001	97	123	0
010	130	143	1
011	67	98	0
100	84	244	1
101	261	532	1
110	518	786	1
111	1031	1134	0



Ex:

000080 : add R1, R2, R3
 0000084: beq R1, R2, 244
 0000088: sub R1, R4, R5
 orr R2, R7, R8

PC=84

IF	ID	EXE	MEM	WB
----	----	-----	-----	----

PC=88 or 244

IF

....
 0000244: add R1, R4, R5

Branch History Table or Branch Prediction Buffer:-> One Bit

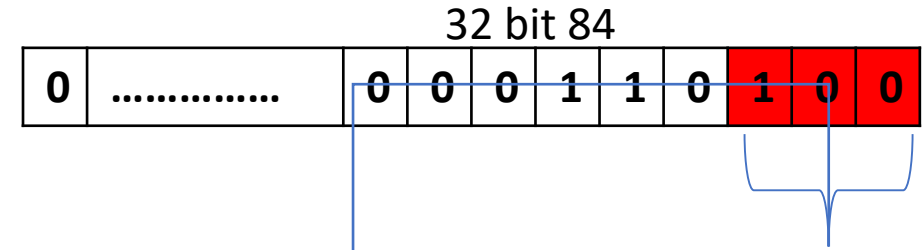
Address	Branch Address	Target Address	Prediction
000	432	456	1
001	97	123	0
010	130	143	1
011	67	98	0
100	84	244	1
101	261	532	1
110	518	786	1
111	1031	1134	0

PC=244

IF	ID	EXE	MEM	WB
----	----	-----	-----	----

IF	ID	EXE	MEM	WB
----	----	-----	-----	----

If Branch Taken,
update PC



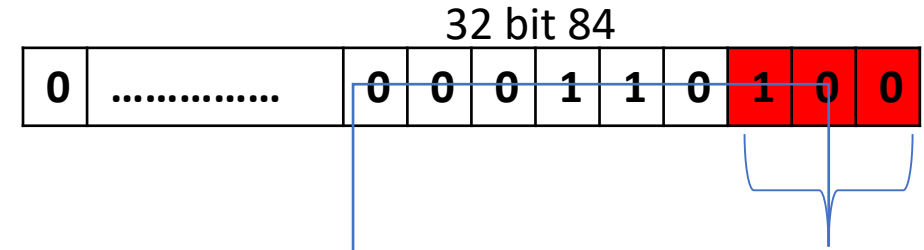
Ex:

000080 : add R1, R2, R3
000084: beq R1, R2, 244
000088: sub R1, R4, R5
 orr R2, R7, R8

....
0000244: add R1, R4, R5

Branch History Table or Branch Prediction Buffer:-> One Bit

Address	Branch Address	Target Address	Prediction
000	432	456	1
001	97	123	0
010	130	143	1
011	67	98	0
100	84	244	0
101	261	532	1
110	518	786	1
111	1031	1134	0



PC=88

IF	ID	EXE	MEM	WB
----	----	-----	-----	----

If Branch Not Taken
Continue with normal
execution

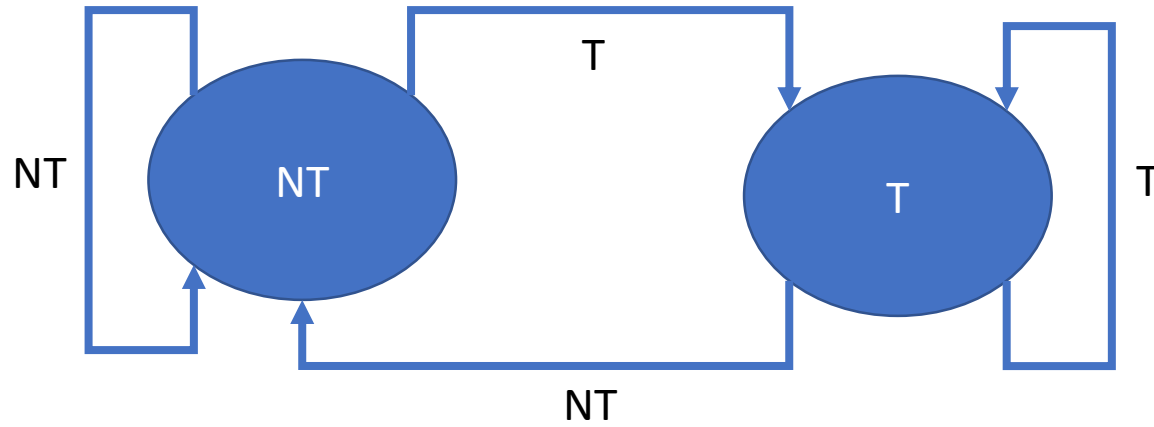
IF	ID	EXE	MEM	WB
----	----	-----	-----	----

Ex:

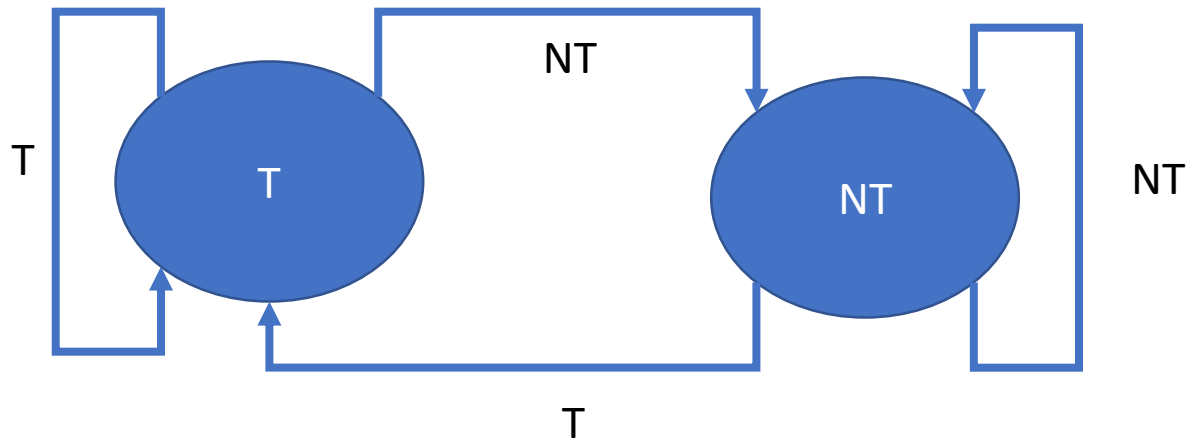
000080 : add R1, R2, R3
 000084: beq R1, R2, 244
 000088: sub R1, R4, R5
 orr R2, R7, R8

....
 0000244: add R1, R4, R5

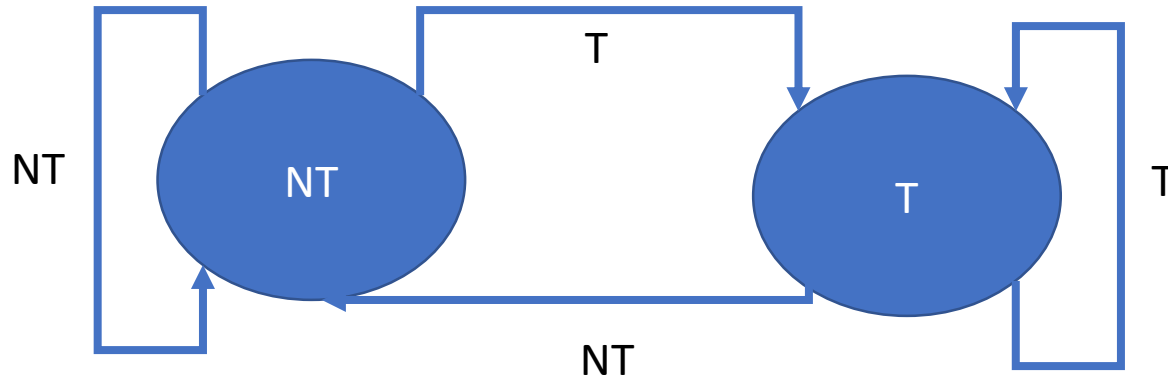
1 Bit Branch Prediction



OR



1 Bit Branch Prediction



Consider a program with two branch statement with the following behavior

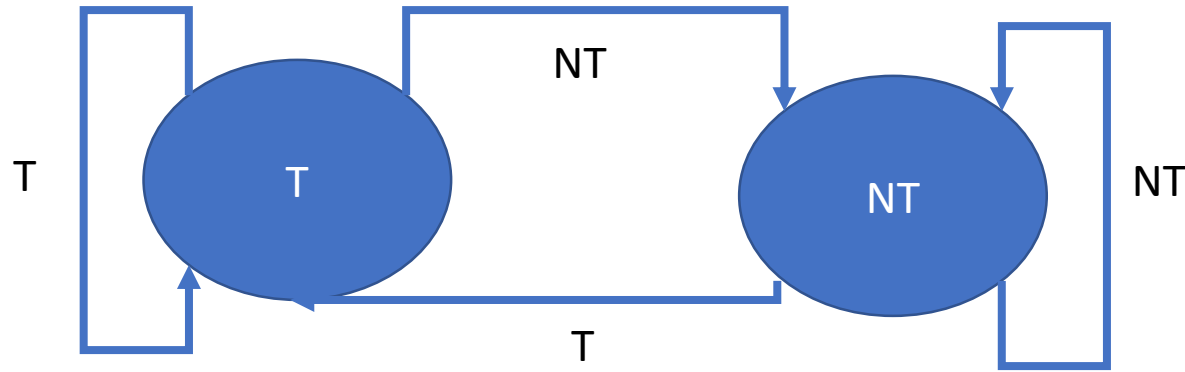
T T NT T NT T T T NT T T T T NT T T T T NT

How many Miss Prediction if the Initial state is NT?

10 miss predictions

T	T	NT	T	NT	T	T	T	NT	T	T	T	T	T	NT	T	T	T	T	NT
X	✓	X	X	X	X	✓	✓	X	X	✓	✓	✓	✓	X	X	✓	✓	✓	X

1 Bit Branch Prediction



Consider a program with two branch statement with the following behavior

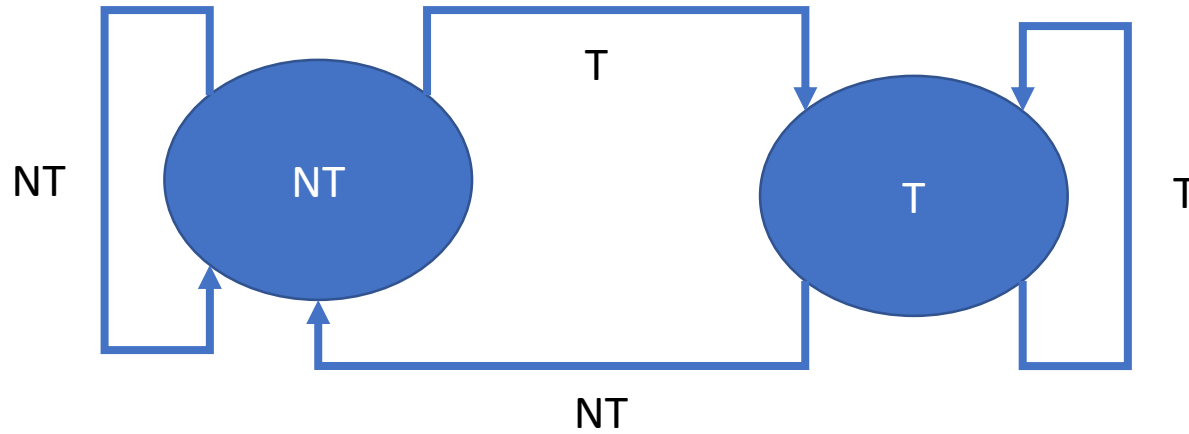
T T NT T NT T T T NT T T T T NT T T T T NT

How many Miss Prediction if the Initial state is T?

9 miss predictions

T	T	NT	T	NT	T	T	T	NT	T	T	T	T	T	NT	T	T	T	T	NT
✓	✓	X	X	X	X	✓	✓	X	X	✓	✓	✓	✓	X	X	✓	✓	✓	X

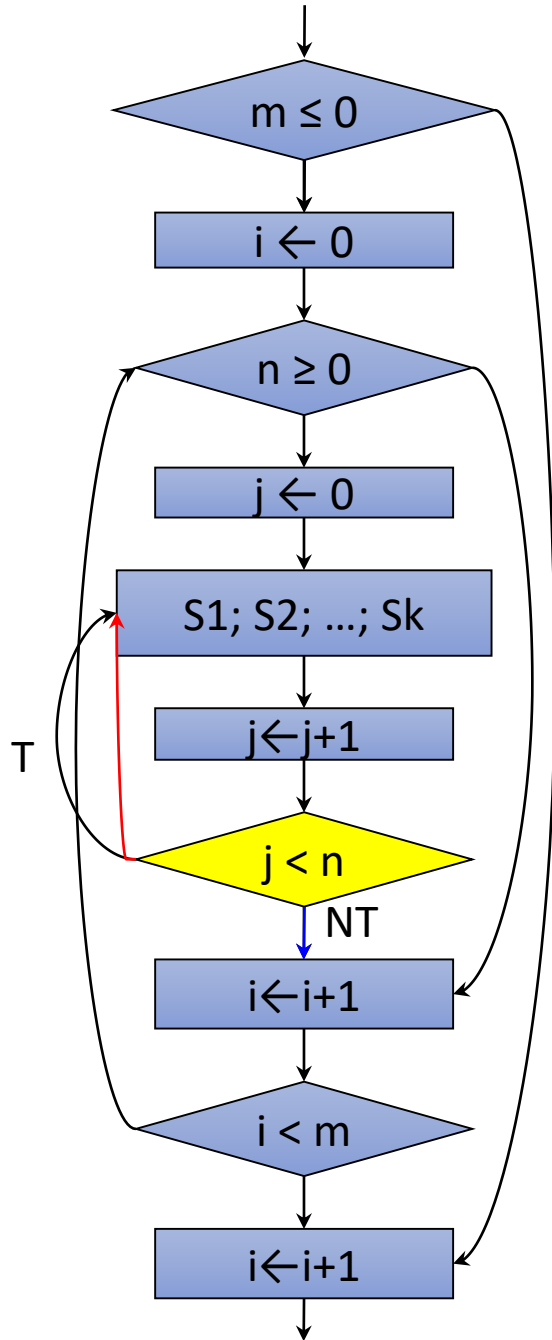
1 Bit Branch Prediction



```
for(j=0; j<n ; j++)
  begin S1; S2; ...; Sk end;
```

2 Missprediction for each loop

J	1-bit	
	Pred	Outc
0	NT	T
1	T	T
n	T	NT
0	NT	T
1	T	T



```

for(i=0 ; i < m ; i++)
  for(j=0; j<n ; j++)
    begin S1; S2; ...; Sk end;
  
```

		1-bit	
i	j	Pred	Outc
0	0	NT	T
0	1	T	T
0	n	T	NT
1	0	NT	T
1	1	T	T

$2 \times m$ misspredictions

2 Bit Branch Prediction

Towards 2-Bit Predictor

What is the problem with 1-Bit predictor?

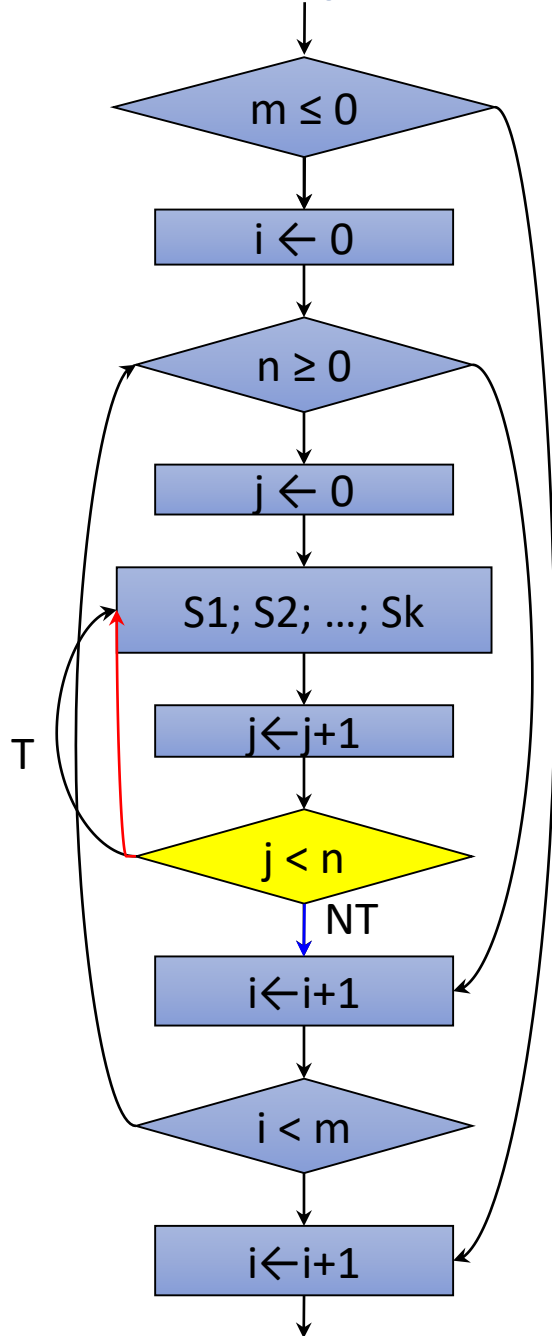
- **Aliasing Problem**

- branches with same lower order bits will reference the same entry, causing mutual prediction

- **Shortcomings with loops**

- Always mispredict twice for every loop
 - Mispredict upon exiting a loop, since this is a surprise
 - If we repeat the loop, we'll miss again since we'll predict, branch not taken

What is the problem with 1-Bit predictor



```

for(i=0 ; i < m ; i++)
  for(j=0; j<n ; j++)
    begin S1; S2; ...; Sk end;
  
```

		1-bit	
i	j	Pred	Outc
0	0	NT	T
0	1	T	T
0	n	T	NT
1	0	NT	T
1	1	T	T

If $m = 100$ & $n = 10$

2 miss prediction per Iteration

$2 \times m$ miss predictions for iterations

$2 \times 100 = 200$ miss predictions

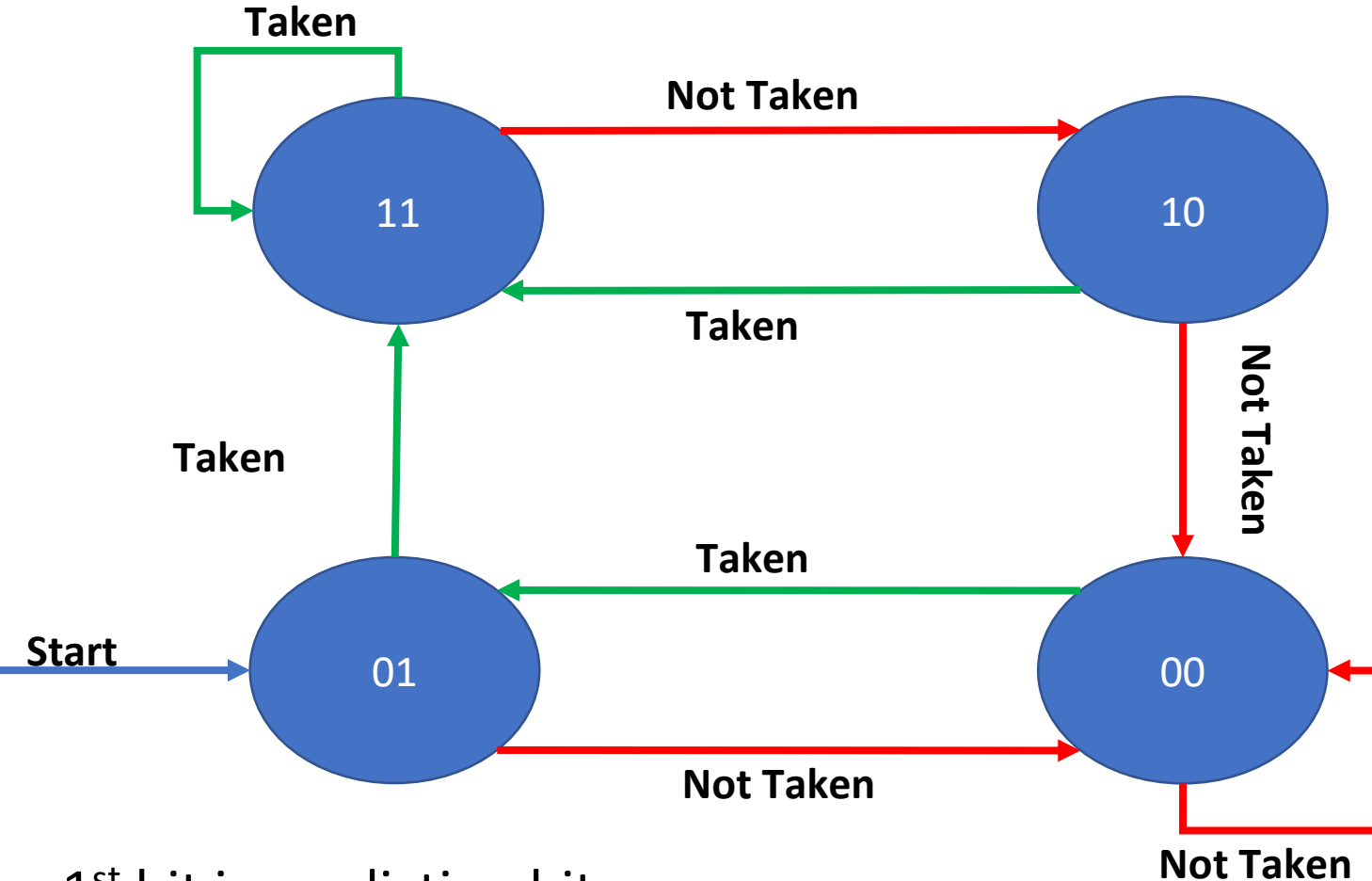
$8 \times n$ Correct Predictions (Single Iteration)

$8 \times 100 = 800$ Correct Predictions for $n \times m$ iterations

Accuracy of Correct Predictions = $800/1000 \times 100 = 80\%$

Two-bit Predictor

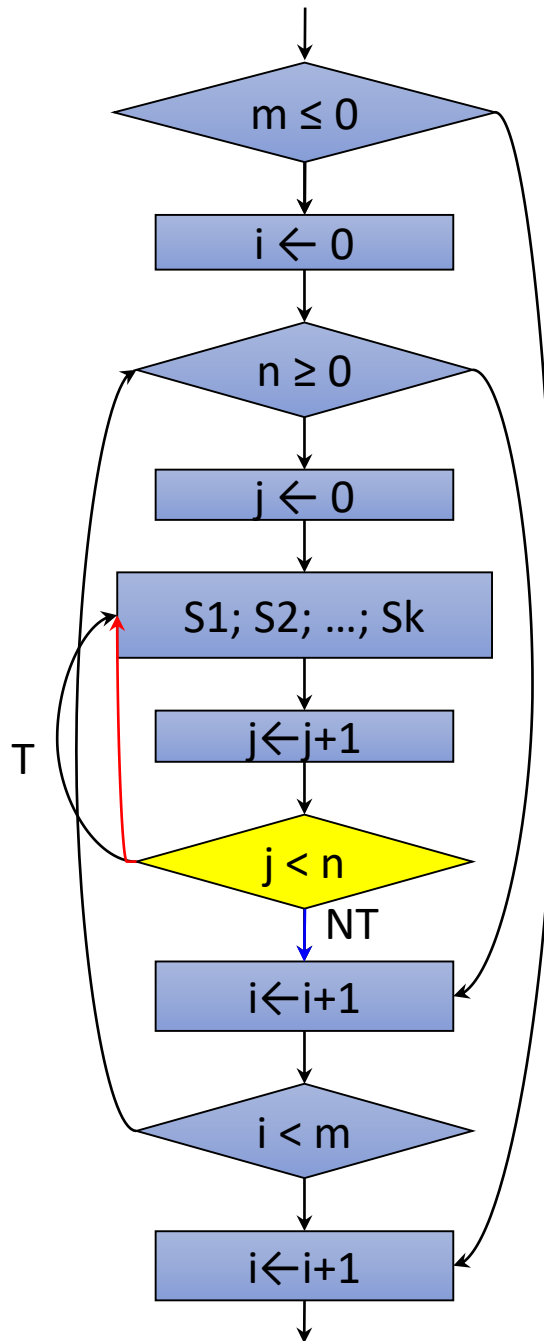
Think Twice Before Miss predicting



00: Strong Not Taken
01: Weak Not Taken
10: Weak Taken
11: Strong Taken

1st bit is prediction bit.

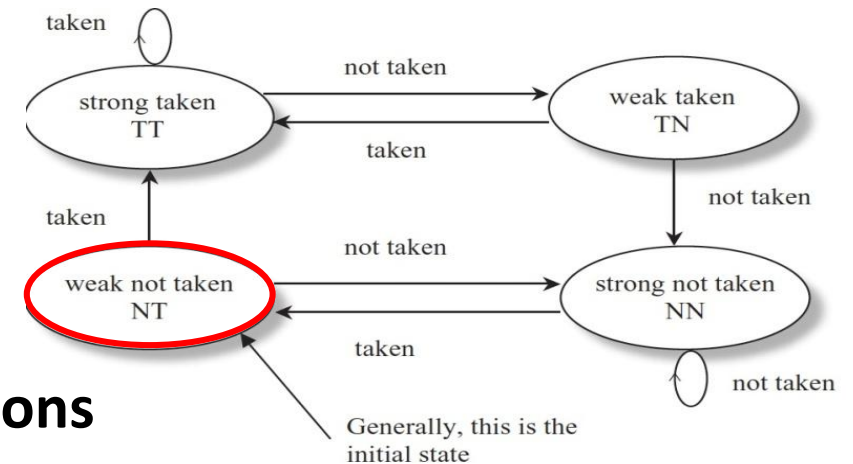
2nd bit is conviction bit (How much sure that the prediction is correct)



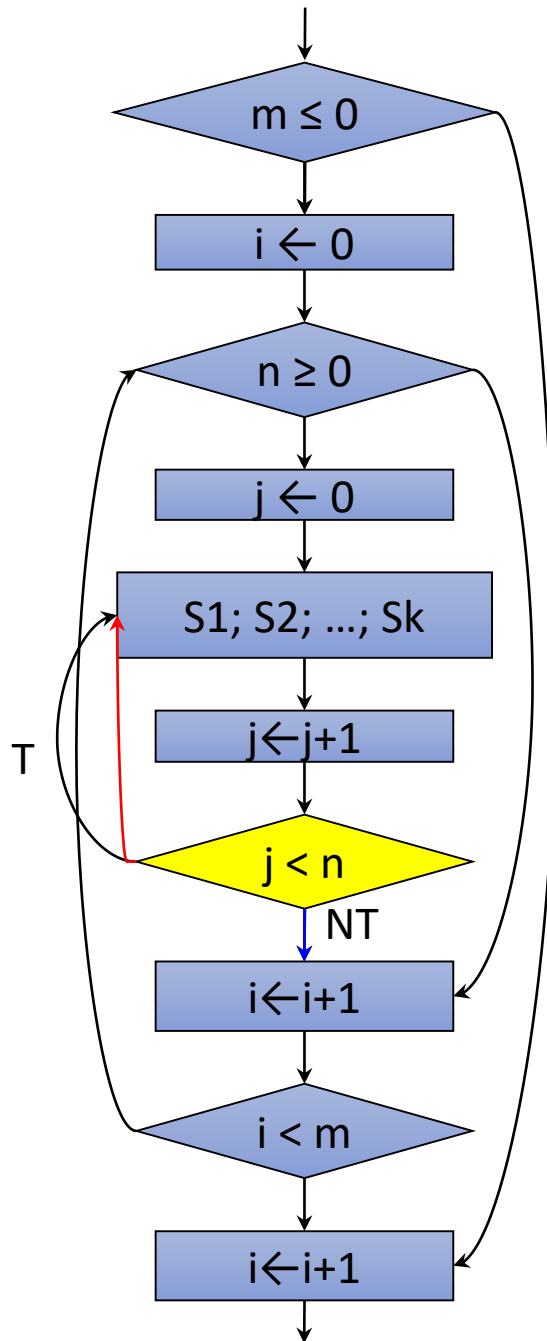
```

for(i=0 ; i < m ; i++)
  for(j=0; j<n ; j++)
    begin S1; S2; ...; Sk end;
  
```

		1-bit		2-bit		
i	j	Pred	Outc	State	Pred	Outc
0	0	NT	T	wNT	NT	T
0	1	T	T	sT	T	T
0	n	T	NT	sT	T	NT
1	0	NT	T	wT	T	T
1	1	T	T	sT	T	T



$m + 2$ miss-predictions



```

for(i=0 ; i < m ; i++)
  for(j=0; j<n ; j++)
    begin S1; S2; ...; Sk end;
  
```

$m + 2$ miss predictions

		1-bit		2-bit		
i	j	Pred	Outc	State	Pred	Outc
0	0	NT	T	wNT	NT	T
0	1	T	T	sT	T	T
0	n	T	NT	sT	T	NT
1	0	NT	T	wT	T	T
1	1	T	T	sT	T	T

If $m = 100$ & $n = 10$

1st iteration – Entry Exit Miss prediction

1 miss prediction per Iteration

Remaining iterations exit miss predictions

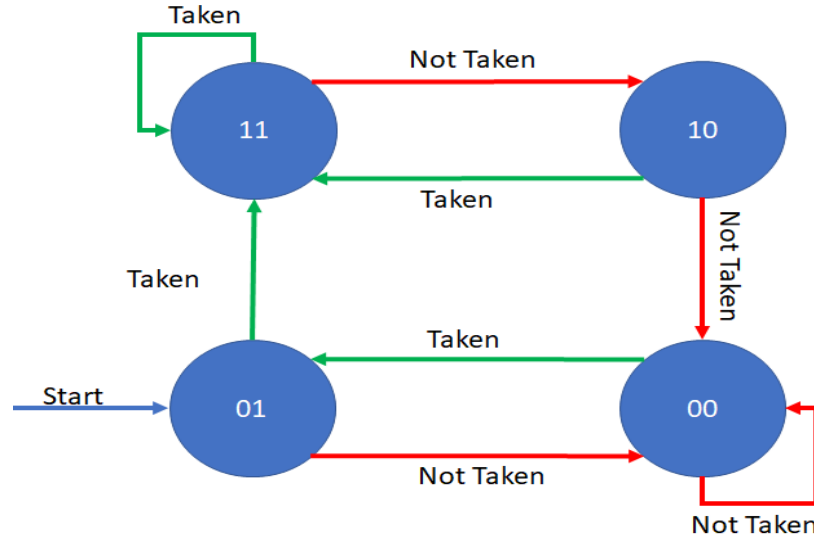
Last iteration entry miss prediction

$2 + 100 = 102$ miss predictions

$1000 - 102 = 898$ Correct Predictions for $n \times m$ iterations

Accuracy of Correct Predictions $\approx 898/1000 \times 100 \approx 90\%$

Two-bit Predictor: Initial state: 01



00: Strong Not Taken
01: Weak Not Taken
10: Weak Taken
11: Strong Taken

Consider a program with two branch statement with the following behavior

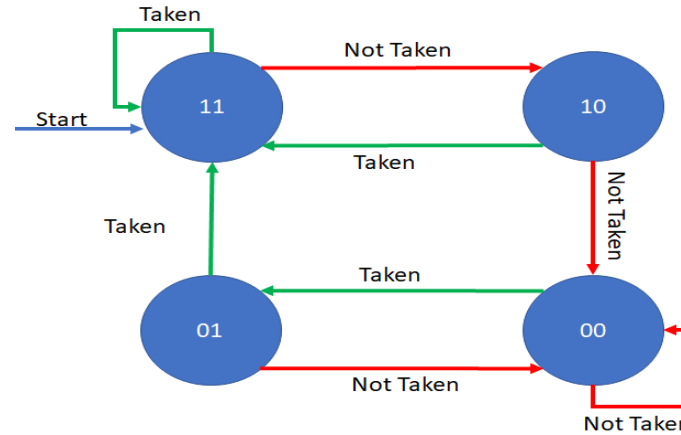
T T NT T NT T T T NT T T T T NT T T T T NT

6 miss predictions

How many Miss Prediction?

01	11	11	10	11	10	11	11	11	10	11	11	11	11	11	10	11	11	11	11	10
T	T	NT	T	NT	T	T	T	NT	T	T	T	T	T	NT	T	T	T	T	NT	
X	✓	X	✓	X	✓	✓	✓	X	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	X	

Two -bit Predictor: Initial state: 11



00: Strong Not Taken
 01: Weak Not Taken
 10: Weak Taken
 11: Strong Taken

Consider a program with two branch statement with the following behavior

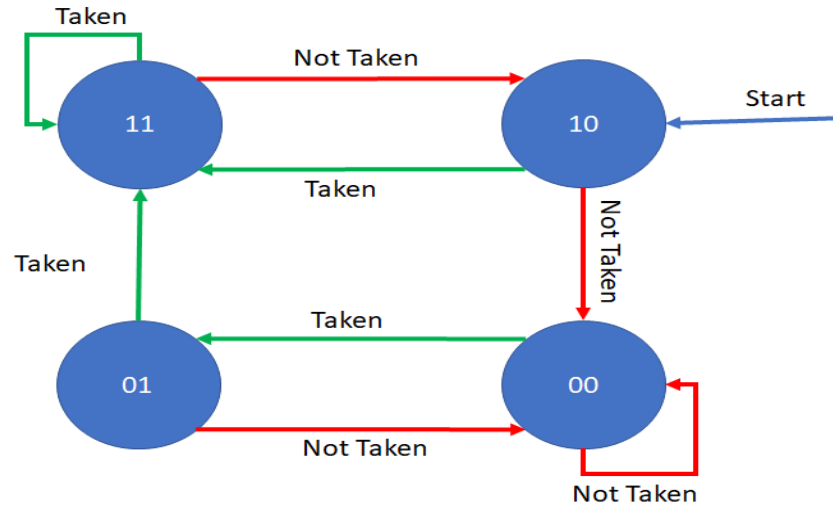
T T NT T NT T T NT T T T T NT T T T NT

5 miss predictions

How many Miss Prediction?

11	11	11	10	11	10	11	11	11	10	11	11	11	11	11	10	11	11	11	11	10
T	T	NT	T	NT	T	T	T	NT	T	T	T	T	T	NT	T	T	T	T	NT	
✓	✓	X	✓	X	✓	✓	✓	X	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	X	

Two -bit Predictor: Initial state: 10



00: Strong Not Taken
 01: Weak Not Taken
 10: Weak Taken
 11: Strong Taken

Consider a program with two branch statement with the following behavior

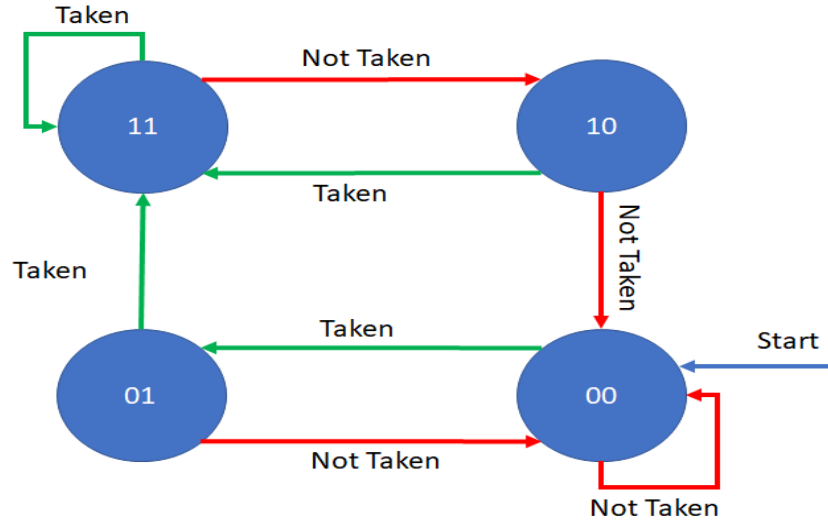
T T NT T NT T T NT T T T T NT T T T NT

5 miss predictions

How many Miss Prediction?

10	11	11	10	11	10	11	11	11	10	11	11	11	11	11	10	11	11	11	11	10
T	T	NT	T	NT	T	T	T	NT	T	T	T	T	T	NT	T	T	T	T	NT	
✓	✓	X	✓	X	✓	✓	✓	X	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	X	

Two-bit Predictor: Initial state: 00



00: Strong Not Taken
 01: Weak Not Taken
 10: Weak Taken
 11: Strong Taken

Consider a program with two branch statement with the following behavior

T T NT T NT T T NT T T T T NT T T T T NT

7 miss predictions

How many Miss Prediction?

00	01	11	10	11	10	11	11	11	10	11	11	11	11	11	10	11	11	11	11	10
T	T	NT	T	NT	T	T	T	NT	T	T	T	T	T	NT	T	T	T	T	NT	
X	X	X	✓	X	✓	✓	✓	X	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	X	

Think About It

Which predictor is best suitable if the following is the likely outcome of branch instruction? Suggest the suitable initial state

T NT T NT T NT T NT T TN

T T T T T T T T T T T T T T T

NT NT NT NT NT NT NT NT NT NT

1. Which of the following describes the "Alternative Static Prediction" strategy
 - A. All branches are predicted as "Taken" regardless of direction.
 - B. Backward branches are predicted as "Taken" and forward branches as "Not Taken"
 - C. It uses a 2-bit counter to track the history of the last two executions.
 - D. It relies on the compiler to profile the code and insert "likely taken" bits.

2. What is the primary shortcoming of a 1-bit branch predictor when handling loops?
 - A. It requires too much memory to store the history bits.
 - B. It always mis-predicts twice for every execution of a loop (once on entry, once on exit)
 - C. It cannot handle backward branches effectively.
 - D. It defaults to "Not Taken" which causes infinite loops.

1. Which of the following describes the "Alternative Static Prediction" strategy?

A. All branches are predicted as "Taken" regardless of direction.

B. Backward branches are predicted as "Taken" and forward branches as "Not Taken"

C. It uses a 2-bit counter to track the history of the last two executions.

D. It relies on the compiler to profile the code and insert "likely taken" bits.

2. What is the primary shortcoming of a 1-bit branch predictor when handling loops?

A. It requires too much memory to store the history bits.

B. It always mis-predicts twice for every execution of a loop (once on entry, once on exit)

C. It cannot handle backward branches effectively.

D. It defaults to "Not Taken" which causes infinite loops.

3. How does the hardware locate the prediction bit for a specific instruction in the Branch History Table (BHT)?
- A. By hashing the instruction's opcode.
 - B. By using the calculated target address of the branch.
 - C. By using the low-order address bits of the branch instruction.
 - D. By checking the ALU Zero flag result.
4. Consider a nested loop where the outer loop runs m times and the inner loop runs n times. Approximately how many mispredictions will a 2-bit predictor generate compared to a 1-bit predictor?
- A. The 2-bit predictor will have roughly half the mispredictions ($\sim m$ vs $\sim 2m$).
 - B. The 2-bit predictor will have zero mispredictions.
 - C. Both predictors will have the same number of mispredictions.
 - D. The 2-bit predictor will have double the mispredictions.

3. How does the hardware locate the prediction bit for a specific instruction in the Branch History Table (BHT)?

A. By hashing the instruction's opcode.

B. By using the calculated target address of the branch.

C . By using the low-order address bits of the branch instruction.

D. By checking the ALU Zero flag result.

4. Consider a nested loop where the outer loop runs m times and the inner loop runs n times. Approximately how many mispredictions will a 2-bit predictor generate compared to a 1-bit predictor?

A. The 2-bit predictor will have roughly half the mispredictions ($\sim m$ vs $\sim 2m$).

B. The 2-bit predictor will have zero mispredictions.

C. Both predictors will have the same number of mispredictions.

D. The 2-bit predictor will have double the mispredictions.



THANK YOU



Team MPCA

Department of Computer Science and
Engineering



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Session - 2.8

Microprocessor & Computer Architecture (μ pCA)



UE24CS251B

Pipeline Performance with Stalls - Exercises

Theoretical Claim – Quick Recap

$$\text{Time taken}_{\text{pipeline}} = \frac{\text{Time taken on unpipelined}}{\text{No.of pipeline stages}}$$

Pipelining – Quick Recap

■ 5-stage Pipeline

- Fetch
- Decode
- Execute
- Mem
- WB

■ n instructions, k-stages, t_c (cycle time)

■ Execution time_{pipeline} = $[k+(n-1)]t_c$

■ $Speedup(S) = \frac{\text{Execution Time on Unpipelined}}{\text{Execution Time on Pipelined}}$

■ $Efficiency(\mu) = \frac{Speedup}{k}$

- $n \ll k$
- $n == k$
- $n \gg k$

Time taken_{pipeline} = $\frac{\text{Time taken on unpipelined}}{\text{No.of pipeline stages}}$

Pipelining – Quick Recap

- n instructions, k -stages, t_c (cycle time)

- Execution time_{pipeline} = $[k+(n-1)]t_c$

- Execution time_{unpipeline} = nkt_c

- $Speedup(S) = \frac{\text{Execution Time on Unpipelined}}{\text{Execution Time on Pipelined}} = \frac{nkt}{(k+n-1)t} = \frac{nk}{k+n-1}$

- $Efficiency(\mu) = \frac{Speedup}{k}$

- $n \ll k$; $S = n$; $\mu < 1$

- $n == k$; $S = n/2$; $\mu = 0.5$

- $n \gg k$; $S = k$; $\mu = 1$ ✓

$$\text{Time taken}_{\text{pipeline}} = \frac{\text{Time taken on unpipelined}}{\text{No. of pipeline stages}}$$

Pipeline Performance with Stalls – Quick Recap

A stall causes the pipeline performance to degrade the ideal performance.

$$\text{Speedup from pipelining} = \frac{\text{Average instruction time unpipelined}}{\text{Average instruction time pipelined}}$$

$$= \frac{\text{CPI}_{\text{unpipelined}} * \text{Clock Cycle Time}_{\text{unpipelined}}}{\text{CPI}_{\text{pipelined}} * \text{Clock Cycle Time}_{\text{pipelined}}}$$

$$= \frac{\text{CPI}_{\text{unpipelined}}}{1 + \text{Pipeline stall cycles per instruction}}$$

Note 1: Ignoring the cycle time overhead of pipelining and assume the stages are all perfectly balanced, then the cycle time of the two machines are equal

Note 2: Ideal CPI of pipeline processor is almost always =1

$$\begin{aligned} \text{CPI}_{\text{pipelined}} &= \text{Ideal CPI} + \text{Pipeline stall clock cycles per instruction} \\ &= 1 + \text{Pipeline stall clock cycles per instruction} \end{aligned}$$

Pipeline Performance with Stalls – Quick Recap

If all instructions take the same number of cycles, which must also equal the number of pipeline stages (the depth of the pipeline) then unpipelined CPI is equal to the depth of the pipeline

$$\text{Speedup} = \frac{\text{Pipeline depth}}{1 + \text{Pipeline stall cycles per instruction}}$$

If there are no pipeline stalls, this leads to the intuitive result that pipelining can improve performance by the depth of pipeline.

Exercise-1

Consider an instruction pipeline with four stages with the stage delay 8 nsec respectively. Assume that the delay of an inter-stage register stage of the pipeline is Nil. What is the approximate speedup of the pipeline in the steady state under ideal conditions as compared to the corresponding non-pipelined implementation when 100 instructions are executed?

Solution:

Exercise-1

Consider an instruction pipeline with four stages with the stage delay 8 nsec respectively. Assume that the delay of an inter-stage register stage of the pipeline is Nil. What is the approximate speedup of the pipeline in the steady state under ideal conditions as compared to the corresponding non-pipelined implementation when 100 instructions are executed?

Solution:

Time taken to execute N instructions in non-pipelined implementation will be $K \times T_c \times N = 8 \times 4 \times 100 = 3200$.

Time taken for the pipelined implementation = $(K+N-1) \times T_c$
= $(4+100-1) \times 8$
= 824

Speedup = $3200 / 824 = 3.8$

Exercise- 2

Consider an instruction pipeline with four stages with the stage delays 5 nsec, 6 nsec, 11 nsec, and 8 nsec respectively. The delay of an inter-stage register stage of the pipeline is Nil. What is the approximate speedup of the pipeline in the steady state under ideal conditions as compared to the corresponding non-pipelined implementation when 100 instructions are executed?

Solution:

Exercise- 2

Consider an instruction pipeline with four stages with the stage delays 5 nsec, 6 nsec, 11 nsec, and 8 nsec respectively. The delay of an inter-stage register stage of the pipeline is Nil. What is the approximate speedup of the pipeline in the steady state under ideal conditions as compared to the corresponding non-pipelined implementation when 100 instructions are executed?

Solution:

Time taken to execute N instructions in non-pipelined implementation will be $(5 + 6 + 11 + 8)N = 30N = 30 \times 100 = 3000$.

Clock period for pipelined implementation = $\max\{5, 6, 11, 8\} = 11$.

Time taken for the pipelined implementation = $(K+N-1) \cdot T_c$
 $= (4+N-1) \cdot 11$
 $= (4 + 99) \cdot 11 = 1133$

Speedup = $3000 / 1133 = 2.6$

Exercise - 3

Consider an instruction pipeline with four stages with the stage delays 5 nsec, 6 nsec, 11 nsec, and 8 nsec respectively. The delay of an inter-stage register stage of the pipeline is 2. What is the approximate speedup of the pipeline in the steady state under ideal conditions as compared to the corresponding non-pipelined implementation when 100 instructions are executed?

Solution:

Exercise - 3

Consider an instruction pipeline with four stages with the stage delays 5 nsec, 6 nsec, 11 nsec, and 8 nsec respectively. The delay of an inter-stage register stage of the pipeline is 2. What is the approximate speedup of the pipeline in the steady state under ideal conditions as compared to the corresponding non-pipelined implementation when 100 instructions are executed?

Solution:

Time taken to execute N instructions in non-pipelined implementation will be $(5 + 6 + 11 + 8)N = 30N = 30 \times 100 = 3000$.

Clock period for pipelined implementation = $\max\{5, 6, 11, 8\} = 11$.

Clock period for pipelined implementation with register overhead = $11 + 2 = 13$

Time taken for the pipelined implementation = $(K + N - 1) * T_c$
 $= (4 + N - 1) * 13$
 $= (4 + 99) * 11 = 1339$

Speedup = $3000 / 1339 = 2.2$

Exercise - 4

Consider an instruction pipeline for the MIPS32 processor where data references constitute 42% of the instructions, and the ideal CPI ignoring memory structural hazards is 1.25. How much faster is the ideal machine without the memory structural hazard versus the machine with the hazard?

Solution:

$$\text{Speedup} = (\text{Ideal CPI} \times \text{Pipeline Depth}) / (\text{Ideal CPI} + \text{Stall cycles per instr})$$

$$\text{So, Speedup}_{\text{ideal}} = 1.25 \times K / (1.25 + 0) = K$$

$$\begin{aligned} \text{Speedup}_{\text{real}} &= 1.25 \times K / (1.25 + 0.42 \times 1) \\ &= 1.25 \times K / 1.67 \end{aligned}$$

$$\begin{aligned} \text{Required answer} &= K / (1.25 \times K / 1.67) \\ &= 1 / (1.25 / 1.67) \\ &= 1.67 / 1.25 \\ &= 1.34 \end{aligned}$$

Exercise - 5 : With Stalls

Consider the following constraints wrt the pipeline processor.

- a) Pipeline contains 5 stages: IF, ID, EX, M and WB
- b) Each stage requires one clock cycle;

```
// ADD TWO INTEGER ARRAYS
```

```
MOV R4, # 400
```

```
L1: LDR R1, [R4]
```

; Load first operand

```
LDR R2, [R4,#400]
```

; Load second operand

```
ADD R3, R1, R2
```

; Add operands

```
STR R3, [R4]
```

; Store result

```
SUB R4, R4, #4
```

; Calculate address of next element

```
BNEZ R4, L1
```

; Loop if (R4) != 0

Exercise - 5 : With Stalls – continued..

Calculate how many clock cycles will take execution of this segment on the regular (non-pipelined) architecture. Show calculations:

```
// ADD TWO INTEGER ARRAYS
    MOV R4, # 400
L1: LDR R1, [R4]
    LDR R2, [R4,#400]
    ADD R3, R1, R2
    STR R3, [R4]
    SUB R4, R4, #4
    BNEZ R4, L1
```

Solution

Number of cycles = [Initial instruction + (Number of instructions in the loop L1) x
number of loop cycles] x number of clock cycles
= [1 + (6) x 400/4] x 5 c.c. = 3005 c.c.

Exercise - 6

Calculate how many clock cycles will take execution of this segment on the simple pipeline without forwarding or bypassing when result of the branch instruction (new PC content) is available after WB stage. Show timing of one loop cycle in Table

Instruction	
L1: LDR R1, [R4]	
LDR R2, [R4,#400]	
ADD R3, R1, R2	
STR R3, [R4]	
SUB R4, R4, #4	
BNEZ R4, L1	
L1: LDR R1, [R4]	

Exercise - 6

Calculate how many clock cycles will take execution of this segment on the simple pipeline without forwarding or bypassing when result of the branch instruction (new PC content) is available after WB stage. Show timing of one loop cycle in Table

Solution:

Note: overlap of write back and decode is used. If overlap of WB and ID is not used for instruction 3 in ID phase stall continues till cycle 7

Instruction	Clock cycle number															
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
L1: LDR R1, [R4]	IF	ID	Ex	M	W											
LDR R2,[R4,#400]		IF	ID	Ex	M	W										
ADD R3, R1, R2			IF	ID	*	*	Ex	M	W							
STR R3, [R4]				IF	*	*	ID	*	*	Ex	M	W				
SUB R4, R4, #4							IF	*	*	ID	Ex	M	W			
BNEZ R4, L1								*	*	IF	D	*	*	EX	M	W

Comments

Instruction	Clock cycle number															
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
L1: LDR R1, [R4]	IF	ID	Ex	M	W											
LDR R2,[R4,#400]		IF	ID	Ex	M	W										
ADD R3, R1, R2			IF	ID	*	*	Ex	M	W							
STR R3, [R4]				IF	*	*	ID	*	*	Ex	M	W				
SUB R4, R4, #4							IF	*	*	ID	Ex	M	W			
BNEZ R4, L1								*	*	IF	D	*	*	EX	M	W

Number of cycles in the loop = 16 c.c.

Number of clock cycles for segment execution on pipelined processor =
 = 1 c.c. (IF stage of the initial instruction) + (Number of clock cycles in the loop L1) x Number of loop cycles

$$= 1 + 16 \times 400/4 = 1601 \text{ c.c.}$$

$$\text{Speedup} = 3005 \text{ c.c.} / 1601 = 2 \text{ times (approx.)}$$

1. A processor uses a 6-stage pipeline, with each stage taking 5 ns.

Inter-stage register overhead is negligible.

If 60 instructions are executed, what is the approximate speedup over a non-pipelined processor?

- A) 3.53
- B) 4.83
- C) 5.53
- D) 6.61

2. An instruction pipeline has 4 stages with delays 3 ns, 7 ns, 5 ns, and 6 ns.

The inter-stage register overhead is 1 ns.

If 80 instructions are executed, what is the approximate speedup compared to a non-pipelined processor?

- A) 2.1
- B) 2.5
- C) 3.0
- D) 3.6

MCQs

1. A processor uses a 6-stage pipeline, with each stage taking 5 ns.

Inter-stage register overhead is negligible.

If 60 instructions are executed, what is the approximate speedup over a non-pipelined processor?

- A) 3.53
- B) 4.83
- C) 5.53
- D) 6.61

ANS- C) 5.53

2. An instruction pipeline has 4 stages with delays 3 ns, 7 ns, 5 ns, and 6 ns.

The inter-stage register overhead is 1 ns.

If 80 instructions are executed, what is the approximate speedup compared to a non-pipelined processor?

- A) 2.1
- B) 2.5
- C) 3.0
- D) 3.6

ANS- B) 2.5

3. A pipelined processor has- Pipeline depth = 5, Ideal CPI = 1, Average stall cycles per instruction = 0.4

What is the speedup of this pipelined processor over a non-pipelined processor?

- A) 3.6
- B) 3.1
- C) 2.5
- D) 4.0

4. A pipelined processor has- Ideal CPI (ignoring memory hazards) = 1.3, 35% of instructions access memory, Each memory access causes 1 stall cycle

How much faster is the ideal processor compared to the processor with memory hazards?

- A) 1.15
- B) 1.22
- C) 1.33
- D) 1.27

3. A pipelined processor has- Pipeline depth = 5, Ideal CPI = 1, Average stall cycles per instruction = 0.4

What is the speedup of this pipelined processor over a non-pipelined processor?

- A) 3.6
- B) 3.1
- C) 2.5
- D) 4.0

ANS- A) 3.6

4. A pipelined processor has- Ideal CPI (ignoring memory hazards) = 1.3, 35% of instructions access memory, Each memory access causes 1 stall cycle

How much faster is the ideal processor compared to the processor with memory hazards?

- A) 1.15
- B) 1.22
- C) 1.33
- D) 1.27

ANS- D) 1.27



THANK YOU



Team MPCA

Department of Computer Science and
Engineering